パワー ユーザー ガイド

ProDG for Nintendo GameCube™ デバッガー

SN Systems Ltd
バージョン **3.9.3**
2003 年 **5** 月

著作権© 2001-2003 SN Systems Ltd. 無断転載を禁ず

"ProDG" および SN ロゴは、SN Systems Ltd の商標です。"GameCube"は、Nintendo Co. Ltd の商標です。"Microsoft"、"Visual Studio"、"Win32"、"Windows"、"Windows NT"は、Microsoft Corporation の登録商標です。この文書で使用される商品名または会社名は、それぞれの所有者の商標です。

NINTENDO GAMECUBE™

NINTENDO GAMECUBE は、任天堂株式会社の登録商標です。
Windows、WindowsNT は米国 Microsoft Corporation の米国およびその他の国における登録商標または商標です。製品および内容物の外見、形状、仕様は、予告なしに変更される場合があります。

ProDGNGC_PU(DBG)-J / パワーユーザガイド ProDG for Nintendo GameCube デバッガ / 2003 年 5 月

Document change control

Ver.	Date	Changes
3.9.3	2003 年 5 月	ProDG for Nintendo GameCube v3.9.3 のリリース。v3.9.2 からの変更: ペインの記述を含む新しい章のレイアウトを関係する章に移動;新しいミックスモードペイン;新しいデバッガ・スクリプティングとスクリプティング API の章;新しいデバッガ・スイッチ-vs7 および -w<optional flags>;数多くのデバッガ・ ペインとショートカットメニューの変更;ハードウェアブレークポイントに関する情報の追加;カーネルペインの変更;デフォルトメモリマップ無効についての追加セクション; ワークスペースペインのセクションを改訂;ディスプレイリストペインに関する章を追加。

目次

第 1 章: はじめに 1

概要 1
ProDG Plus 1
テクニカル サポートとアップグレード 2
Web を経由した ProDG のアップデート 2
ProDG デバッガおよび Target Manager をアップデートする 3

第 2 章: ユーザインタフェース 5

デバッガの概要 5
デバッガを起動する 5
ngcdbg のコマンドライン書式 6
ngcdbg のコマンドラインの具体例 8
メニューオプション 8
ファイルメニュー 8
デバッグメニュー 9
設定メニュー 9
ウィンドウメニュー 10
ウィンドウとペイン 10
新しいデバッガウィンドウを作成する 11
ペインのショートカットメニュー 13
ペイン別ビューを作成する 14
デバッガのペイン更新 16
ターゲットとプロセッサステータス 17
テキスト選択およびクリップボード 17
マウスで選択する 17
キーボードで選択する（ソースペインのみ） 18
使用の際の注意事項 18
数式 19
関数ブラウザを使用する 19
名称の再構築 20
数式アドレスを入力する 20
ソースペインでのホバー評価 20
名称の補完 21
数式をビルドする 21
ペイン間でデータを移動する 23
ドラッグ＆ドロップとソースペイン 24
ペイン修飾キー 25
ユーザインタフェースを設定する 25
ペインの色とフォント 26

プロジェクト設定を保存する 27
アクセラレータキー 29
キーボードショートカットリファレンス 30
ProDG デバッガショートカットキー 31

第 3 章: ゲームのビルドおよびロード 35

デバッグ用ビルド 35
デフォルトメモリマップの無効 35
ターゲットへの接続 36
GameCube でデバッグ中の複数ユーザ 37
ELF ファイルのロード、実行 38
Nintendo GameCube 上でゲームを実行 39
ワークスペースペインを使用する 40
ワークスペースペインショートカットメニュー 42
オプション 44

第 4 章: ブレークポイントおよびステッピング 47

ブレークポイントの使用 47
コンパイル時のブレークポイントの設定 47
ブレークポイントの設定閲覧 48
プログラムを 1 ステップずつ実行する 48
メモリアクセス、またはハードウェアブレークポイント 49
条件付きブレークポイント 51
ブレークポイントペイン 52
ブレークポイントペインショートカットメニュー 53
ブレークポイントプロパティダイアログ 54

第 5 章: ソースおよび逆アセンブリを閲覧する 55

ソースを閲覧する 55
ソースペイン 55
ソースペインショートカットメニュー 57
ソース内の検索 59
ソースコードの編集 61
正規表現 62
ソースファイルの配置 63
ポップアップヘルプ を使って数式を評価するには 64
逆アセンブリを閲覧する 64
逆アセンブリペイン 65
逆アセンブリペインショートカットメニュー 66
ソースと逆アセンブリを同時に閲覧する 68
ミックスモードペインショートカットメニュー 69

第 6 章: メモリとレジスタを閲覧・変更する 73

メモリを閲覧変更する 73
メモリペイン 73

メモリペインショートカットメニュー 75
ターゲットメモリを保存・ダウンロードする 76
レジスタを閲覧する 78
レジスタペイン 78
レジスタペインショートカットメニュー 79
テンプレート化レジスタビュー 80

第 7 章: ローカル変数とウォッチ変数を閲覧する 81

ローカル変数を閲覧する 81
ローカルペイン 81
ローカルペインショートカットメニュー 82
変数のモニタリング 82
ウォッチペイン 82
ウォッチペインショートカットメニュー 83

第 8 章: コールスタックとスレッド情報を閲覧する 87

コールスタックを閲覧する 87
コールスタックペイン 87
コールスタックペインショートカットメニュー 88
スレッド情報を閲覧する 88
カーネルペインショートカットメニュー 89

第 9 章: TTY 出力を閲覧する 91

TTY 出力を閲覧する 91
TTY ペイン 91
TTY ペインショートカットメニュー 92

第 10 章: 基本 Gekko プロファイリング 95

Gekko プロファイリング概要 95
プロファイルペイン 95
プロファイルペインショートカットメニュー 96
留意すべき点 97
特定の状況下でのターゲットプロファイリング 97

第 11 章: GX ディスプレイリストを閲覧する 99

ディスプレイリストを閲覧する 99
ディスプレイリストペイン 99
キーボードショートカット 100
ディスプレイリストエラーを検出する 100
ディスプレイリストペインのショートカットメニュー 101
トラブルシューティング 101

第 12 章: デバッガー スクリプト 103

デバッガー スクリプトとは 103
インストールの注意 103
トラブルシューティング 104
スクリプトの使用方法 104
スクリプト ペイン 105
スクリプト ペインのショートカット メニュー 105
イミーディエイトモードのステートメント 106
スクリプトの読み込みと実行 106
autoexec.eic を使用する 106
ngcdbg コマンドラインから 107
ショートカットメニューからスクリプトダイアログを実行する 107
デバッガーの起動と同時にスクリプトを読み込む 107
使用可能なライブラリ関数 108
標準 C ライブラリ関数 108
SN Systems 関数 108
スクリプト サンプル 109
GDI 関数 110
タイマーコールバックの設定 111
数式の評価 111
ターゲット変数に新しい値を割り当てる 112
各種タイプ用のカスタム表示テンプレートの設定 113
テンプレート文字列の仕様 113
ショートカット メニュー項目の追加 114
スクリプティングの制限 115

付表:デバッガスクリプティング API 117

SNSetCallback 117
SNHookMessage 117
SNUnhookMessage 118
SNSetTimer 119
SNBindKey 119
SNSetstdout 120
SNTxtSetPos 120
SNTxtSetConsole 121
SNTxtSetTTY 121
SNTxtHome 122
SNTxtClrEol 122
SNTxtClrEop 123
SNEvaluate 123
SNGetMemory 123
SNSetMemory 124
SNLoadElf 124
SNLoadBin 125
SNSaveBin 125
SNStart 126
SNStop 126
SNSetBP 126

SNClrBP 126
SNIsRunning....127
SNUpdateWindows 127
SNRefreshAll....128
SNSetTypeDisplay 128
SNGetTypeDisplay 129
SNDelTypeHandlers 129
getcwd....130
chdir 130
ShellExecute....130
SNAddMenu....131
SNGetMenuInfo....132
SNGetPaneInfo 132
SNGetPaneTitle 133
SNStep 133
SNStepOver....134
SNAddr2Line....134
SNLine2Addr....134
SNAddr2Label 135
SNRun2Addr....135
SNExit 136
SNGetWindowSize 136
SNCreateBmp 136
SNSetGraphMode 137
SNClearBmp 138
SNSetColor....138
SNSetPixel 139
SNSetLine 139
SNRectangle 140
SNEllipse....140
SNPolygon....141
SNTextOut 141
SNSetFont....142
SNEnumFonts 142

インデックス **143**

［このページは意図的に空白にされています］

第 1 章: はじめに

概要

ProDG は、ニンテンドーゲームキューブ をコンソールとするゲームのビルドやデバッグを行なうための、SN Systems 製フル装備 Win32 ツール群です。ProDG ツールを使用すれば、Win32 PC 上ですぐにアプリケーションをビルドし、ニンテンドーゲームキューブの開発ハードウェアにアップロードして、実行/デバッグできます。

ProDG の構成は下記のとおりです:

- **ProDG Build Tools**: Microsoft Visual Studio インテグレーションオプション; ProDG コンパイラドライバ; ProDG リンカ; SN Systems ユーティリティとライブラリ; ProDG アセンブラ. 「*Power User's Guide to ProDG for Nintendo GameCube Build Tools*」参照。

- **ProDG Target Manager** ー ネットワークの GameCube ターゲットへの接続をコントロールすることができます。「 *User Guide to ProDG Target Manager for Nintendo GameCube*」参照。

- **ProDG Debugger** は、GameCube アプリケーションのデバッグに必要 な完璧な機能を備えた Win32 デバッガです。

本マニュアルは、 ProDG デバッガと ProDG Plus の高性能デバッガについてカバー しています。

ProDG Plus

ProDG Plus は、GameCube ゲーム開発/デバッグ用の高性能ツール パッケージです。

- **ProDG** ー このパッケージには、C/C++ コンパイラ、アセンブラ、リンカ、デバッガ、ターゲット マネージャが含まれます。またオプションの Visual Studio 統合機能 には、実行ファイルやライブラリのビルド、および Visual Studio 内から ProDG Debugger を起動するための App-Wizards が含まれます。

- **ProDG Debugger** の高度な機能 ー デバッガのスクリプト機能や、強化されたその他の機能により、ProDG Plus ユーザーにとって GameCube デバッガの新しい標準となります。

- **Tuner** ー プログラムの動作をキャプチャして視覚化することにより、コードにおけるコンフリクトや問題の解決を支援します。このツールにより、ハイパフォーマンスなゲームを「手探り」なしで、プログラムすることが可能になります。またホスト PC へのデータ キャプチャは、ゲームのプレイと同時にリアルタイムで行われます。

キャプチャしたデータは、フレーム単位で分析することが可能で、保存したデータと最適化したコードと比較することもできます。

テクニカル サポートとアップグレード

当社 Web サイトのサポート エリアには、SN Systems の全製品に関する各種の最新サポートが用意されています。このページには、SN Systems のユーザー 名 とパスワードを所有する登録ユーザーのみがアクセスできます。

- ユーザー 名 やパスワードを忘れた場合は、電子メールで **webmaster@snsys.com** までご連絡ください。

www.snsys.com/support (英語)
www.snsys.jp/support (日本語)

サポート エリアで解決法を見つけられない場合は、当社サポート チーム (下記アドレス) まで電子メールでご連絡ください。

support@snsys.com (英語)
j-support@snsys.com (日本語)

ご連絡の際は、発生している問題を明記し、ご使用のソフトウェア バージョンおよびハードウェア環境の詳細も記載してください。SN Systems サポートのログ番号 (LN 番号) をお持ちの場合は、問題に関するすべての電子メールにその番号を記載してください。

SN Systems Limited
4th Floor - Redcliff Quay
120 Redcliff Street
Bristol BS1 6HU
United Kingdom

電話:+44 (0)117 929 9733
電話:+44 (0)117 929 9251

URL:**www.snsys.com** (英語)
www.snsys.jp (日本語)

Web を経由した ProDG のアップデート

ProDG のアップデートは、SN Systems の Web サイトに定期的にアップロードされます (2 ページの「テクニカル サポートとアップグレード」を参照)。ダウンロードのページにアクセスするには、[テクニカル サポート]、[ダウンロード] を選択します。 このページにアクセスするには、SN Systems のユーザー 名 とパスワードが必要です。

- ProDG の製品に関する全面的なアップデートは、[**フルインストール**] のセクションに表示されます。

- 十分テストされたコンポーネントに関するアップデートは、[**プロダクトアップグレード**] のセクションに表示されます。

- Beta バージョンのコンポーネントに関しては、[**プリ・リリース**] のセクションに表示されることがあります。

ProDG デバッガおよび Target Manager をアップデートする

ProDG デバッガおよび Target Manager の.zip アーカイヴをダウンロードしたら、前回ファイルをインストールしたシステムの同じ場所に配置して下さい。デフォルト位置は C:\Program Files\ProDG for Nintendo GameCube です。

プログラムをその他の位置に配置したい場合は、コマンドに可視性を与えるようパスを設定して下さい。その際、autoexec.bat ファイルでもしくは Windows 2000 または NT の時はコントロールパネルの**システム**アイコンの**アドバンス**タブで ProDG デバッガおよび ProDG Target Manager 実行可能ファイルに PATH=環境変数パスを追加して下さい。

例:

```
SET PATH=%PATH%;"C:\NGC\SN Programs"
```

グローバルにこの変更を適用したい場合は、マシンを再起動する必要があります。これで MS-DOS プロンプトでどのディレクトリでもアプリケーションコマンド(ngcdbg もしくは ngctm)をタイプして、関連アプリケーションが起動するようになります。

[このページは意図的に空白にされています]

第 2 章: ユーザインタフェース

デバッガの概要

Nintendo GameCube 用 ProDG デバッガは、GameCube コンソール用スタンドアロンのソースレベルデバッガです。

本デバッガは GameCube 専用にデザイン、ビルドされ、GameCube に移植された GameCube デバッガに留まりません。また、複数の GameCube プロセッサで実行中のアプリケーションをロード、実行、デバッグ可能にします。

このデバッガの主な特徴は以下の通りです。

- CPU レジスタ（64 ビットフローとやペアードシングル）、メモリ、ディスアセンブリ、ソース、ローカル変数、ウォッチポイントなど、異なるタイプのデバッガペインをいくつでも作成できる機能。
- Gekko プロセッサからの printf ストリームを表示する TTY ペイン.
- メイン CPU のソースレベルデバッグが、無数のソフトウェアブレークポイント、ハードウェアブレークポイント、シングルステップ、ステップオーバー、カーソルまで実行など、直接ソースコードに提供
- ソースコード検索パスにより、誰がビルドしたかに関わらず、ソースコードがある全プログラムのソースレベルのデバッグを実行可能.
- 業界標準である ELF ファイル形式を DWARF2 デバッグ情報と共に使用
- Microsoft Visual Studio との統合
- 対象情報の迅速な更新と表示.
- カスタム設定可能なデバッガ画面の配置を保存・復旧可能.
- 機能追加を含むアップデートが、ユーザの要求に応じて定期的にウェブサイトにポストされる.

デバッガを起動する

ProDG デバッガは、コマンドライン、または Windows のショートカットから起動します。Windows のショートカットを使用して起動する場合は、ショートカットのプロパティで、必要

に応じてデバッガへのコマンドラインパラメータと、デバッガの実行ディレクトリを設定する必要があります。

ProDG for Nintendo GameCube 2 の Microsoft Visual Studio 統合機能をインストールしている場合は、ツールバーボタンを使用して、Microsoft Visual Studio からの ProDG デバッガ起動も可能です。

デバッガが起動した際、既に起動している場合は ProDG Target Manager と共同作業を開始、Target Manager が起動していない場合はデバッガが自動的に起動（ただし ngctm.exe 実行可能ファイルがディレクトリパスにストアされている場合のみ）

> **注:** ProDG デバッガは、ProDG Target Manager で最低一つターゲットが設定されていないと、起動しません。その際は ProDG Target Manager を起動し、ターゲットを追加します。

なお、Windows ショートカットに実行ディレクトリを必ず設定して下さい。この設定を怠ると、そのときに使用しているディレクトリによっては、デバッガ設定ファイルが別の領域に保存されてしまうためです。デバッガの設定に関する詳細は、25ページの「ユーザインタフェースを設定する」をご覧下さい。

まず SN Systems ビルドツールでコンパイルされた ELF ファイルをロード・デバッグしないと、デバッグ作業を開始できませんので、ご注意下さい。

ngcdbg のコマンドライン書式

Nintendo GameCube 用 ProDG デバッガプログラムは ngcdbg.exe です。ngcdbg のコマンドライン書式は、以下の通りです。

```
ngcdbg <switches> <file.elf> <app_params>
```

<switches>	デバッガのオプションを指定する省略可能なスイッチ。
<file.elf>	ロードする ELF アプリケーションファイル名を指定できる。このファイルをコマンドラインで指定しない場合は、後で ELF ファイルをメニューからロードする必要がある。 **注:** ファイル名にスペースが含まれている場合は、ファイル名を二重引用符で囲む必要があります。
<app_params>	main()に引き渡される argc パラメータと argv[]パラメータ。

ELF を指定したにもかかわらず、ターゲットにダウンロードしなかった場合は、シンボルのみがロードされます。このモードは、破壊されたターゲットの事後デバッグを行なう場合に適しています。

有効なスイッチは以下の通りです。

-b	は、デバッガの実行中にブレークポイントが設定ファイルで存続しないよう指定します。
-c	SAMBA および Linux を適用している開発者のために、デバッガが大文字／小文字を区別しないようにします。

-d	は、ファイルをロードすると同時にデバッガが実行する「メインまで自動実行」動作が行なわれないようにします。このスイッチを設定すると、ターゲットでは、作成したアプリケーションを手動で起動 ([スタート]のツールバーボタン、または F9 の**デバッグ → 実行**)するまで、そのアプリケーションは起動されません。
-da	は、デバッガの実行中、常にターゲットを接続解除した状態にします。この設定は、デバッガが起動した時の状態でターゲットから接続解除するというデバッガのデフォルト設定に優先します。
-dv "<dvdroot>"	は、DVD のルートを設定します(必ず、二重引用符で囲んでください)。
-e	は、作成した ELF ファイルに含まれるアプリケーション実行ファイル、およびシンボルをロードします。
-f	ターゲットが接続しているファイルサーバのルートディレクトリを、<file.elf> 引数によってロードされた ELF ファイルのディレクトリに再設定します。
-m	複数のデバッガコピーの実行を可能に(二つの異なる GameCube で実行中のコードをデバッグするため、デバッガのふたつのインスタンスを実行可能に)。このスイッチを設定しない場合、次にデバッガを起動すると、現在実行しているバージョンが最初に実行されます。
-nd	は、デバッガの実行中、ターゲットから接続を解除しないようにします。この設定は、デバッガが起動した時の状態でターゲットから接続解除するというデバッガのデフォルト設定より優先します。
-r	は、実行可能ファイルをダウンロードする前に GameCube をリセットします。
-ra"<args>"	は、実行ユーティリティの引数を特定します(必ず、二重引用符で囲んでください)。ターゲットメーカのマニュアルで利用可能な引数を確認してください。
-s	は、シンボルを確実にロードします。デバッガのバージョン 1.19 以来、大容量の.elf を伴うシンボルのダウンロードが高速化。これは、各々のモジュールにある同じ名称のタイプが全て同じであるとデバッガが仮定するよう、デフォルトで設定されているためです。しかし、同じ名称の異なるモジュールに異なるタイプがあり、デバッガにそれを適切に処理させたい場合は、このスイッチを指定する必要があります。
-S<filename>	デバッガが起動時に使用するスクリプトファイルに名前を付けます。103ページの「デバッガー スクリプトとは」参照。
-t<name>	指定の名称で接続するターゲットを指定します。Target Manager のターゲットリストに該当する名称がない場合、または–t オプションを空白にした場合は、デバッガ起動時に使用可能なターゲットから選択するようダイアログが現れます。ターゲット名にスペースがある場合は、コマンドライン上で二重括弧で括る必要があります。
-vs	は、デバッグセッションの開始・終了時点に、Microsoft Visual Studio のブレークポイントをインポート・エクスポートするなど、

	Microsoft Visual Studio の互換機能を有効にします。
-vs7	–vs と同じ。ただし、Visual Studio .NET に使用されます。
-x	コードは、ターゲットのロード後にその対象上で実行されます。

ngcdbg のコマンドラインの具体例

コマンドラインの一例を以下に示します。

```
ngcdbg -r -e main.elf
```

このコマンドラインは、デバッガを起動し、main.elf というファイルからシンボルをロードして、ゲームキューブをリセットした後、main.elf をロードし、プログラムを main()まで自動実行します。

もう一つ例をあげてみましょう。

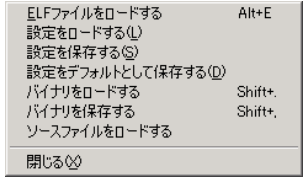
```
ngcdbg -rex main.elf param1 param2
```

このコマンドラインは、デバッガを起動し、main.elf からシンボルをロードして、ターゲットのゲームキューブをリセットしてから、main.elf から実行コードをロードし、そのコードを実行して、main()にパラメータ argc=3、argv[0]="main.elf"、argv[1]="param1"、and argv[2]="param2"を引き渡します。

メニューオプション

ProDG デバッガは、**ファイル、デバッグ、設定、ウィンドウ**というメニューオプションを備えています。以下は、これらのオプションの説明です。

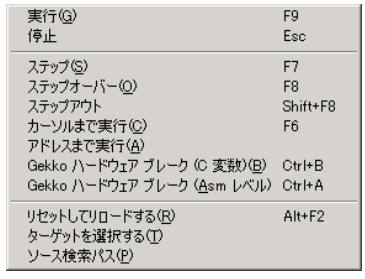
ファイルメニュー



ELF ファイルをロードする	GameCube プロセッサに ELF ファイルをロードする。ファイル名の選択とファイルロードオプションの設定ができるダイアログが表示される。.
設定をロードする	現在の実行ディレクトリから、デバッガ設定ファイルを再ロードし、現在の設定を上書きする。
設定を保存する	現在のデバッガ設定を保存する。現在の実行ディレクトリの dbugngc.ngc に自動的に保存される。
設定をデフォルトとして保存する	現在のデバッガ設定を、現在の実行ディレクトリで dbugngc.ngc ファイルが見つからなかった場合にロードされるデフォルト設定として保存する。

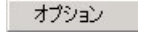
バイナリをロードする	バイナリファイルをターゲットメモリにダウンロードできる。
バイナリを保存する	ターゲットメモリをバイナリファイルとして保存できる。
ソースファイルをロードする	ソースファイルを検索して新しいソースペインにロードできる。
閉じる	ProDG デバッガを終了する。

デバッグメニュー



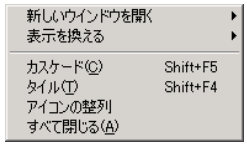
実行	アプリケーションの実行を開始する。
停止	アプリケーションの実行を中止する。
ステップ	デバッガを 1 ステップずつ実行する。
ステップオーバー	現在のソース行にある関数のコードをとばして、現在の関数の次の行まで 1 ステップずつ実行する。
ステップアウト	現在の関数から移動し、呼び出し関数の次の行まで 1 ステップずつ実行する。
カーソルまで実行	ペインのカーソルがある行にプログラムカウンタを設定する。
アドレスまで実行	指定したメモリアドレスにプログラムカウンタを設定する。
Gekko ハードウェアブレーク(C 変数)	C 変数ハードウェアブレークを設定できる。
Gekko ハードウェアブレーク(Asm レベル)	アセンブラレベルハードウェアブレークを設定できる。
リセットしてリロードする	プログラムの開始ポイントにプログラムカウンタを再設定する。
ターゲットを選択する	ターゲット選択を可能にする。
ソース検索パス	ソースファイルの検索パスを設定・変更可能。

設定メニュー



オプション	ProDG デバッガのペインの色とフォント、およびアクセラレータキー、検出された エラーを設定可能。

ウィンドウメニュー

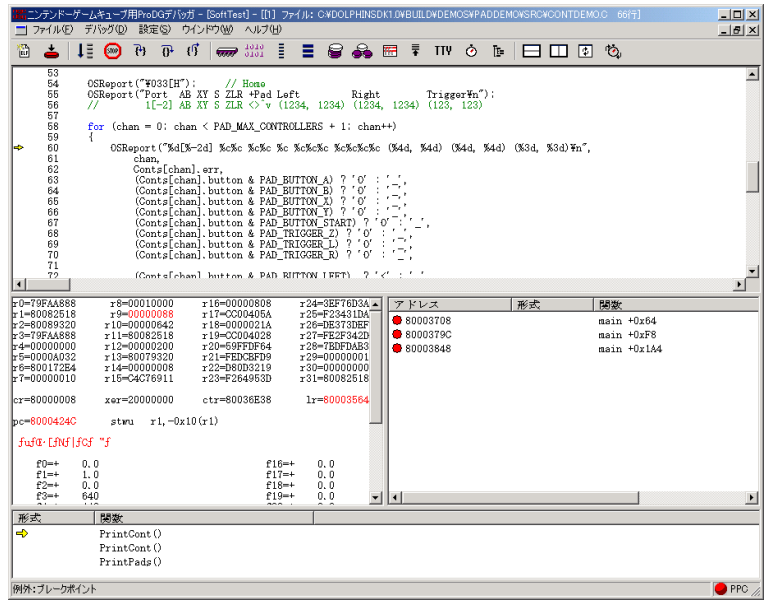


新しいウィンドウを開く	デバッガで新しいウィンドウを作成できる。
表示の切換	このサブメニューを使用すると、現在のペインタイプを別のペインタイプに変更できる。
カスケード	全てのウィンドウを標準の Windows カスケード形式で (重ねて)表示する。
タイル	全てのデバッガウィンドウを標準の Windows タイル形式で (重ねずに)配置する。
アイコンの整列	全ての最小化されたウィンドウを整列させる。
すべて閉じる	開いている全てのウィンドウを閉じる。

ウィンドウとペイン

デバッガの主な機能は、メニュー、ツールバーボタン、ショートカットキーからアクセスできます。このセクションでは、デバッガで使用できるペインについての情報と、キーボードショートカット、メニュー、ツールバーを使用してそれらのペインを作成・操作する方法を解説します。

ProDG デバッガは、ターゲットから取得したさまざまなタイプの情報を閲覧できるペインを含む、1 つまたは複数のウィンドウから構成されています。各ウィンドウには、1 つまたは複数の異なるタイプのペインが含まれています。



1 つのウィンドウに複数のペインがある場合は、上図の例のように、ペインごとに分割されて表示されます。

ウィンドウとペインの配置は、各デバッグセッション終了時にデバッガ設定ファイルに保存されます (25ページの「ユーザインタフェースを設定する」参照)。

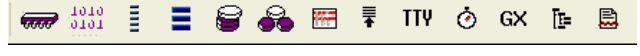
新しいデバッガウィンドウを作成する

いずれかのデバッガペインを含む新しいウィンドウは、いつでも開くことができます。これらのウィンドウは、メインツールバーボタン、または**ウィンドウ→ 新しいウィンドウを開く**メニューを使用して開くことができます。さらに、ショートカットメニューの**表示の切換**コマンドを使用すれば、既存のペインのタイプを変更できます。

既存ウィンドウの中に新しいペインを作成するには、そのウィンドウを分割します。詳しくは14ページの「ペインを横方向または縦方向に分割するには」をご覧下さい。

デフォルトでは、アプリケーションがターゲット上で実行を中断するたびに、デバッガペインが現在の情報に更新されます。この他にも、いくつかの更新オプションを設定できます。詳しくは、16ページの「デバッガのペイン更新」をご覧下さい。

各ペインタイプには、マウスの右クリックでアクセスできる固有のショートカットメニューがあります。このショートカットメニューには、各ペインタイプに特有のコマンドが含まれています。最も頻繁に使うコマンドの中には、キーボードショートカットからもアクセスできるものがあります。以下の表では、デバッガで使用できるペインのタイプを簡単に説明していますが、さらに詳しい解説は:



これらのボタンをクリックすると、以下の順でデバッガペインが開きます。

レジスタ画面	閲覧している GameCube 上の現在のレジスタ値を閲覧できます。プログラムカウンタ、およびターゲットのステータスも表示されます。
メモリ画面	閲覧している GameCube ユニット上のメモリを閲覧できます。
逆アセンブリ画面	閲覧している GameCube ユニット上で現在動作している逆アセンブリを閲覧できます。プログラムカウンタが設定されている命令が表示され、ターゲットユニットに、ブレークポイントやシングルステップを設定できます。
ソース画面	閲覧している GameCube ユニット上で現在動作しているプログラムのソースを閲覧できます。現在のプログラムカウンタが表示され、ターゲットユニット上で動作しているソースを通じて、ブレークポイントやシングルステップを設定できます。
ローカル変数画面	現在の関数の全ローカル変数の値を閲覧できます。ペインのショートカットメニューで、どんな構造や配列のディスプレイも拡大したり閉じたりできます。
ウォッチ画面	追跡したい特定の変数の組み合わせを閲覧できます。ウォッチを追加・削除したり、ウォッチされている構造または配列の構成要素の表示を拡大したり閉じたりできます。
ブレークポイント画面	アプリケーションに設定された全てのブレークポイントのリストを閲覧できます。ブレークポイントは、メモリ中にあるソースや逆アセンブリの行アドレスで示されます。
コールスタック画面	プログラム中の現在の場所に到達するために作成されたコールスタック上の関数コールを閲覧できます。最後に呼び出された関数が、コールスタックの一番上に表示されます。別の関数コールを選択すれば、デバッガのコンテクストを変えられます。
TTY コンソール画面	ターゲットの GameCube が生成した任意の標準出力を閲覧できます。
プロファイラ画面	プロセスにどれに時間がかかっているか調べるため、メイン CPU 使用状況のベーシックプロファイルを作成できます。
ディスプレイリスト(GX)	指定したメモリアドレスからの GX ディスプレイリストの逆アセンブリを表示します。
ワークスペース画面	ツリー状ビュー内で ELF のソースファイル、関数、変数およびクラスを閲覧するワークスペースを作成します。
カーネル画面	ターゲットで実行中のスレッドに関する情報を閲覧できます。

ウィンドウ間を移動するには

複数のウィンドウを作成した場合は、目的のウィンドウ上をクリックするだけで、そのウィンドウを選択できます。ただし、キーボードショートカットを使用してウィンドウを順に選択することもできます。

1. Windows に標準で設定されているショートカットである<Ctrl+tab>を押すと、アクティブウィンドウは、リスト中の次の ProDG デバッガに変わります。

2. <Ctrl+[0-9]>キーを使用してウィンドウを切り替えることができます。最初に作成された 10 個の MDI ウィンドウに番号が割り当てられ、その番号はタイトルバーに表示されます。この番号は各ウィンドウをペインレイアウトの全面に切り替える際に使用するインデックスとなります。

3. 例えば、ウィンドウ[1] を全面に移動する場合は、<Ctrl+1>を押します。10 以降のウィンドウはインデックスが割り当てられないので、簡単に切り替えることができません。ウィンドウを 1 つ削除すると、そのインデックスが次に新たに作成したウィンドウに再利用されます。その他のウィンドウのインデックスはそのままで変わりません。

> **注:** ウィンドウのインデックスはデバッガのプロジェクト独自の構成ファイルに残り続けるため、再びデバッガをロードアップするとその状態のウィンドウインデックスが維持されます。

ソースペイン、逆アセンブリペイン、ミックスモードペインを行き来する

ソース、逆アセンブリ、ミックスモードのいずれかが選択されている時に他に移るにはスペースバーを押します。順序はソースペイン、逆アセンブリペイン、ミックスモードペイン、またソースペインとなります。

この機能のためのキーストロークは、ソースおよび逆アセンブリペインのアクセラレータグループにある **Toggle src to dis** オプションによって設定可能です (29ページの「アクセラレータキー」参照)。

ペインのショートカットメニュー

各ペインには、ショートカットメニューがあり、ペイン上を右クリックすると表示されます。メニューのフォーマットは様々ですが、全てのペインタイプに共通するメニューアイテムもあります。

ビューを切り替える	このサブメニューを使用すると、現在のペインのタイプを別のペインタイプに変更できる。
ペイン	このサブメニューを使用すると、ペイン別ビューを操作するコマンド(分割、削除など)にアクセスできる。
ペインを取消す	現在選択されているペインを削除する。
フォント設定	現在のメモリペインに関してのみ、情報表示のフォントを変更できる。全てのペインの表示フォントをグローバルに変更したい場合は、[アプリケーション設定]ダイアログ (26ページの「ペイン

の色とフォント」参照) を使用する。

個々のペインのショートカットメニューの説明においては、再度、これらのオプションの説明はありません。

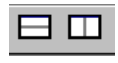
ペイン別ビューを作成する

1 つのペインを含む新しいウィンドウを作成すると、そのウィンドウを多種多様な方法で分割し、新しくできたペイン部分に割当てられるペインタイプを指定できます。

ペインを横方向または縦方向に分割するには

1. 分割したいペインを選択します。

2. 右クリックでペインのショートカットメニューを表示させ、**ペイン→横方向に分割する**または**縦方向に分割する**を選択するか、デバッガのツールバーから、**ビューを横方向に分割する**または**ビューを縦方向に分割する**ツールバーボタンをクリックします。

   ペインが横方向、または縦方向に分割され、新しくできたペインに、最初に選択したペインと同じペインタイプが割当てられます。

3. ここで、ショートカットメニューの**ビューを切り替える**オプションを使用して、新しく作成されたペイン部分の中で必要なペインを示す必要があります (14ページの「ペインタイプを変更するには」参照)。

ペインタイプを変更するには

既存ペインのタイプは、ペインショートカットメニューの**ビューを切り替える** オプションで、いつでも変更できます。この操作で、各々のペインタイプのサブメニュー全てにアクセスできます。

この他に、[ペインタイプを変更]アクセラレータキーを使用して、以下の手順でペインタイプを変更することもできます。

1. **設定**メニューオプションから**オプション**を選択して、[アプリケーション設定]ダイアログを表示させます。ここで、**アクセラレータ**タブが選択されていることを確認します。

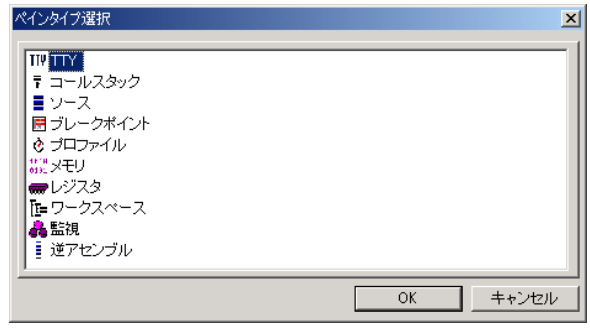
2. 右上にあるリストボックスから **Application** を選択します。

3. **コマンド名**リストをスクロールして、**Change pane type** のアクセラレータ キー設定を表示させ、該当する**キーシーケンス** (あれば) を確認します。

4. **キャンセル** をクリックして[アプリケーション設定]ダイアログを閉じます。

5. 別のペインタイプに変更するペインを選択します。

6. 確認した順序でキーを押すと、ダイアログが表示され、新しいペインタイプを選択できるようになります。



7. 上下矢印キーを使用してリスト内を移動し、該当するペインが選択されたらエンターキーを押し、**OK** をクリックします。

既存のペインを削除するには

1. 削除するペインの領域まで拡大したいペインを選択します。

2. **ペイン → ペインを取消す** をクリックし、表示されるサブメニューで、 削除する部分を選択します。

3. 該当する辺を削除すると、現在選択中のペインが、削除されたペインの領域まで拡大します。

ペイン間でフォーカスを移動するには

デバッガでは、一度に一つのペインがフォーカスされ、そのペインで作業できます。フォーカスは、ペイン上をクリックするだけで設定できます。

この他に、以下の手順で[Move Focus]アクセラレータキーを使用して、フォーカスを ペイン間で移動できます。

1. **設定**メニューオプションから**オプション**を選択して、[アプリケーション設定]ダイアログを表示させます。ここで、**アクセラレータ**タブが選択されていることを確認します。

2. 右上のリストボックスで **Application** を選択します。

3. **コマンド名**リストをスクロールして **[Move Focus up** / **down** / **left** / **right]** のアクセラレータキー設定を表示させ、該当する[キーシーケンス] (あれば) を確認します。

4. **キャンセル** をクリックして[アプリケーション設定]ダイアログを閉じます。

5. 該当するキー順序でキーを押し、移動したいペインにフォーカスを移動します。選択した矢印の方向にペインがない場合、フォーカスは現在のペインにとどまります。

ペイン分割バーを移動するには

ペイン別ビューの分割バーを移動して、その両側にあるペインのうちいずれかの領域を広げることができます。バーは、マウスを使用して簡単に選択・ドラッグできます。

この他に、以下の手順で[Move bar]アクセラレータキーを使用して、ペイン分割バー を移動することもできます。

1. **設定**メニューオプションから**オプション**を選択して、[アプリケーション設定]ダイアログを表示させます。ここで、**アクセラレータ**タブが選択されていることを確認します。

2. 右上のリストボックスで **Application** を選択します。

3. **コマンド名**リストをスクロールして **[Move top** / **bottom** / **left** / **right]** のアクセラレータキー設定を表示させ、該当する**キーシーケンス**(あれば) を確認します。

4. **キャンセル** をクリックして[アプリケーション設定]ダイアログを閉じます。

5. 境界バーを移動したいペインを選択します。

6. 該当するキー順序でキーを押し、移動したい方向にバーを移動します。その境界がウィンドウの境界である場合は、ペイン境界に変化はありませんのでご注意下さい。

デバッガのペイン更新

デフォルトでは、全てのデバッガのペインは、ターゲットが動作を中止すると更新されます。この他に、デバッガツールバーの**全てのビューを更新する**ボタンと**自動更新トグル**ボタンでも全てのペインを更新できます。

全てのビューを更新するボタンが、全ペインの内容を一度だけリフレッシュするのに対して、**自動更新トグル**ボタンを押して全ての自動更新を有効にすると、ターゲットに最新情報がポーリングされ、ペインが常に更新されます。

また、–w コマンドラインスイッチを使用して、ターゲット停止時に自動更新されるペインを限定することができます。

全てのペインを手動で更新するには

- ツールバーにある下図の**全てのビューを更新する**ボタンをクリックします。

この操作を行なうと、ターゲットに最新情報を 1 回問い合わせます。取得された情報は、開いている該当ペインに再表示されます。

全てのペインを自動で更新するには

- ツールバーにある下図の**自動更新トグル**ボタンをクリックします。

この操作を行なうと、ターゲットに最新情報を継続的に問い合わせます。取得された情報は、開いている該当ペインに再表示されます。

- **自動更新トグル**ボタンは、自動更新機能がオンになっている時は、選択されてへこんだ状態となります。自動更新機能をオフにする場合は、もう一度**自動更新トグル**を押します。

ターゲットとプロセッサステータス

ProDG デバッガのウィンドウには、下図のように、ターゲットの接続ステータスと、ターゲットプロセッサのステータスを表すグラフィックが表示されるステータスフッタがあります。

プロセッサが動作していない時は、「Smarty」(=M&M) は赤色です。この色は、プロセッサがアクティブ状態になると、赤色から緑色に変わります。

テキスト選択およびクリップボード

テキスト選択は、テキストを選択し、クリップボードにコピーできるように、ソースペインおよび TTY ペインで実行されています。この選択操作は、Notepad あるいは Microsoft Word などの、Windows のテキストエディタやワードプロセッサと同様です。

テキストは、マウスかキーボードを使用して（あるいはその両方を組み合わせて）選択できます。ただし、TTY ペインはマウスを使った選択のみをサポートします。

メモリへはメモリペインでマウスを操作して選択してもアドレス可能です。詳しくは73ページの「メモリペイン」をご覧下さい。

マウスで選択する

マウスを使って選択するには、選択する最初の文字にカーソルを移動し、マウスの左ボタンを押し、選択する最後の文字までマウスをドラッグして、マウスボタンを離します。マウスをドラッグしている間、選択されたテキストは反転表示され、選択されたことを示します。選択されると、テキストは(マウスが選択したテキスト上にある間に)コンテキストメニューを使用して、あるいはキーボードアクセラレータ([ソース設定]タブからユーザ設定可能、ただしデフォルトは`<Ctrl+C>`)を使用してクリップボードにコピーできます。その後、ウォッチペインあるいは他のアプリケーションにペーストされます。

この選択は行に対して平行に行なうものです。マウスを数行下方に移動した場合、最初の行は選択開始位置から、その行の終わりまでを選択し、最後の行は行の始めから、（マウスがある）終わりの位置までを選択します。その 2 つの行の間は全て選択されます。

選択を定義する際に、(マウスボタンを押す前に)`<Alt>`キーを押すと、コラムモードが利用できます。これによって、全ての行範囲ではなく長方形部分の指定をできます。例えば、ソースペインの左上を起点とし、ペインの底部分を下方にドラッグして各行 6 文字ずつを入れ

ることによって、ウィンドウ内における各行の最初の 6 文字を選択できます。通常、これはペイン全体の内容選択に使用します。

キーボードで選択する（ソースペインのみ）

ソースペインには、全幅バーあるいは垂直ラインのいずれかのカーソル含まれます。矢印キーを使用してこのカーソルを移動できます。

<Shift>キーを押しながら矢印キーを使用し、マウスボタンを押した場合と同様に選択します。矢印キーを使用して行（上下）の選択、または列（左右）の選択、あるいはその両方行えます。

その他、特定タイプの選択をすばやく行なうための方法もあります。

マウスでテキストを選択するには

- 単語上でマウスを左ダブルクリックすると、その単語全体を選択できます。
- 左の端で(この位置で、カーソルが縦バーから右向き矢印に変わる時に)マウスをクリックすると、行全体を選択できます。
- 上記選択後、カーソルを上下移動し複数行を選択できます。

キーボードでテキストを選択するには（ソースペインのみ）

- <Ctrl>キーを押して、単語選択できます。（選択キー–<Shift>とあわせて）<Ctrl>キーを押すと、左右の矢印キーはそれぞれ前あるいは次の単語を選択します。
- <Ctrl+Shift+Home>で、カーソル位置からファイルの上部までを選択します。
- <Ctrl+Shift+End>で、カーソル位置からファイルの下部までを選択します。
- <Shift+Home>で、カーソル位置からその行の始めまでを選択します。
- <Shift+End>で、カーソル位置からその行の終わりまでを選択します。

その他

- コンテキストメニューには、[全て選択]と[クリア]メニューアイテムが含まれます。（これらはユーザ設定可能なアクセラレータが含まれます。）

マウスあるいはキーボードで選択を定義する際、ペインの端でファイルは適切な方向にスクロールします。

マウスを使った場合、ペインの端と画面の端の間におけるマウスの距離に応じて、スクロール速度が調整されます。マウスが画面の端に近いほどスクロール速度が速くなります。

使用の際の注意事項

- ペイン内でマウスをクリックし(あるいは矢印キーの 1 つを押して)現在の選択をキャンセルします。

- 全て選択キーボードアクセラレータは、**設定**ダイアログの**アクセラレータ**タブからユーザ設定が可能です。

- [テキスト検索]ダイアログボックスを表示すると、ダイアログが表示されると同時にテキスト選択が自動的にテキストコントロールに挿入されます。複数行選択を行なうと、最初の行のみが追加されます。

- ソースペインの縁の部分でダブルクリックすると、コード内の該当行にブレークポイントが追加されます。

- ウォッチペインには**ペースト**機能があります(その機能はショートカットメニューに表示されます-<Ctrl+V>のデフォルトアクセラレータ)。それによってソース ペインでさまざまな名称を選択し、クリップボードにコピーし、ウォッチペインにペーストできます(複数行選択では最初の行のみがペーストされます)。

数式

デバッガ使用中に、数式入力が必要、またはそうしたほうが便利なときがあります。数式には、16 進法の定数や C 変数などの単純なものから、配列からポインタを脱参照して得られる構造タイプキャストメンバーなどの複雑なものまであります。

ProDG デバッガでは、プログラムにあることがわかっている変数と、広範な選択肢から選べる C および C++スタイルの演算子を使用して、こうした全ての数式をビルドできます。

また、数式をその場で評価して、正しく定義したかどうかを確認することもできます。

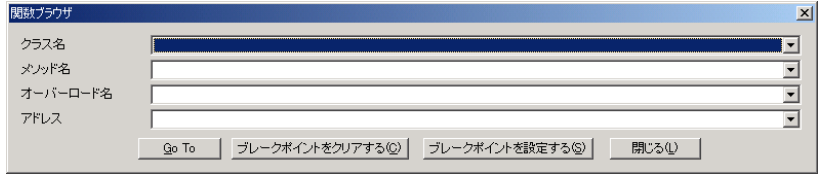
- 数式とアドレスの評価機能は、全てのアクティブなシンボル表を使用して数式を評価します。

数式は、デバッガのダイアログからアドレスや数式、値を尋ねられた場所なら、どこでも使用できます。ウォッチペインやローカルペインは、絶えずこの数式を参照するため、数式の全ての機能を活用するには、その働きを理解することが重要です。

関数ブラウザを使用する

ソースペインや逆アセンブリペイン、ブレークポイントペインで関数ブラウザを使い、アプリケーションのソースや逆アセンブリ内を、その中にある関数に従って移動できます。さらに、ブレークポイントをグローバルに特定の関数に設定したり、解除できます。

関数ブラウザを開くには、ソースペインまたは逆アセンブリペインのショートカットメニューにある**関数をブラウズする**コマンドを使用します。



名称の再構築

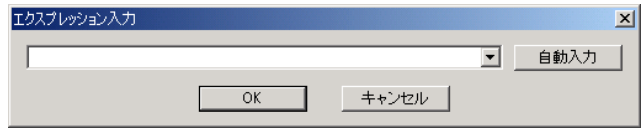
デバッガのさまざまな場所、たとえばブレークポイントやコールスタックフレームで、「再構築された」C++関数名を表示するよう指定できます。

C++関数名を再構築するには

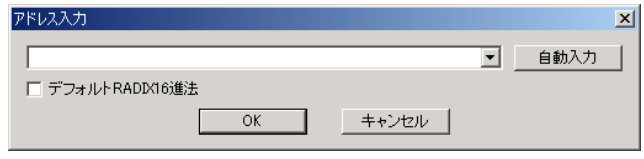
C++関数名の再構築は、demangle.dll ファイルを Nintendo GameCube ログラムディレクトリにコピーするだけで実行できます。デバッガがこの DLL を検出すると、「破壊された」関数名を「再構築された」関数名に、自動的に変換します。

数式アドレスを入力する

[式入力]ダイアログは、ウォッチペインにウォッチを追加する時に、数式を入力するために使用します。



[アドレス入力]ダイアログは、ブレークポイントペインでブレークポイントを設定する時に、アドレスを入力するために使用します。このダイアログの動作は、[式入力]ダイアログと非常によく似ています。:



[式入力]ダイアログと[アドレス入力]ダイアログは、どちらも以下の機能を提供します。:

- 名称の補完。
- クリップボードから、またはクリップボードへ、式／アドレスのコピー＆貼り付け。
- ドロップダウンメニューからアクセスできる最近使用した式またはアドレスの履歴。

ソースペインでのホバー評価

ProDG デバッガには「ホバー評価」機能があります。ソースペインで短い時間マウスカーソルをホールドし、ソースコードでの式を評価します。

以下のコードフラグメントの[main]上にカーソルを置くと、ツールチップ内の関数タイプおよびアドレス情報が表示され、変数上では「front_end_game_data.selected_country」数式全体を評価し、タイプおよび数値情報を表示します。

テキストを選択すると、その選択した範囲のテキストを評価できます。これは下記の例において、変数名の一部を評価するのにとても便利です。

例えば以下の「front_end_game_data.selected_country」テキストを選択すると、以前通りに動作しますが、変数の「front_end_game_data」部分のみを選択し、その上にカーソルを置くと、選択した部分のみを評価し、（この例の場合）そのクラスのタイプおよびアドレス情報が表示されます。

```
main()
{
    front_end_game_data.selected_country = 0;
    front_end_game_data.selected_game_mode = WWR_TIME_ATTACK_MODE;
    front_end_game_data.selected_class = NOVICE_CLASS;
    front_end_game_data.selected_car = WWR_DOUBLE_MOON;
    ……................
}
```

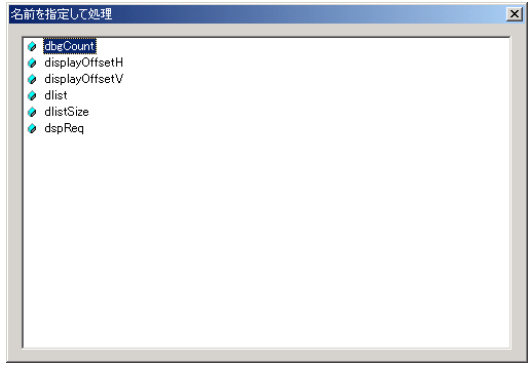
名称の補完

自動入力ボタンを使用すると、部分的に補完した名称を使い、その部分的文字列で始まる全ての変数のリストを表示することで、変数名の入力を高速化できます。

- 名称の補完機能は、全てのアクティブなシンボル表で名称を検索します。

使用されている名称に、検索文字列と一致するものがなければ、**自動入力**オプションは機能しません。検索する先頭文字列と一致する名称が 1 つだけ検出された場合は、リストボックスは表示されずに、その名称が適用されます。

検索文字列と一致する名称が複数検出された場合は、[名称を指定して処理]ウィンドウが表示されます。たとえば、“d”を入力して**自動入力**をクリックすると、下図のように“d-”という文字で始まる名称のリストが表示され、そのうちの必要なものを選択できます。



各シンボルは、そのタイプを反映するアイコンと共に表示されることに注意して下さい。水色のダイヤモンドは変数、ピンクのダイヤモンドは関数、ツリーはクラスまたは構造体です。

数式をビルドする

数式の主な要素は、数値と変数名です。

- 数値は、10 進法、16 進法のいずれかの形式で入力できます。
- 16 進法の数値を入力する時は、0x を省略できる機能を提供する**デフォルト 16 進法基数**チェックボックスがオンになっている場合以外は、始めに 0x を付ける必

要があります (数値が 10 進法の数字で始まっている場合、たとえば"A0" ではなく"0A0") 。

- 変数名を打ち込めば、変数を入力できます。また、自動入力ボタンを使用すれば、入力を簡略化したり、正しい名称を参照しているかどうかを確認できます (21ページの「名称の補完」参照)。

数式をビルドするより複雑な方法には、C および C++演算子、ラベルアドレスと関数アドレス、タイプキャストなどがあります。

レジスタ名

数式でレジスタ名を使用できます。レジスタ名の前には、変数と区別するため、$記号をつける必要があります。

レジスタには、キャストされていない場合は long128 の C 形式、キャストされている場合は該当するタイプの C タイプがあります (22ページの「タイプキャストとタイプデフ」参照)。

タイプキャストとタイプデフ

全ての数式は、ちょうど C と同様に、タイプキャストできます。たとえば、ウォッチペインに (int*)$fp と入力すると、以下の文字列が表示されます。

```
+ (int*)$fp   int *  -> 00000001
```

構造タグを使用してタイプキャストを行なえますが、構造タグにキャストする際に、struct というキーワードを入力する必要はありません。構造またはクラスにタイプキャストすると、以下の文字列が表示されます。

```
-Tester* (Tester*)$fp = 0x807ff88
-Tester
+unsigned char* m_pName = 0x00000645
+unsigned char* mpLongName = 0xFFFFFFFF
```

タイプデフにキャストすることもできます。たとえば、`(daddr_t)p` と入力すると、以下の文字列が表示されます。

```
long (daddr_t)p = 0x00003024
```

ラベル

数式でラベルを使用できます。評価機能は、まず変数名の一致作業を行ない、次にラベルを検索します。

ラベルには、`int` の `C` タイプがあります。

関数

数式で関数名を使用できます。関数名の値は、その関数のアドレスです。

関数は、ウォッチウィンドウで以下のように表示されます。

```
main   int ()     @ 00201020
```

関数は int の C 形式です。

名称の一致作業の優先順位

数式中の名称を検索する順序は、以下の通りです。

1. 退避させたレジスタ名 ($が先頭についた名称)
2. C 名
3. ラベル名

ペイン間でデータを移動する

ドラッグ &ドロップは、異なるペイン間の移動を行なうために実行されています。例えば、特定の関数にブレークポイントを追加したいが、正確な関数名あるはスペルが思い出せない場合、ワークスペースペインをブラウズし、関数を検索できます。そして関数名をブレークポイントペインに「ドラッグ」後、「ドロップ」します。これによって関数のアドレスにブレークポイントが追加されます。

注: 通常の Windows での使用と同様に、ドラッグする情報上にカーソルを移動し、マウスの左ボタンを押してドラッグ &ドロップを実行します。次にカーソルを(マウスボタンを押し続けながら)ドロップするペイン上に移動します。その後マウスボタンを離すと、データが新たなペインにコピーされます

ドラッグ &ドロップ作業の結果は、データをドラッグする、またドロップするペインによって異なります。起こりうる組み合わせの表を以下に示します。:

	ドラッグ先						
ドラッグ元	SOURCE	WRKSPC	MEMORY	WATCH	LOCALS	BRKPNT	DISASM
ソース	Goto function		Goto address	Add watch		Add bp	Goto
ワークスペースファイル	Load					Add bp to all funcs in file	
ワークスペースブラウズ	Goto function		Goto address	Add to watch		Add Bp	Goto
メモリ	Goto function		Goto address	Add to watch		Add bp	Goto function
ウォッチ	Goto function		Goto address	Duplicate/ split watch entry		Add bp	Goto function
ローカル			Goto address	Add to watch			

ブレークポイント	Goto function		Goto address	Add to watch			Goto function
逆アセンブル							
コールスタック	Goto function		Goto address	Add to watch		Add Bp	Goto function

マウスがデータをドラッグしている間は、マウスはデータがドラッグ中であることを示す表示に変わります。

ドラッグ中を示すカーソル— ドラッグ＆ドロップが進行中

ドロップ不可を示すカーソル— ここへのドロップは不可能

- Goto 操作（ソースペインにドラッグする）は、現在のところ関数上でのみ 有効です。
- メモリペインからタイプの異なるペインへのドラッグは、オブジェクトの先頭アドレスからのみ実行可能です。この操作を確認するには、メモリペインに関数あるいは変数をドラッグした後に、メモリペインからウォッチペインにドラッグします。この場合、変数のアドレスからドラッグする場合のみ正しく評価され、ウォッチペインに追加されます。その他のメモリペインにドラッグする場合はあらゆるアドレスから実行できます。
- メモリペイン上では、ペイン左側のアドレスからドラッグします。
- ソースペイン上では、テキスト選択がドラッグされます。複数行を選択すると、最初の行のみがドラッグされます。先頭のスペースは削除されます。
- ウォッチペインにストラクチャが追加され、拡張されると、アイテムメンバーがウォッチペインにドロップされ、そのメンバーに関するウォッチ数式が追加されます。例えば「front_end_game_data」がウォッチペインに追加され、拡張されると、'selected_country'のメンバー変数をドラッグし、ウォッチペインにドロップしなおすことによって、「front_end_game _data.selected_country」の新たなウ ォッチ数式が追加されます（23ページの「ドラッグ＆ドロップとソースペイン」を参照のこと）。
- ウォッチペインにおいて、それ自身にドラッグ＆ドロップするには、カーソルを少なくともドロップする行の高さにまで移動する必要があります。これば誤ったドラッグ操作を防ぐためです（ストラクチャを拡張するためにクリックする時のマウス移動など）。

ドラッグ＆ドロップとソースペイン

ソースペインから情報をドラッグするには、テキスト選択を行ないます。その後、選択したデータ上でマウスボタンを押して、通常通りのドラッグ操作を行ないます。選択が複数行におよぶ場合は、最初の行のみが「ドロップ」されます。

例えば、コード内の「front_end_game_data.selected_country」を選択し、ウォッチペインにドラッグすると、このクラスメンバーがウォッチペインに追加されます。しかし行の「front_end_game_data」部分のみを選択し、ウォッチペインにドロップすると、実際のクラスがウォッチ変数として追加され、全てのメンバー変数を表示するように拡張されます。

```
main()
{
    front_end_game_data.selected_country = 0;
    front_end_game_data.selected_game_mode = WWR_TIME_ATTACK_MODE;
    front_end_game_data.selected_class = NOVICE_CLASS;
    front_end_game_data.selected_car = WWR_DOUBLE_MOON;
    ……————————
}
```

ペイン修飾キー

データをペインにドロップする場合、存在するペインに追加するより新たなペインを作成理想的です（既存のペインは変更せずに、ソースファイルを新たなペインにロードします）。

これを有効にするために、3つのキーボード修飾子(<Ctrl>、<Shift>、<Alt>)が使用されています。これらのキーは、現在ユーザ設定が不可能です。データをペインにドロップする間にこれらのキーの 1 つを押すと、修飾子が起動します。

- <Ctrl>キーは、ドラッグ先ペインを垂直方向に分割し、データを（既存のペイン下の）新たなペインにドロップします。
- <Shift>は水平方向に分割し、最も右側のペインにドロップします。
- 最後に<Alt>キーは、ドロップされたデータのための（浮動）ペインを新たに作成します。

データを受けつけないペイン（あるいはペインを含まない画面領域）にデータがドラッグされた場合、カーソルは「No Entry sign」を表示します。（23ページの「ペイン間でデータを移動する」を参照）。No Entry cursor が表示されている間 にマウスボタンを離すと、ドラッグ操作は中止され、データは放棄されます。

ユーザインタフェースを設定する

ProDG デバッガのペインは、完全かつ個別に設定が可能となったため、ユーザインタフェースの外観を望み通りに設定できます。[アプリケーション設定]ダイアログで、以下のパラメータを設定できます。

- ペインの色とフォント。ソースペインの書式の色も含む。
- アクセラレータキー。

また、ペインのレイアウトや、最後に接続されたターゲット、およびその他のプロジェクト設定を、プロジェクト設定として保存、レストアできます。

デバッガアプリケーションの終了時、アプリケーションは現在の設定を、通常のテキストファイルである dbugngc.ngc の形で、実行ディレクトリに保存します。また、デバッガアプリケ

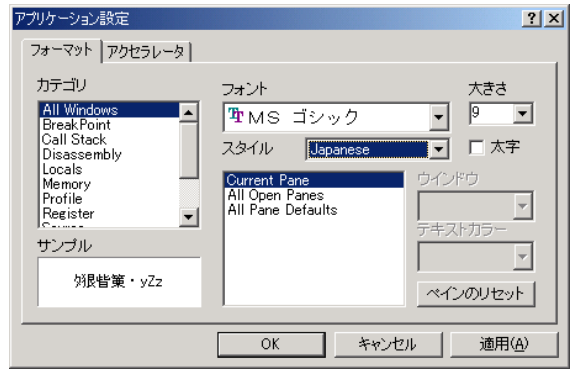
ーションが起動すると、現在の実行ディレクトリにあるこのファイルから、現在の設定を復元します。

ペインの色とフォント

ペインの色とフォントは、[アプリケーション設定]ダイアログで決定します。

1. **設定**メニューオプションから**オプション**を選択します。**フォーマット**タブが選択されていることを確認します。

   下図のような[アプリケーション設定]ダイアログが表示されます。



2. **[カテゴリ]**リストには、全ての選択可能なペインタイプが表示されます。現在の設定と異なるペインタイプを選択すると、サブカテゴリリストが更新され、そのペインに対応する CPU タイプが表示されます。ただし、**[Source]** カテゴリの サブカテゴリリストでは、書式の色を決定するための追加入力項目が表示されます(27ページの「ソースペインの書式の色を設定するには」参照)。また、**[All Windows]**カテゴリのサブディレクトリリストでは、使用可能な全ての CPU と **[All Windows]**が表示されます。 決定した色とフォントを特定のタイプの 全ウィンドウに適用したい場合は、**[All Windows]** を選択し、それ以外の 場合は、リストから該当するペインタイプ (例:メモリペインなら **[Memory]**) を選択します。

3. 右側にあるサブカテゴリセクションは、[カテゴリ]で選択した項目によって内容が変わります。たとえば[**メモリ**]を選択すると、各メモリペインに色とフォントを設定できます。

4. 各々のペイン形式でフォント(**フォント**)やポイントサイズ(**サイズ**)を個別に設定できます。 また、**太字**チェックボックスにチェックを入れることにより、太字で表示させることもできます。

5. **フォント**の選択項目によっては、**スタイル**リストボックスから別の字体を選択できる場合があります。

6. 最後に、**テキストカラー**ドロップダウンリストボックスからテキストの色を、**ウィンドウ**ドロップダウンリストボックスからウィンドウの背景色を選択できます。**Automatic** オプションを選択すると、テキストと背景がシステムで設定された色で表示されます。

7. **OK** をクリックして、選択したペインの色とフォントを保存します。設定は、新しく作成したペインに適用されます。このフォーマット設定は、ngcdbg.exe 実行ファイルと同じディレクトリにある設定ファイル ngcdbg.ini に保存されます。

ソースペインの書式の色を設定するには

ソースペインの書式に色を設定すると、ソースの異なる書式関連コンポーネントを別の色で表示できます。C、C++および VSM コードで書かれたソースファイルの全てが書式を色設定できます。

[Source]カテゴリでは、ソースペインのフォントと色の指定を変更できますが、同時に書式の色を指定することもできます。サブカテゴリリストは **Comment**、**Keyword**、**Number**、**Operator**、**PPC**、**String** および **User Defined** からなり、使用された色、その他のソースペイン構成要素を設定できます。

1. **設定**メニューオプションから**オプション**を選択して、[アプリケーション設定]ダイアログを表示させます。**フォーマット**タブが選択されていることを確認します。

2. **[Source]**カテゴリを選択します。[アプリケーション設定]ダイアログは下図のようになります。

3. コメント、キーワード、数値、演算子、文字列、ユーザ指定キーワードの各項目について、サブカテゴリセクションで個別に色を設定できるようになります。

> **注**: *#include 行の<> マーカーは文字列識別子として認識されます。*

プロジェクト設定を保存する

デバッガアプリケーションの終了時、現在のアプリケーション設定は、通常のテキストファイルである dbugngc.ngc の形で、自動的にデバッガ実行ディレクトリに保存されます。

実行ディレクトリは、ngcdbg コマンドを入力した時にいるディレクトリ (または Windows の [ショートカットのプロパティ]ダイアログの場所フィールドで指定されたディレクトリ) です。

デバッガアプリケーションの起動時、アプリケーションは、以下のディレクトリで dbugngc.ngc ファイルを検索して、現在の設定を呼び出します。

1. 現在の実行ディレクトリ.

2. 上記のディレクトリにファイルがなかった場合は、デバッガ実行ファイル(ngcdbg.exe)があるディレクトリから、デフォルト設定ファイルをロードします。

3. デフォルト設定ファイルが見つからなかった場合は、組み込みのデフォルト設定で開きます。

注: デバッガを、デフォルトの実行ファイルを設定せずに Windows ショートカットから実行している場合、デバッガ設定ファイルは、その時にいるディレクトリによって、その都度違う場所に保存されます。よって、デバッガ設定が思い通りに保存されない場合があります。

デバッガ設定には、以下の情報が保存されています。

- デバッガアプリケーションのデスクトップでのサイズと位置
- 最後に接続したターゲット名
- [Elf をロード]ダイアログで使用するフラグ
- ソースや逆アセンブリで設定されている全てのブレークポイント(起動時に-b コマンドラインオプションが使用されていない場合)
- 新しいペインに適用されるデフォルトのフォント
- 全ての開いているデバッガウィンドウとセットアップに関する以下の情報

  - サイズと位置

  - 形式

  - その他の表示モード情報(バイト/ワード単位のメモリ、1 行当りバイト数の 設定、開始アドレス、カーソル位置、ウォッチウィンドウに追加されたウォッチ、ソースウィンドウにロードされているファイル名、等)
- 現在アクティブ状態のデバッガウィンドウ、およびペイン
- ソース検索パス.

デバッガ設定を保存/リロードするには

現在のデバッガ設定は、**ファイル**メニューの**設定を保存する**を使用して、いつでも保存できます。デバッガ設定は、自動的に現在の実行ディレクトリにある dbugngc.ngc に保存されます。

さらに、現在の実行ファイルで dbugngc.ngc が見つからない場合にロードされるデフォルト設定として、設定を保存することも可能です。これには、**ファイル**メニューの**設定をデフォルトとして保存する**オプションを使用します。

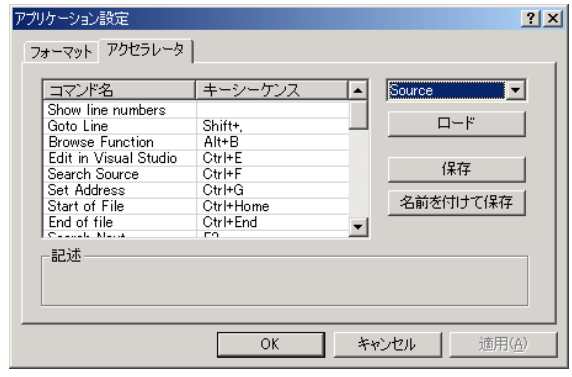
現在の実行ディレクトリから設定を再ロードして、現在の設定を上書きするには、**ファイル**メニューの**設定をロードする**を使用します。

アクセラレータキー

[アクセラレータ]ダイアログでは、デバッガの全てのコマンドについて、キー操作の設定・変更を行なうことができます。

1. **設定**メニューオプションから**オプション**を選択し、[アプリケーション設定]ダイアログを表示させます。**アクセラレータ**タブが選択されていることを確認します。

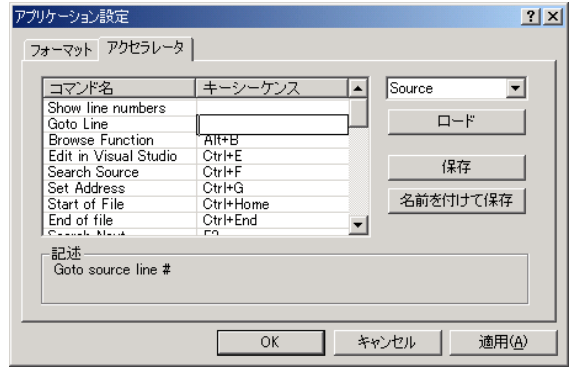
2. 右上のリストボックスで、ショートカットメニューにアクセラレータキーを設定したいペインタイプ（**ソース**ペインなど）を選択します。また、リストの最後に、ProDGデバッガアプリケーションのメニューオプションにアクセラレータキーを設定できる**アプリケーション**ペインタイプがあります。選択するペインタイプによって、ダイアログの中心部分に表示されるコマンド名とキー順序のリストが変わります。



3. **詳細**フィールドには、行を選択すると、各コマンド名の詳細が表示されます。

アクセラレータキーを設定するには

1. アクセラレータキーを設定したいコマンド名を選択し、それに対応する**キーシーケンス**フィールド内をクリックして、編集ボックスを表示します。



2. 選択したコマンド名のアクセラレータキー順序として、設定したいキーの組み合わせを押します。キー順序が編集ボックスに記憶されます。

3. 選択したキー順序が、同じペインの別のコマンドに既に設定されている場合は、「(〇〇 (選択したキー順序) はすでに××(コマンド名)に定義されています)」というエラーメッセージダイアログが、表示されます。この場合、別のキー順序を選択する必要があります。

アクセラレータキーを解除するには

設定したアクセラレータキーは、以下の手順で簡単に未設定状態に戻せます。

1. 解除したいアクセラレータキーを含む行を選択して、コマンド名とキー順序の両方を強調表示させます。

2. **[Delete]** キーを押すか、マウスを右クリックして**アクセラレータをクリアする**ショートカットメニューオプションを選択します。

アクセラレータキー設定を保存／ロードする

アクセラレータキーは、「Accelerator Key Mapping」ファイル（.akm ファイル） に保存されます。ngcdbg.ini ファイルに、前回ロードした.akm ファイルの完全パスとファイル名が保存されています。このファイルが、次にデバッガが起動したときに再ロードされます。

デフォルトのキー操作が、ngcdbg.exe 実行ファイルと同じディレクトリにあるデフォルトの ngcdbg.akm ファイルに保存されています。しかし、自分でいずれかのファイル名に指定したアクセラレータキー操作を保存することもできます。

1. 現在のアクセラレータキーをデフォルトの ngcdbg.akm に保存する場合は、**保存**をクリックします。保存を確定するか中止するかを確認するプロンプトが表示されます。

2. 現在のアクセラレータキーを別の名称の akm ファイルに保存する場合は、**名前を付けて保存**をクリックします。保存するディレクトリとファイル名を選択するブラウザが表示されます。

3. **ロード**をクリックして、ファイルブラウザを表示させます。ここで、アクセラレータキーのロード元となる何らかの名称が付けられた akm ファイルを選択できます。デフォルトファイル名は ngcdbg.akm です。

Visual Studio のキーマップも、ngcvs.akm というファイルの形で提供されています。SN Systems と Visual Studio のデフォルトのキーマップについて解説したリファレンスチャート が30ページの「キーボードショートカットリファレンス」にありますので、ご覧下さい。

キーボードショートカットリファレンス

このセクションでは、Nintendo GameCube 用 ProDG デバッガのショートカットキーの説明をします。

ProDG デバッガショートカットキー

このセクションでは、ProDG デバッガのデフォルトアクセラレータキーについて説明します。アクセラレータ キー設定は、キーマップファイルに保存されます。

デバッガは、2つのキーマップオプションを備えています。

設定メニュー内のオプションを使用して、これらのオプションの切り替えができます（30ページの「アクセラレータキー設定を保存／ロードする」参照）。

以下の表では、各キーマップで使用できるキーボードショートカット(SN および Visual Studio 用)を示します。

SN キーマップ	**Visual Studio キーマップ**	**解説**	**コンテキスト**
Shift+,	Shift+,	バイナリファイルを保存する。	いずれのペインでも使用可
Shift+.	Shift+.	バイナリファイルをロードする。	いずれのペインでも使用可
Ctrl+a	Ctrl+a	ハードウェアブレークポイントダイアログ(アセンブリ)を設定する	いずれのペインでも使用可
Ctrl+b	Ctrl+b	ハードウェアブレークポイントダイアログ(C 変数)を設定する	いずれのペインでも使用可
Alt+b	Alt+b	C/C++変数、または関数ブラウザ	ソースペイン、逆アセンブリペイン、ウォッチペイン
Ctrl+d	Ctrl+d	ハードウェアブレークポイント ダイアログ(チャネル)を設定する	いずれのペインでも使用可
Ctrl+e	Ctrl+e	Microsoft Visual Studio (既に起動していると仮定)で現在のソース行を編集する。	ソースペイン
Alt+e	Alt+e	[ELF ファイルをロード]ダイアログ	いずれのペインでも使用可
Ctrl+f	Ctrl+f	[検索文字列を入力]ダイアログ	ソースペイン
Ctrl+g	Ctrl+g	[アドレスを入力]ダイアログ	メモリペイン、ソースペイン、逆アセンブリペイン
Ctrl+l	Ctrl+l	アドレスにロックする。	逆アセンブリペイン
Ctrl+w	Ctrl+w	ワードサイズを切り換える。	メモリペイン
Tab	Tab	プログラムカウンタがある行へ移動する。	ソースペイン、逆アセンブリペイン

Shift+tab	Shift+tab	現在のカーソル位置にプログラムカウンタを設定する。	ソースペイン、逆アセンブリペイン
Ctrl+tab	Ctrl+tab	次のウィンドウへ循環する。	いずれのペインでも使用可
Return	Return	新しいメモリ値を入力する。	メモリペイン
Insert	Insert	新しいウォッチを追加する。	ウォッチペイン
Insert	Insert	ブレークポイントを追加する。	ブレークポイントペイン
Delete	Delete	選択したウォッチを削除する。	ウォッチペイン
Delete	Delete	選択したブレークポイントを削除する。	ブレークポイントペイン
Ctrl+Home	Ctrl+Home	ソースの最初の行にカーソルを移動する。	ソースペイン
Ctrl+End	Ctrl+End	ソースの最終行にカーソルを移動する。	ソースペイン
Left cursor arrow	Left cursor arrow	拡張した配列変数インデックスを減少して、配列メンバーを閲覧する。	ローカルペイン、ウォッチペイン
Right cursor arrow	Right cursor arrow	拡張した配列変数インデックスを増加して、配列メンバーを閲覧する。	ローカルペイン、ウォッチペイン
Shift+numpad*	Shift+numpad*	全てのペインを更新する。	いずれのペインでも使用可
numpad +	numpad +	ローカル変数、またはウォッチ変数を拡張する。	ローカルペイン、ウォッチペイン
numpad +	numpad +	選択したメモリの増加分を示す	メモリペイン
numpad +	numpad +	簡潔に表示する	DMA ペイン
numpad -	numpad -	ローカル変数、またはウォッチ変数を減算する。	ローカルペイン、ウォッチペイン
numpad -	numpad -	選択したメモリの減少を示す	メモリペイン
Esc	Esc	簡潔に表示する	DMA ペイン
Ctrl+Shift+ Insert	Ctrl+Shift+ Insert	ターゲット上で実行中のアプリケーションを停止する。	アプリケーションはターゲット上で動作している。
Alt+ arrow keys	Alt+ arrow keys	現在のペインタイプから変更することの可能ペインタイプのメニューを表示する。矢印キーを使用して、メニューを上下に移動可能。	いずれのペインでも使用可

Alt+F2	Ctrl+Shift+F5	ペイン別ビューに表示されているペイン間を、矢印の方向に移動する。	ペイン別ビューに表示されている全てのペイン
Shift+ arrow keys	Shift+ arrow keys	ターゲットをリセットして、アプリケーションを再ロードする。	いずれのペインでも使用可
Alt+Shift+ arrow keys	Alt+Shift+ arrow keys	ペイン分割バーを(矢印の方向に)外側に移動する	いずれのペインでも使用可
Ctrl+arrow keys	Ctrl+arrow keys	ペイン分割バーを(矢印の方向に)内側に移動する。	いずれのペインでも使用可
Ctrl+Shift+ arrow keys	Ctrl+Shift+ arrow keys	矢印キーを使用して現在のペインをどのうに分割するかを指定しながら、現在のペインの分割を指定可能。左右矢印キーでは現在のペインを水平方向に、上下矢印キーでは垂直方向に分割する。	いずれのペインでも使用可
F2	F2	次のブックマーク	ソースペイン
Shift+F2	Shift+F2	前のブックマーク	ソースペイン
F3	F3	検索文字列を再度指定する。	ソースペイン
Shift+F4	Shift+F4	メインウィンドウの全ての可視ペインを重ねて表示する	いずれのペインでも使用可
F5	F9	現在の行のブレークポイントを切り換える。(DMA ペイン用カーソル D ビットブレークポイント)	ソースペイン、逆アセンブリペインもしくは DMA ペイン
Shift+F5	Shift+F5	メインウィンドウの全ての可視ペインを重ねて表示する。	いずれのペインでも使用可
F6	Ctrl+F10	カーソル位置まで実行する。	ソースペイン、逆アセンブリペイン
F7	F11	「~まで」ステップ。アプリケーションを 1 ステップずつ実行して、呼び出された関数へ移動する。	ソースペイン、逆アセンブリペイン
F8	F10	ステップオーバー (~をとばす)。アプリケーションを 1 ステップずつ実行して、呼び出された関数をとばす。	ソースペイン、逆アセンブリペイン
Shift+F8	Shift+F11	ステップアウト(~から移動)。現在実行している呼び出された関数から移動する。	ソースペイン、逆アセンブリペイン
F9	F5	ターゲット上で動作しているアプリケーションを起動する。	アプリケーションはターゲットにロードされている。

Shift+F9	Shift+F9	クイックウォッチ。ウォッチペインの先頭にウォッチを追加する。	いずれのペインでも使用可
Ctrl+P	Ctrl+P	以前のファイルメニュー	ソースペイン
Ctrl+F	Ctrl+F	テキスト検索	ソースペイン
F3	F3	次を検索	ソースペイン
Ctrl+Return	Ctrl+Return	メモリ補完	メモリペイン
Ctrl+Insert, Ctrl+C	Ctrl+Insert, Ctrl+C	クリップボードにコピー	いずれのペインでも使用可
Shift+Insert, Ctrl+V	Shift+Insert, Ctrl+V	ペースト	いずれのペインでも使用可

第 3 章: ゲームのビルドおよびロード

デバッグ用ビルド

ProDG デバッガでプログラムをビルドする際、以下の点に留意して下さい。

1. プログラムが適切な ProDG 起動ライブラリに必ずリンクされていること。これを確実に行なう最も簡単な方法は、libsn.a ライブラリが、最初にリンクするライブラリのリストにあることを確認することです。

2. 開発対象に合った適切な通信ライブラリが必ずリンクされていること

   ニンテンドーゲームキューブ開発システムは、それぞれ別の通信ライブラリを使用します。任天堂の makefile は、サンプルをビルドする際に以下のルールを適用します。

```
if $AMCDDKBIN
  link amcexi2.a + amcnotstub.a
else
  amcstub.a
if $ODEMUSDKBIN
  odemuexi2.a + odenotstub.a
else
  odemustubs.a
```

3. 適切な通信ファイルがリンクされているかどうかわからない場合は、ngccc –v パラメータを使って詳細情報を表示させ、使用しているハードウェアに合ったライブラリがリンクされているかどうか確認します。

4. コンパイラ最適化を削除。makefile の CFLAGS= または CXXFLAGS=の 両方の変数の-Ox エントリなど

5. デバッグ情報フラグを設定。makefile の CFLAGS=または CXXFLAGS=の両方の変数の-g スイッチ

デフォルトメモリマップの無効

このセクションでは、GameCube デフォルトメモリマップを無効にする方法を説明します。

GameCube ターゲットに接続する前にデバッガは、memmap.ngc ファイルを検索します。まず始に、現在のデイレクトリを検索し、見つからない場合 ngcdbg.exe が位置しているデイレクトリを検索します。これにより、1つのグローバルメモリ設定あるいはカスタムメモリ

設定をプロジェクト/デイレクトリごとに使用できるようになっています。このファイルが全く見つからない場合は、通常の GameCube で使用できるメモリレンジを使用します。

ファイル memmap.ngc が見つかった場合、このファイルからデバッガメモリレンジを読み込みます。ファイルは、標準テキストで次のようになっています:

```
80000000 817FFFFF 2     read-write main mem
88000000 881DFFFF 2     read-write EFB
E0000000 E0003FFF 2     read-write 16K of locked cache
```

ラインごとに 1 つのメモリレンジがあります。それぞれのラインは、スタートアドレス、スペース、エンドアドレス(含む)、タイプ(0=illegal, 1=read only, 2=read/write)の 形式になっています。ラインの始めの余白は無視されます。このラインでは、スペース後は無視されコメントとして使用できるようになっています。64 メモリレンジ以上設定しないでください。64 以降は全て無視されます。また、アドレス順序で配列する必要があります。

ターゲットへの接続

コマンドラインオプションを使用して、ELF を自動的にロードした状態で デバッガを起動できます(シンボルのみ、シンボル+実行可能ファイルおよびオプションでまずターゲットをリセットするなど)。

コマンドライン パラメータなしでデバッガを起動した場合でも、デバッガのスタートアップ機能にアクセスできます。例えば、ELF ファイルをツールバーやメニューからロードできます。

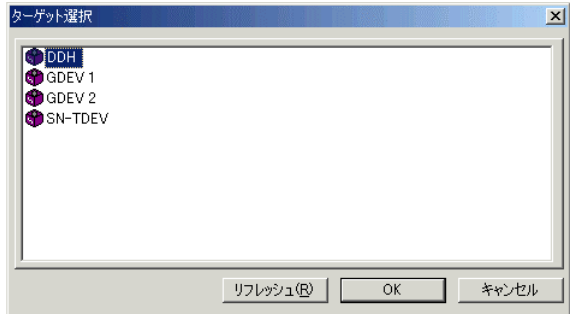
デバッガを起動すると Target Manager も自動で起動し、ダイアログで設定したターゲットがリストされます(ご使用のデバッガコマンドラインオプションによる)。また、デバッガ使用中に、**デバッグ**メニューの**ターゲットを選択する**を使用して作業中のターゲットを変更することもできます。

GameCube ターゲットを Target Manager で選択していない場合は別に起動し、ターゲットを設定する必要があります。

ターゲットに接続する

現行デバッガセッション中、デバッガが接続されているターゲット GameCube を変更できます。新規ターゲットが確実に接続され次第、.elf ファイルをロードして下さい(38ページの「ELF ファイルのロード、実行」参照)。

1. **デバッグ**メニューの**ターゲットを選択する**をクリックすると、以下のダイアログが現れます。



これがデバッガを初めて起動した時、または-t オプションを使用した時、もしくはターゲット名がない時に表示される画面です。Target Manager で設定したターゲットセッションのリストを表示します。

使用中のターゲットは赤の×印、使用可能なターゲットは黒で表示されます。また、既に使用しているターゲットは、ターゲットアイコン上が LED でハイライトされます。

2. 最新の接続、および接続解除状況を表示するには**リフレッシュ**をクリックします。

3. このリストから使用可能なターゲットを選択し、**OK** をクリックします。

   デバッガがただちに新規ターゲットに接続(可能であれば)します。必要な ELF ファイルを(**デバッグ**>**リセットしてリロードする**)ロードして下さい。デバッガが新規ターゲットに接続できない場合はダイアログが現れ、新規接続できず、デバッガが現行ターゲットに接続されていることを示します。

4. 一度ターゲットセッションを選択すれば、次回からデバッガが起動した際、デバッガ終了時に作業していたターゲットに自動的に接続します。

> **注**: 接続中の GameCube ターゲットを変更した際、以前のターゲットに接続されたままである可能性があります。Target Manager でターゲットの接続ステータスを確認し、必要なくなったセッションは 接続解除して下さい。

GameCube でデバッグ中の複数ユーザ

デバッグ目的では、GameCube には一回あたり1人のユーザしか接続できません。デバッガが接続しようとしている GameCube に他のユーザが接続している場合は、別のセッションを選択するよう、使用可能な他のターゲットセッションを表示します。

ProDG Target Manager では。現在接続中のユーザのアイデンティティを閲覧できます。そのターゲット GameCube の使用を継続したい場合は、直接そのユーザに交渉して下さい。

ELF ファイルのロード、実行

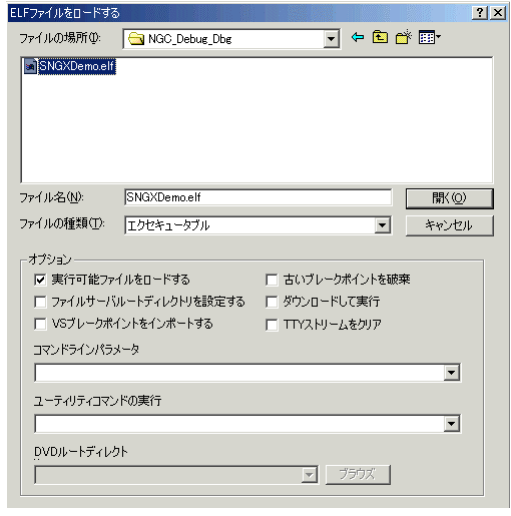
ターゲット上の ELF ファイルをデバッガから直接ロード、実行可能です。コマンドライン上で ELF ファイルを指定していない場合、デバッガを起動して ELF ファイルをロ ード、実行できます。 あるいは新たにビルドされたアプリケーションをロードする場合は、 ターゲットを交換した後にロード、実行できます。

> **注:** Target Manager もしくは ngcrun コマンドラインユーティリティを用いて ELF ファイルをロードすることもできます

アプリケーション ELF ファイルを手動でロードするには

まずデバッガを起動して下さい。

1. **ファイル**メニューの **ELF をロード**ファイルをクリック
2. 表示されたダイアログで、ロード、デバッグする ELF ファイルを配置、選択



3. ファイルのロードに必要なオプションを以下から設定

実行可能ファイルをロードする	シンボル情報と ELF ファイルに含まれるアプリケーション実行ファイルのロードを示す
ファイルサーバーのルートディレクトリを設定する	ELF ファイルディレクトリにアプリケーションファイルサービングディレクトリを設定
VS ブレークポイントをインポートする	ELF ファイルがターゲットにリロードされると、Microsoft Visual Studio 内のソースで設定 された 全てのブレークポイントをデバッガにインポートするよう指定
古いブレークポイントを破棄	ProDG デバッガでプログラムに設定されていた全てのブレークポイントを破棄するよう設定

ダウンロードして実行	ロードされ次第 ELF ファイルを実行するよう設定。メインルーチンまで実行というデバッガのデフォルト動作を置き換え
TTY ストリームをクリア	ELF ファイルをロードする前に全ての TTY ストリームを空にする。
コマンドラインパラメータ	GameCube アプリケーション用のコマンドラインパラメータを指定
ユーティリティコマンドの実行	ELFファイルを実行するのに使われるいろいろなスイッチをコマンドラインへ渡す。利用可能なスイッチについては、利用しているツールのドキュメントを参考のこと。
DVD ルートディレクトリ	DVD エミュレーションドライブとして使うディレクトリを指定する

4. **開く**をクリックして、ロード開始

   ファイルがロードされ、選択したオプションに応じて「メインまで実行」されるか、ターゲット上で実行されます。その後、新たにロードしたアプリケーション ELF ファイルのデバッグが可能になります。

Nintendo GameCube 上でゲームを実行

デバッガを起動して ELF ファイルをダウンロードすれば、ターゲットの GameCube 上で ELF ファイルを実行できるようになります。備付けのコマンドを使えば、いつでもアプリケーションを実行開始、または中止できます。

また、ブレークポイントをソースや逆アセンブリで設定したり、GameCube を再起動したり、アプリケーションをシングルステップで実行したい場合もあると思われます。以下のセクションで、**デバッグ**メニューのターゲット制御コマンドを使用して、アプリケーションの実行を開始または中止する方法を解説します。ブレークポイントやアプリケーションのステッピングを設定する方法については、47ページの「ブレークポイントの使用」をご覧下さい。

ターゲットを再起動するには

アプリケーションは、いつでも再起動できます。アプリケーションを再起動すると、ターゲット GameCube をリセット、ELF ファイルをリロード、アプリケーションが再度メインまで実 行されます。

- **デバッグ**メニュー、またはツールバーの**リセットしてリロードする** をクリックします。

これは GameCube ターゲットを変更したり、ただ単にターゲットを最初のロード状態にリセットする必要がある際に特に有効です。

Nintendo GameCube 上でアプリケーションを実行するには

ターゲット上アプリケーションが実行中止した場合 (ブレークポイントに到達したため、メインサブルーチンでストップしたため、など)、デバッガから再実行可能です。

プログラムカウンタからアプリケーションを再起動するには、

- **デバッグ**メニューの**実行**をクリックするか、ツールバーの**ターゲットを開始**をクリック

Nintendo GameCube 上でアプリケーション実行開始、レジスタペインに「実 行中」が 表示されます。

Nintendo GameCube 上でアプリケーションを中止するには

ブレークポイントや例外でアプリケーションが実行中止していない場合は、以下の手順に従っていつでも手動で中止できます

- **デバッグ**メニューの**停止**、またはツールバーの**ターゲットを中止** をクリック

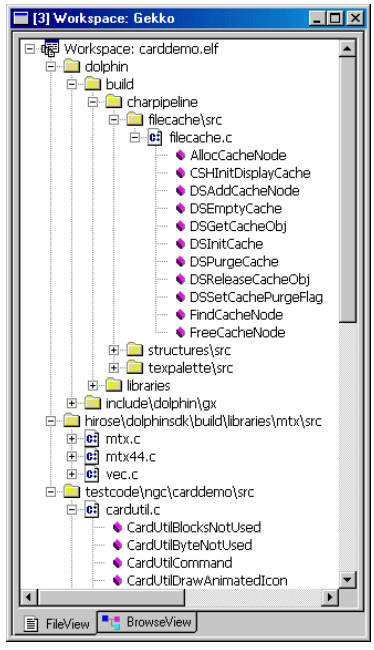
  ターゲット上でアプリケーション実行中止すると、その他の開いているペインが更新され、新プログラムカウンタ位置に現在の情報が表示されます。

ソースペインや逆アセンブリペインで **PC へ移動する**コマンドを使用すると、プログラムカウンタの現在位置を閲覧可能です。また、ソースペインか逆アセンブリペインのどちらかをアクティブにしておくと、ターゲットが実行中止した時に自動的に内容が更新され、現在のプログラムカウンタの位置を表示できます。

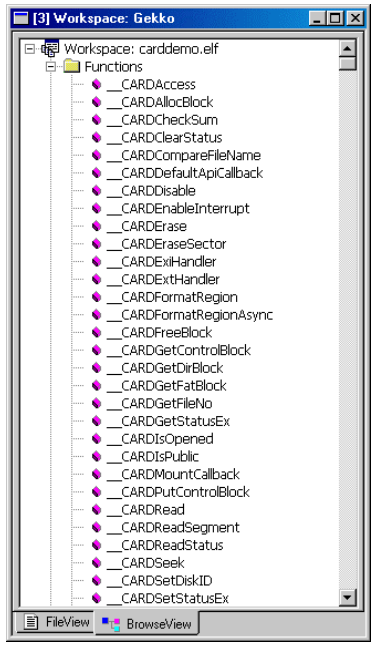
注: ソースペインには、オリジナルのソースコードに正しくマップされない限りプロ グラムカウンタは表示されません。

ワークスペースペインを使用する

ワークスペースペインによって、ELF プロジェクトのファイル、機能およびグローバル変数をツリー状ビューに表示できます。このワークスペースペインには[ファイルビュー]と[ブラウズビュー]の 2 種類があり、ペイン下部のタブから選択できます。



[ファイルビュー]タブを選択したワークスペースペイン



[ブラウズビュー]タブ を選択したワークスペースペイン

ファイルビュー

現在ロードされている ELF モジュールのプロジェクト名を含む各アイテムを 1 つ以上、ルートレベルでツリービューに表示します。プロジェクトのディレクトリレイアウトは、これらプロジェクト項目内で複製されます。しかし、単一のサブフォルダだけを含む

複数のフォルダは、散乱を緩和するために連結されています。ツリーの各ノードは個々のプロジェクトソースファイルを検出するため拡張／縮小できます。プロジェクト項目はモジュールとしてツリーに追加され、ロード／アンロードされます。

ブラウズビュー　プロジェクト内の全てのクラスタイプ、グローバル関数およびグローバル変数をブラウズする際に使用する。各クラスには方法および静的メンバーに関するサブアイテムが含まれる。[ブラウズビュー]はクラス、関数、および変数という名称のフォルダを含み、全てのクラスタイプ、グローバル関数およびグローバル変数が各々のフォルダに含まれる。

フォルダの内容を見る場合、例えば ELF 内の関数リストを表示させる場合、[+] 記号をクリックしてフォルダを拡張できます。クラスフォルダを拡張すると、プロジェクト内の全てのクラス定義が表示されます。各クラス定義は順に拡張され、全ての方法および静的メンバー変数のリストが表示されます。

[-] シンボルをクリックすると、フォルダを縮小できます。

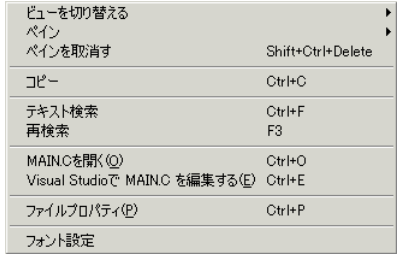
ワークスペースペインショートカットメニュー

ワークスペースを右クリックすると、標準ペイン管理アイテムの他にワークスペースプロパティオプションを含むショートカットメニューが表示されます。その他のいずれかのフォルダを 右クリックすると、ペイン管理アイテムのみを含むショートカットメニューが表示されます。

拡張されたフォルダのリストアイテムのいずれかを右クリックするとより詳しいショートカットメニューが表示されます。標準ペイン管理メニューアイテムの他に、現在選択しているアイテムに関する操作を実行するメニューも含まれます。

> **注:** ファイルに関連する一連のメニューオプションは、ウィンドウの背景を右クリックしない限り表示されません。

[ファイルビュー]アイテムでは、ショートカットメニューに新しいソースペインでファイルを開く、または Visual Studio でファイルを開く操作が含まれます。 [ブラウズビュー]アイテ ムでは、ショートカットメニューがアイテムのタイプによって変化します。



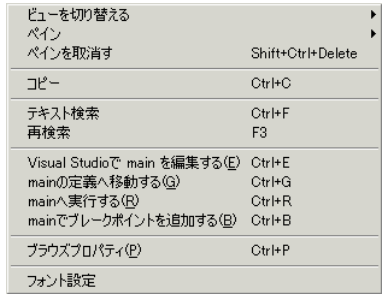
ファイルビューショートカットメニュー – ファイルを選択

コピー　ペイン全体をプレーンテキストとしてクリップボードにコピーする。

テキスト検索　特定のテキストを含むアイテム（ファイル、クラス、関数、変数）を検索できるダイアログを開きます。テキストが検出さ

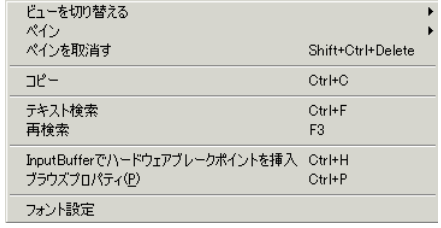
れたら、そのアイテムを含む行が現在の選択になります。

再検索	前回のテキスト検索コマンドを繰り返します。
<file>を開く	最後に選択したソースペインで選択したファイルを開く。ソースペインが使用できない場合は、他のペインを変更してソースを表示させることが可能。
Visual Studio で<file>を編集する	Visual Studio で選択したファイルを編集する。
ファイルプロパティ	選択したファイルのプロパティ情報を含むメッセージウィンドウを表示する。



ブラウズビューショートカットメニュー – 関数を選択

コピー	ペイン全体をプレーンテキストとしてクリップボードにコピーする。
テキスト検索	特定のテキストを含むアイテム（ファイル、クラス、関数、変数）を検索できるダイアログを開きます。テキストが検出されたら、そのアイテムを含む行が現在の選択になります。
再検索	前回の**テキスト検索**コマンドを繰り返します。
Visual Studio で<function>を編集する	**<function>定義へ移動する**と同様の操作を行なうが、ソースファイルを Visual Studio にロードする。
<function>の定義へ移動する	関数を含む新しいソースペインを開き、ロードし、関数の先頭にカーソルを置き、必要に応じてファイルを下方にスクロールする。
<function>へ実行する	プログラムを実行し、プログラムカウンタが関数の先頭に到達した時にデバッガ内で停止する。
<function>でブレークポイントを追加/削除する	選択した関数にブレークポイントを追加する。ブレークポイントが関数上に設定されている場合、メニューテキストは **<function>でブレークポイントを追加する** から、**<function>でブレークポイントを削除する**に変わる。
ブラウズプロパティ	選択した関数のプロパティ情報を含むメッセージウィンドウを表示する。



ブラウズビューショートカットメニュー – 変数を選択

コピー	ペイン全体をプレーンテキストとしてクリップボードにコピーする。
テキスト検索	特定のテキストを含むアイテム(ファイル、クラス、関数、変数)を検索できるダイアログを開きます。テキストが検出されたら、そのアイテムを含む行が現在の選択になります。
再検索	前回のテキスト検索コマンドを繰り返します。
<variable>でハードウェア ブレークポイ ントを挿入	ハードウェアブレークポイントダイアログボックスを表示し、 変数名フィールドに該当する<variable(変数)>名を入力する。
ブラウズプロパティ	選択した変数のプロパティ情報を含むメッセージウィンドウを表示する。

これらのコンテキストメニューアイテムには全て、**設定**メニューの**アクセラレータ**タブから、ユーザが設定可能なキーボードアクセラレータが含まれています。

プロパティのダイアログはトグルウィンドウになっています。すなわち、ダイアログが開くと、コンテキストメニューアイテムがチェックされ、再びコンテキストメニューを選択するか、ダイアログの[閉じる]ボタンをクリックするまでダイアログが画面に表示されます。ダイアログが表示されている間、デバッガは通常通り機能し、メニューおよびツールバーへのアクセスも可能です。ワークスペースペインで選択を変更すると、プロパティダイアログが更新され、新しい選択が表示されます。

このコンテキストメニューに加え、アイテムをダブルクリックして、選択したファイルあるいは定義に[Goto(移動)]できます。

キーボードもまた、ワークスペースペインでの操作に使用できます(アイテム変更のためのアップ、ダウン、破壊および拡大のための左右移動、ダブルクリックの代わりとしての<Enter>など)。

ワークスペース内の[ブラウズ]アイテム上に、少しの間カーソルを置いたままにすると、そのアイテムの評価が行なわれます。それによってそのアイテムのアドレスまたは現行値、あるいはその両方が表示されます。

オプション

ペインの機能性を強化するため、ワークスペースペインにはペインを有効／無効にできるいくつかのオプションがあります。これらオプションは[ワークスペース]セクション内の実行可能な ngcdbg.exe と同じディレクトリにある環境設定ファイル ngcdbg.ini に収められています。デフォルト値は以下のとおりです。

[ワークスペース]
CheckFileExists=0
ShowNonExistentFiles=1
ShowFunctionsInFiles=1

CheckFileExists オプション
これを有効にすると、FileView 中のプロジェクト項目が初めて拡張された際、ファイルがあるかどうか判断するため、ペインは各ファイルを開けようとします。これを無効（デフォルト）にすると、そのような確認は行われず、ファイルはすべて存在するものとペインは見なします。

注:これを使用すると、プロジェクト項目の最初の拡張速度が低下します。（特にファイルがネットワークドライブ上にある場合）

ShowNonExistentFiles オプション
このオプションは CheckFileExists オプションが有効な場合のみ、効果があります。ShowNonExistentFiles が「0」に設定されていると、検出されないファイルはどれも FileView に追加されません。この機能は、インストールされていない、すべての第三者のライブラリーソースファイルを廃棄します。ShowNonExistentFiles が「1」に設定されていると、すべてのソースファイルが FileView に追加されます。しかし、これらファイルを見分けるため、ファイルの名前は他のファイル名より明るい色で表示されます。さらに、実在しないファイルだけが入ったフォルダも他のフォルダより明るい色で表示されます。

ShowFunctionsInFiles オプション
これが有効な場合、ファイルビュー内に表れたソースファイル項目は、その内部で宣言された機能であれば、どれでも拡張して表示することができます。ファイル内で機能が宣言されなければ、拡張／縮小制御は消滅します。

[このページは意図的に空白にされています]

第 4 章: ブレークポイントおよびステッピング

ブレークポイントの使用

コード内のバグを追跡する際、ターゲット上でのコードの実行を完全に制御しつづけることがカギとなります。このセクションでは、コードにブレークポイントを設定する方法と、コードをステップする手順について解説します。

- ターゲット上でプログラムの実行を自由に開始・中止できます (39ページの「Nintendo GameCube 上でゲームを実行」参照)。
- ターゲット上でプログラムの実行を自由に開始・中止できます (39ページの「Nintendo GameCube 上でゲームを実行」参照)。
- デバッガで、プログラムのソースまたは逆アセンブリビューのいずれかにブレークポイントを設定すると、実行パスのどこで実行しているプログラムも、インタラクティブに中止させられます。
- ターゲット上のアプリケーションを手動で中止させたり、その他の理由(ブレークポイントなど)でアプリケーションが中止した場合、それ以降は実行パスのコード 1 行を 1 度に実行できます。
- アドレス(またはアドレス域)にアクセスされるごとに、トリガーするメモリアクセスまたはハードウェアブレークポイントを設定できます(49ページの「メモリアクセス、またはハードウェアブレークポイント」参照)。
- 条件が TRUE(正)かどうかにかかわらず、条件付きブレークポイントを設定できます(51ページの「条件付きブレークポイント」参照)。

コンパイル時のブレークポイントの設定

故意に不法、またはブレークポイントオプコードをソースコードに埋め込んで、CPU 例外でプログラムを一時停止し、デバッガで視覚的に確認するという方法があります。このような方法で一時停止されたプログラムでは、例外時にアクティブだった領域をデバッガで検証(すなわちコールスタック、ローカルおよびグローバル変数、メモリなど)できます。

この時点からプログラムの実行を続けようとする場合、ブレークポイントオプコードが予め PC を繰り上げるように設定しないと、すぐにまた一時停止してしまいます。これは通例不法指令というよりも良くある必要事項で、このプロセスを自動的に処理するため特定の "break"オプコードを設けてあります。お使いのプログラムが"break 1"オプコードで一 時停止した場合は、デバッガの実行をクリックして下さい。その後デバッガが自動的に PC を break オプコードに繰り上げ、実行を再開します。ステートメント使用して、"break 1"をご使用の C ソースコードに埋め込むこともできます。

```
asm("break 1");
```

ブレークポイントの設定閲覧

プログラムが C や C++、またはアセンブリのコードの指定行に到達した時に、ブレークポイントはをプログラムカウンタプログラムの実行を中止したい場合に設定します。

アプリケーションコードのブレークポイントの位置は、デフォルトでは dbugngc.ngc 設定ファイルで、セッション間で保存されます(デバッガを同じ.elf ファイルで起動し、かつそれがデバッガコマンドラインで指定され ている場合)。次のセッション間にブレークポイン トを保存したくない場合は、-b コマンドラインオプションを使用する必要があります(6ページの「ngcdbg のコマンドライン書式」参照)。

さらに、デバッガコマンドラインで-vs オプションを使用することにより、Visual Studio のブレークポイントをインポートまたはエクスポートするよう指定することもできます。

ソースまたは逆アセンブリにブレークポイントを設定するには

1. ターゲットが動作していないことを確認し、ソースペインまたは逆アセンブリペインを開く

2. 閲覧しているのが目的の Nintendo GameCube ソース、または逆ア センブリであることを(ショートカットメニューを通じて)確認

3. ブレークポイントを設定したいソースまたは逆アセンブリの行へスクロールします。

4. 目的の行をダブルクリックするか、ショートカットメニューの**ブレークポイント**をクリックします。その場所にすでにブレークポイントが設定されている場合、その設定は解除されます。ブレークポイントが設定されていない場合は、その行に新しいブレークポイントが設定され、行の色が変わって、ブレークポイントが設定されていることを示します。

> **注**: プログラムカウンタが停止できない行(コメントなど)にブレークポイントを設定した 場合、コードの選択した行の次に来る停止可能な行にブレークポイントが設定されます。さらに、ブレークポイントを正確に設定できない場合や、アプリケーションがプログラムの不適切な場所のブレークポイントで停止する場合は、ソースコードのコンパイル時に最適化が適用されている可能性があります。この問題を解決するには、アプリケーションをリビルドするか、コンパイラ引数 (makefile 内) から-0x フラグを削除する必要があります。

これでターゲットを起動すると、ブレークポイントを設定した行にプログラムカウンタが到達した時点で実行中止し、全てのデバッグ情報ペインが更新されるようになります。さらに、ソースペインまたは逆アセンブリペインが開いており、かつアクティブ状態である場合は、現在のプログラムカウンタの位置を表示するよう更新されます。

プログラムを 1 ステップずつ実行する

上ターゲットで実行している Nintendo GameCube のプログラムは、1 ステップずつ実行できます。これは、逆アセンブリ、ソースのいずれでも実行可能です(ソースを現在のプログラムカウンタで閲覧できる場合)。

プログラムを 1 ステップずつ実行するには

1. ターゲットが動作していないことを確認して、ソースペインまたは逆アセンブリペインを開きます。

2. ペインショートカットメニューの**ステップ**または**ステップオーバー**コマンドを使 用して、(選択したペインにより)ソースまたは逆アセンブリを 1 ステップずつ処理します。**ステップ**コマンドを選択すると、任意の関数コールまでプログラムを実行し、呼び出された関数の中にあるコードの行を実行します。

> **注: デバッグ**メニューの**ステップ**、**ステップオーバー**コマンドを使用することも できます。逆アセンブリペインがアクティブ状態になっている場合は、逆アセンブリを実行します。そうでない場合は、アプリケーションのソースを実行します(何らかのソース情報がある場合)。ソースを実行できない場合は、逆アセンブリを実行します。

逆アセンブリペインとソースペインのショートカットメニューには、他にもプログラムを**実行**できるメニューがあります。これらのメニューについて詳しくは、65ページの「」、および55ページの「」をご覧下さい。たとえば、**カーソルまで実行する**コマンドを使用すると、現在のカーソル位置までプログラムを実行できます。

現在の関数からステップアウトするには

現在関数を 1 ステップずつ処理している場合、その関数のコードの残っている行全てを実行しなくても、関数を呼び出したソースの行に戻れます。

- **デバッグ**メニューの**ステップアウト**、またはツールバーの**現行関数をステップアウト**をクリックします。

プログラムカウンタは、関数コールの次のコード行まで進みます。

但し、関数が終了するまでのコードに(またはそれ以前に呼び出された関数に)ブレークポイントがある場合、ペインが切り替わって、次にブレークポイントが設定された行で一時停止する場合があります。

メモリアクセス、またはハードウェアブレークポイント

ハードウェアブレークポイントは Gekko processor 当り一つ設定できます。また、ハードウェアブレークポイントは、指定のアドレスまたはアドレス域がアクセスされた時すぐプログラムを一時停止するよう指定できます。

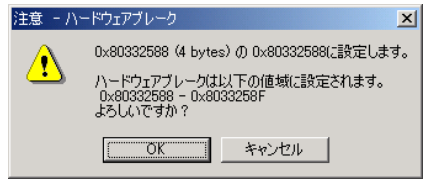
ハードウェアブレークポイントがトリガーされると、プログラムの一時停止などのように、ソフトウェアブレークポイントと似たような動作をします。

ニンテンドーゲームキューブでは、ハードウェアブレークポイントが特定の C 変数がアクセスされた時にトリガーするか、アセンブリアドレスがアクセスされた時にトリガーするかのいずれにも設定できます。ハードウェアブレークポイントの設定には適切なコマンド、**[Gekko ハードウェアブレーク(C 変数)]**もしくは**[Gekko ハードウェアブレーク (Asm レベル)]** を選択して下さい。両者の違いは **C 変数と Asm レベル**です。ハードウェアブレー クが C モードの時、デバッガが対処するにせよ、実際の処理は必ずしも優れたものでは

ありません。そこで、アセンブリモードをハードウェアブレークポイントの制御を容易にするために用意しました。

ハードウェア上の制約のため、Gekko ハードウェアブレークポイント8バイト整列の8バイト域にのみ設定できます。

ハードウェアブレークポイントを設定する際、これらの要件を満たさない場合は、デバッガがブレークポイントの設定が不可能である旨伝えると共に、別の手段を示唆します。



ハードウェアブレーク域内部データへのアクセスは、そのアクセスが要求された変数かに関わりなくハードウェアブレークを設定します。

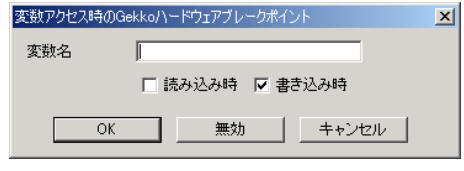
		8 バイトハードウェアブレーク域	
int temp; //(align 8)		XXXX????	
int temp; //(align 4)		????XXXX	
int temp[2]; //(align 4)	****	XXXX????	
int temp[3]; //(align 8)		XXXXXXXX	****

X：関数データはハードウェアブレイク内
?：道のデータがハードウェアブレイク内
*：関数データははハードウェアブレイク外

C 変数ハードウェアブレークポイントの設定

ハードウェアブレークポイントが、C 変数がアクセスされた時にトリガーする(C モード)か、特定のアドレスが打込まれたときにトリガーするか（アセンブリモード）のどちらにも設定できます。

1. **デバッグ**メニューから **Gekko ハードウェアブレーク(C 変数)**をクリックし て下さい。以下のようなダイアログが表示されます。



C モードでのハードウェアブレークポイント設定ダイアログ

2. アプリケーションの変数名 C コードを**変数名**フィールドに入力して下さい

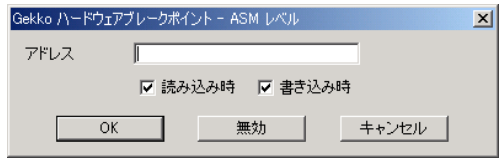
3. 必要であれば**読み込み時**または**書き込み時**オプションを選択して下さい

4. **OK** をクリックしてハードウェアブレークポイントを設定、有効にして下さい

   これで指定変数がアクセスされつつ、ブレークポイントで設定したその他の基準が全て満たされている場合は、お使いのプログラムをターゲット Nintendo GameCube で実行の際、一時停止するようになります。

アセンブリレベルでのハードウェアブレークポイントの設定

1. **デバッグ**メニューの **Gekko ハードウェアブレーク（Asm レベル）** をクリックして下さい。以下のようなダイアログが表示されます。



アセンブリモードのハードウェアブレークポイント設定ダイアログ

2. **アドレス**フィールドに絶対アドレスを入力して下さい

3. 必要であれば**読み込み時**または**書き込み時**オプションを選択して下さい

4. **OK** をクリックしてハードウェアブレークポイントを設定、有効にして下さい

   これで指定変数がアクセスされつつ、ブレークポイントで設定したその他の基準が全て満たされている場合は、お使いのプログラムをターゲット GameCube で実行の際、一時停止するようになります。

条件付きブレークポイント

条件付きブレークポイントは他のソフトウェアブレークポイント同様、トリガーされた時にターゲットを停止します。なお、条件付きブレークポイントは、*条件*または*ヒットカウント*に基づいてブレークポイントがターゲットを停止する場合、ユーザーが選択できるという利点があります。

条件は、式評価機能により認められた式です。設定が済み次第、ブレークポイントには条件が TRUE（正）の時のみ、その条件が適用されるようになります。条件が FALSE（誤）と判断された場合、ターゲットは実行を継続します。

別の方法として、ブレークポイントヒットのタイミングを指定するヒットカウントがあります。まず、回数に対するヒットカウントを設定した後、以下の起こり得る3事象の1つに対してチェックします。

- ヒットカウントと同じ回数ブレークポイントがヒットされた場合
- ヒットカウントの乗数と同じ回数ブレークポイントがヒットされた場合
- ヒットカウント以上の回数ブレークポイントがヒットされた場合

ヒットカウントと条件は相互排除的ではないため、どのブレークポイントにおいても条件とヒットカウントの両方を設定できます。この場合は、ブレークするためにはヒットカウントおよび条件の両方とも TRUE（正）である必要があります。

条件付きブレークポイントを設定する

1. 通常のソフトウェアブレークポイントを設定します。詳しくは48ページの「ブレークポイントの設定閲覧をご覧下さい。

2. ブレークポイントペインを開き、ブレークポイントを選択します。詳しくは52ページの「ブレークポイントペイン」をご覧下さい。

3. ブレークポイントされた行を右クリックし、ショートカットメニューを表示します。

4. ブレークポイントプロパティオプションを選択し、ブレークポイントプロパティダイアログからブレークポイントの条件またはヒットカウント、もしくはその両方を設定します。

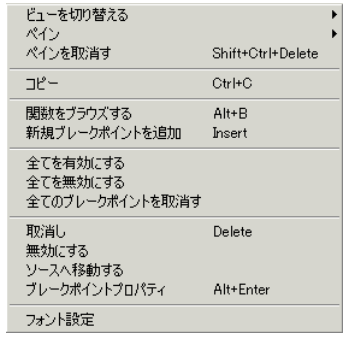
   これでブレークポイントは条件付きブレークポイントとして設定されました。

ブレークポイントペイン

アプリケーションのソースや逆アセンブリに設定された全てのブレークポイントは、ブレークポイントペインでいつでも閲覧できます。このペインではブレークポイントが設定されている位置と、ブレークポイントが設定されている機能が一覧表示されています。このペインで、ブレークポイントを追加したり、選択したブレークポイント、またはブレークポイント全てを削除できます。

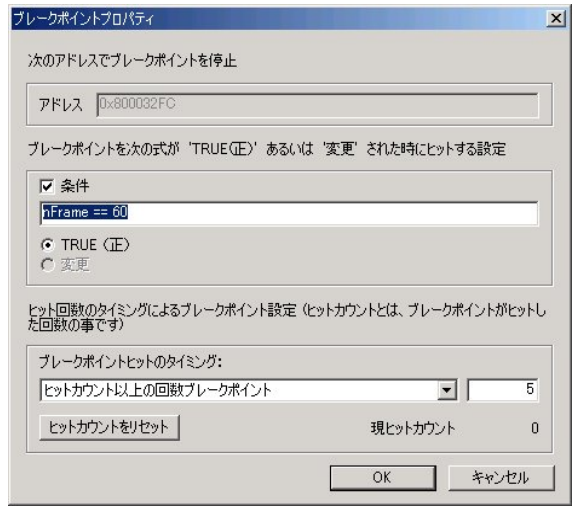
ブレークポイントペインは、最初に開いた時点では空白ですが、ブレークポイントが設定・解除されると、直ちに更新されます。矢印は、プログラムが現在どのブレークポイントで停止しているのかを示すために使用されます。

ブレークポイントペインショートカットメニュー



コピー	ペイン全体をプレーンテキストとしてクリップボードにコピーする。
関数をブラウズする	[関数ブラウザ]ダイアログを表示する。このダイアログでは、クラス、メソッド、過負荷状態の名称とアドレスの、アルファベット順の各リストを含むドロップダウンリストボックスから関数を選び、ブレークポイントの解除、または設定が可能。
新規ブレークポイントを追加する	[アドレスを入力]ダイアログを表示して、ブレークポイントを設定するアドレスを指定できるようにする。
全てを有効にする	全てのブレークポイントを有効にする。
全てを無効にする	選択されたプロセッサに関する全てのブレークポイントを削除する。
全てのブレークポイントを取消す	現在のプロセッサについて、全てのブレークポイントを削除する。
取消し	指定のブレークポイントを削除する。
無効 / 有効にする	指定のブレークポイントを無効、または有効にする。
ソースへ移動する	ソース／逆アセンブリペイン内のそのブレークポイントに移動する。
ブレークポイントプロパティ	[ブレークポイントプロパティ]ダイアログを表示する。54ページの「ブレークポイントプロパティダイアログ」参照。

ブレークポイントプロパティダイアログ



ブレークポイントペインのショートカットメニューから**ブレークポイントプロパティ**を選択した際、上記プロパティダイアログが表示されます。条件付き式を入力したり、ヒットカウントの条件を設定したり、現在のヒットカウントをゼロに戻したりできます。

アドレス	ブレークポイントアドレスを表示
条件	条件を設定する時はこのチェックボックスにチェックを入れる。式入力ボックスに、ブレークポイントに到達した際評価する式を入力
ブレークポイントヒットのタイミング	ヒットカウントを設定する時は, ヒットカウント評価形式およびヒットカウントターゲット値(ブレークポイントをトリガーするために比較するヒットカウントを入力
ヒットカウントをリセット	現在のヒットカウントをゼロに戻す
現ヒットカウント	現在のヒットカウントを表示

第 5 章: ソースおよび逆アセンブリを閲覧する

ソースを閲覧する

ソースペインが開いていると、ターゲットが停止する度にペインの内容が更新され、アプリケーションソースのプログラムカウンタの現在位置が表示されます (➪ というマークで表示)。しかし、プログラムカウンタがソースで閲覧できない場合もあり、その場合はソースペインは更新されません。

デバッガは、アプリケーションのソースファイルがデフォルトの場所(アプリケーションの `.elf` ファイルがビルドされた時の場所)にあれば、開く必要のあるソースファイルを検索できます。しかし、ソースファイルを後で移動した場合は、デバッガに追加の検索パスを指定する必要があります (**デバッグ**メニューの**ソース検索パス**)。

プログラムのソースを閲覧するには

現在のプログラムカウンタの位置がソースコードで閲覧できる場合は、ソースペインを開くだけでソースを閲覧できます。

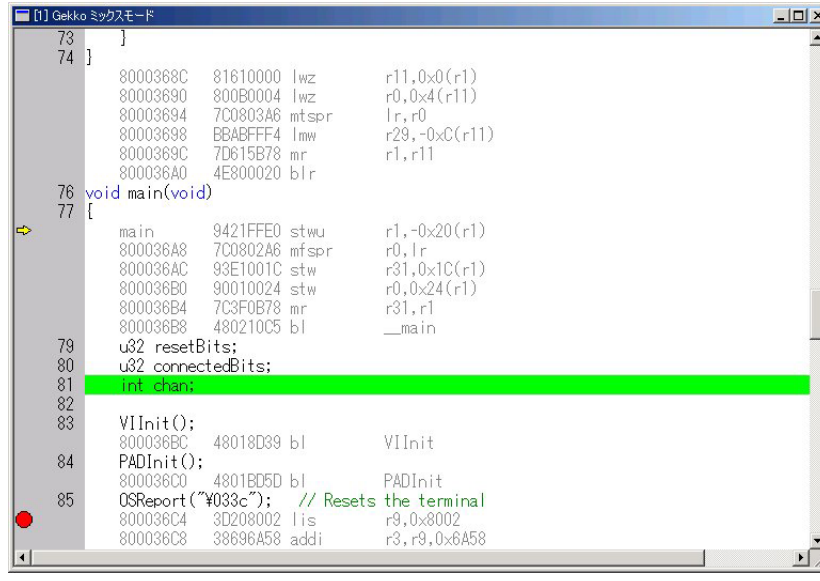
- **ウィンドウ**メニューの**新しいウィンドウを開く→ソース**をクリックするか、ツールバーの**ソースファイルビュー**ボタンをクリックします。

  ソースペインを含む新しいウィンドウが開き、現在のプログラムカウンタが➪というマークで示されます。プログラムカウンタが現在ソースで閲覧できない場合、またはソースファイルが見つからない場合は、"ソースファイルがロードされていません" というメッセージを含むペインが表示されます。

> **注:** ソースペインショートカットメニューの **PC へ移動する**オプションを使用すると、表示をすばやく動かして新しいプログラムカウンタの位置を表示できます。

ソースペイン

ソースペインは Nintendo GameCube で現在実行中のソースの閲覧に使用されます。



アプリケーションのソースファイルの場所は、ELF ファイルに記述されています。ソースペインを開くと、デバッガは、プログラムカウンタが元の場所にあるソースファイルを検出します。しかし、この場所が変更されている場合、または ELF ファイルがマシンにビルドされていない場合は、**デバッグ**メニューの**ソース検索パス**オプションを使用して、デバッガがソースファイルを検索・検出できるディレクトリを入力できます。

このペインを開くと、通常、現在ソースファイルのプログラムカウンタがある位置が表示されます。しかし、ソースにプログラムカウンタが表示されない場合は、ソースを表示できないことを示すメッセージがペインに表示されます。

カーソルは、<Ctrl+Home>ショートカットキーを押すとソースコードの最初の行に、<Ctrl+End>ショートカットキーを押すとソースの最終行に、いつでも移動できます。

Nintendo GameCube で一回当り一行のコードを実行してアプリケーションソー スを適宜ステップするか、使用中のアプリケーションのある地点で実行を停止するようブレークポイントを設定します。

ソースペインがアクティブである場合、ターゲットの状態が変わる度にプログラムカウンタが自動的に更新されるため、いつでもプログラムカウンタの現在の場所をソースペインで閲覧できます。

複数行のコメント(C スタイルの/* */マーカー使用)を認識、カラー化されています。

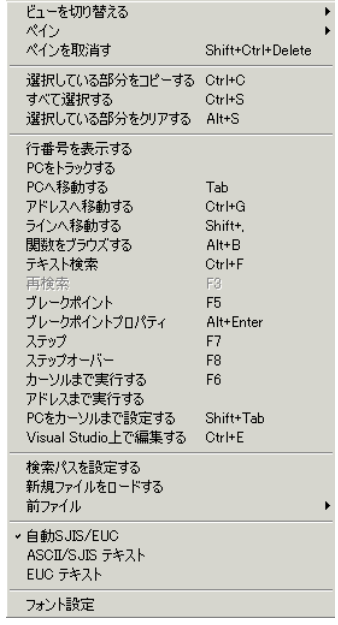
ペインの左側のボーダーに行番号、ブックマーク、ブレークポイントマーカーその他が表示されています。前面／背面色は9ページの「設定メニュー」で変更可能です。

ソースペインの同じ作業位置に後日戻ってくるため、ブックマークを使用することもできます。

スペースバーを押して、ソースペイン、逆アセンブリペイン、ミックスモードペイン(各行に関連する逆アセンブリ命令に差し込まれるソースコードを表示する)を順次切り替えることがで

きます。順序はソースペイン、逆アセンブリペイン、ミックスモードペイン、ソースペインとなります。

ソースペインショートカットメニュー



コピー / 選択している部分をコピーする	ある部分が選択されている時は**選択している部分をコピーする**と表示され、選択部分がクリップボードにコピーされる。それ以外の場合は**コピー**と表示され、ペイン全体がプレーンテキストでクリップボードにコピーされる。
すべて選択する	ソースペイン内の全てのテキストを選択する。
選択している部分をクリアする	選択アイテムをクリアする
行番号を表示する	このチェックボックスをオンにすると、ソースファイルの各行に対して行番号が表示される。
PC をトラックする	このチェックボックスをオンにすると、逆アセンブリペインのプログラムカウンタの場所が自動的に追跡される。
PC へ移動する	ペインで閲覧しているソースを変更して、プログラムカウンタが現在設定されている行を表示するようにする。Nintendo GameCube が停止している時のみ有用。
アドレスへ移動する	閲覧希望の Nintendo GameCube ユニットメモリの特定の アドレス入力を可能にするダイアログを表示する。16 進法のアドレスを入力する場合は、アドレスの前に 0x をつける必要がある。必要なアドレス、またはシンボルを入力して、閲覧したいメモリの部分を指定できる。
ラインへ移動する	ソースを変更して、指定行番号を中心に表示するようにする。

関数をブラウズする	アプリケーションの関数に沿ってソースを閲覧できるダイアログを開く。さらに、このダイアログからアプリケーションの特定の関数にブレークポイントを設定したり、ブレークポイントを解除できる。
テキスト検索	ソースファイルの特定のテキストを検索できるダイアログを表示する。テキストが検出されたら、そのテキストを含む行がカーソルで示される。
再検索	前述の**テキスト検索**コマンドを繰り返す。
ブレークポイント	ソースペインの現在選択されているコード行に、ブレークポイントを設定できる。このコマンドを再度実行すると、ブレークポイントが解除される。ブレークポイントが設定されている行は、赤色の帯状線で示される。
ブレークポイントプロパテイ	ブレークポイントプロパティダイアログを表示する。54ページの「ブレークポイントプロパティダイアログ」参照。
ステップ	アプリケーションを 1 ステップずつ実行できる(呼び出されたサブルーチンに移動する)。Nintendo GameCube 2 ユニット で一回当り一つの命令を実行し、ソースコードを適宜ステップするためターゲットを停止してから、このコマンドは使用可。1 ステップ実行する度に、次に実行される命令へプログラムカウンタが移動する。
ステップオーバー	アプリケーションを 1 ステップずつ実行できる(呼び出されたサブルーチンをとばす)。Nintendo GameCube ユニットで一回当り一 つの命令を実行し、逆アセンブリを適宜ステップするためターゲットを停止してから、このコマンドは使用可。1 ステップ実行する度に、次に実行されるコード行へプログラムカウンタが移動する。
カーソルまで実行する	アプリケーションが、Nintendo GameCube ユニットにソースペインで現在指定のアドレスまで到達した行のコードまで実行するよう指定できる。
アドレスまで実行する	アプリケーションが、Nintendo GameCube ユニットに指定のアドレスまで到達した行のコードまで実行するよう指定できる。アドレスは表示されるダイアログで指定し、16 進法で入力する場合は、前に 0x をつける必要がある。
PC をカーソルまで設定する	プログラムカウンタを、ソースペインで現在選択されている行に設定できる。「>」記号が指定行へ移動し、プログラムカウンタがリセットされたことを示す。新しいプログラムカウンタまでの間にある行は実行されないので注意。
Visual Studio 上で編集する	Microsoft Visual Studio で現在のソース行を検出する。この作業を行なうには、あらかじめ Visual Studio IDE を起動させておく必要がある。
検索パスを設定する	アプリケーションソースファイルの検索パスにディレクトリを追加できる。複数のパスを追加する場合は、各ディレクトリをセミコロンで区切る必要がある。これ、ELF ファイルをビルドした時と同じ場所にソースファイルを保存していない場合や、ELF ファイルをビルドしなかった場合に役立つ機能である。このコマンドは、**デバッグ**メニ

ューの**ソース検索パス**オプションと同じ働きをする。

新規ファイルをロードする	このコマンドにアクセスしているソースペインに、別のソースファイルをロードするよう指定できる。[ファイルを開く]ダイアログが表示され、ロードするソースファイルを選択できる。ロードするファイルを検出すると、そのファイルが、ソースペインにもともと表示されていたファイルに代わって表示される。このコマンドは、常に新しいソースペインに指定のソースファイルを開く**ファイル**メニューの**ソースファイルをロード**コマンドとはやや異なる。
前ファイル	これまでロードされた 10 のソースファイルをサブメニューに表示する。任意のファイルにクリックしてロード可能。Ctrl+P で、ソースペインの左上端にこれをポップアップメニューとして表示。
自動 SJIS/EUC	ソースファイルで使われているエンコーディングの検出を試みる。
ASCII/SJIS テキスト	エンコーディングの検出を無効にし、指定された Shift-JIS か標準 ASCII テキストかを指定する。
EUC テキスト	エンコーディングの検出を無効にし、指定された EUC (Extended Unix Code)テキストを指定する。

ソース内の検索

テキストエディタでソースファイル内を移動するのと同じように、ソースの別の場所を閲覧できます。

プログラムソース内を移動するには

- Windows 標準ショートカットである［PgUp］と［PgDn］を使用して、ソースを 1 ページずつスクロールできます。
- 矢印キーを使用して、ソースを 1 行ずつスクロールできます。

ソースファイルの最初の行にすばやく移動するには、以下の手順に従って[先頭ファイル]アクセラレータキーを使用します。

1. **設定**メニューオプションから**オプション**を選択して、[アプリケーション設定]ダイアログを表示させます。**アクセラレータ**タブが選択されていることを確認します。
2. 右上のリストボックスから **Source** を選択します。
3. **コマンド名**リストを下側にスクロールして **Start of File**（**先頭ファイル**）のアクセラレータキー設定を表示させ、該当する**キーシーケンス** (あれば) を確認します。
4. **キャンセル** をクリックして[アプリケーション設定]ダイアログを閉じます。
5. ファイルの先頭に移動するために必要なキー順序を押します。

ソースファイルの最終行にすばやく移動するには、以下の手順に従って[ファイル末]アクセラレータキーを使用します。

1. **設定**メニューオプションから**オプション**を選択して、[アプリケーション設定]ダイアログを表示させます。**アクセラレータ**タブが選択されていることを確認します。

2. 右上のリストボックスから **Source** を選択します。

3. **コマンド名**リストを下側にスクロールして **End of File(ファイル末)** のアクセラレータキー設定を表示させ、該当する**キーシーケンス**（あれば）を確認します。

4. **キャンセル** をクリックして[アプリケーション設定]ダイアログを閉じます。

5. ファイルの終端に移動するために必要なキー順序を押します。

ブックマークを使用してプログラムソース内を移動するには

ブックマーク(しおり)を使用してソースペインの任意の場所に後で簡単に戻ってくることができます。

- <Ctrl+F2>でブックマークを追加します。カーソルのある行の境界線に、青のマーカーが表示されます。

- 異なる部分のソースファイルを閲覧する時は<F2>でブックマークのある行まで戻ります。

- ソースファイルには複数のブックマークを挿入できます。<F2>を押すとカーソルが現在の位置から最後のブックマークに到達するまで、次のブックマークに移動します。最後まで到達すると、ファイルの初頭から検索を開始します。

- カーソルのある場所から逆方向に検索したい時は<Shift+F2>を使用して下さい。

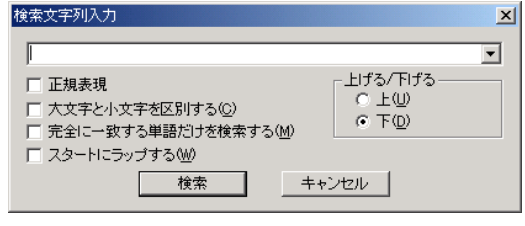
- デバッガを終了すると、全てのファイルのブックマークは設定ファイルに保存され、次回デバッガを実行する時にレストアされます。

ソースファイル内のテキストを検索するには

テキスト検索、**再検索** の各ショートカットメニューオプションを使用して、ソースペイン内で検索文字列を簡単に探し当てられます。必要ならば、大文字・小文字を区別して検索することもできます。

1. テキスト文字列を検索するには、**テキスト検索**ショートカットメニューオプションを使用します。

   [検索文字列入力]ダイアログが表示され、ここで検索文字列を特定できます



2. テキスト入力フィールドには、最近入力された検索文字列を表示する履歴ドロップダウンリスト(最後に入力された文字列が先頭)がついているため、繰り返し検索する文字列をその都度入力しなおす必要はありません。

3. 以下をチェックすることにより、一致する検索結果数を制限したり、増加したりできます。

正規表現	このチェックボックスをオンにすると、検索文字列が正規表現であることを示します。正規表現書式に関する詳細は62ページの「正規表現」をご覧下さい。
大文字と小文字を区別する	このチェックボックスをオンにすると、一致文字列の大文字・小文字が検索文字列と同じでなければ、一致と見なしません。
完全に一致する単語だけを検索する	このチェックボックスをオンにすると、一致文字列が完全一致でなければ (一致するサブ文字列が見つからないなど)、一致と見なしません。
スタート時にラップする	このチェックボックスをオンにすると、検索がファイルの先頭まで来た時は下方向に、ファイルの終端まで来た時は上方向に検索を続けます。
上げる/下げる	これらのラジオボタンは、検索する方向、つまりカーソル位置から上方向に検索するか下方向に検索するかを決定します。

注: これらのオプションは、アプリケーションの継続時間内のみ有効であり、アプリケーション終了時に設定ファイルに保存されることはありません。

文字列が検出されたら、最初に検出された文字列を含むソース行がソースペイン内で強調表示されます。

4. テキスト文字列の検索を続けるには、**再検索**ショートカットメニューオプションを使用します。

ソースコードの編集

デバッガソースペインのコンテキストメニューから **Visual Studio 上で編集する** オプションを実行する場合(または<Ctrl+E>を押した場合) ― 57ページの「"ソースペイン」参照 ― 、デバッガは、このソースファイル を同じ場所で編集 するために外部エディタを起動しようとします。これは、以下の手順で実行します。

1. ProDG Plus スクリプティングの機能を使用する場合(104ページの「スクリプトの使用方法」参照)、デバッガは、まず特定のスクリプト関数を探し、それが検出されれば、エディタを実行するためにその関数を呼び出します。

   適正なスクリプト関数が検出されなかった場合、デバッガは、デバッガ構成ファイル(dbugngc.ngc ― 27ページの「プロジェクト設定を保存する」参照)内の[opentemplate]変数の有無をチェックします。テンプレートが検出された場合、外部エディタをどのように起動かを決定するため、そのテンプレートを使用します。例えば、

```
[opentemplate]
c:\cw32\cw32.exe %f -G%l
```

このテンプレートの場合、%f は編集用ファイルのパス名に置換され、%l は行番号に置換されます。

現在のディレクトリ内に適正なローカル構成ファイルを伴わずにデバッガが起動された場合、デバッガが実行可能となるように同じディレクトリから（同名の）デフォルト構成ファイルをロードしようとするので、そのファイルにも2行を追加するようにしてください。デバッガが現在のディレクトリにローカル構成ファイルを保存して終了するので、その設定は保持されます。

2. スクリプト関数が検出されず、デバッガ構成ファイル内に[opentemplate]変数が検出されなかった場合、デバッガは、ファイル編集のために Microsoft Visual Studio を実行しようとします。

正規表現

正規表現は、文字列検索で使用される文字列のパターンを記述する標準的な方法です。正規表現を作成するには、所定の書式に従う必要があります。

正規表現は、文字、文字列、および特殊記号からなり、算数式に類似しています。また算数式のように、必要に応じて括弧(すなわち(expr))で囲んで正規表現を式のほかの部分から区切ることができます。

- 文字リテラル。これは文字、例えば「A」は正規表現の一部であることを示します。非印字文字は拡張テキストフォーマット文字、すなわち'\ '、 '\n'、 '\r'、 '\t' 、'\v'、あるいは 16 進数とみなされます（「\xnn」など。ここで nn は 16 進数を示します）。'*'、 '\+'、 '\?'、 '\.'、 '\|'、 '\['、 '\]'、 '\('、 '\)'、 '\-'、 '\$'、 '\^' などのの句読点および記号もまた、正規表現書式においては特別な意味をもつために、エスケープ文字となります。

- 文字集合。これは、ありうる値の範囲において一致があることをしめし、式は「[set]」のように角括弧でくくられます。式の集合は、複数のサブセットからなり、例えば、式「[0-9A-Za-z]」はいずれかの英数字文字集 合と一致します。 文字集合を、始めの角括弧のすぐ後に「^(not、否を示す)」を置いて反意に できます。したがって、「[^0-9A-Za-z]」はいずれかの 非英数字文字集 合と一致することになります。特殊文字「.」もいずれかの文字に一致します。

- 連結式。2 つの式を連結して、より大きな式を生成できます。例えば「_[A-Za-z]」は、第 1 の文字がアンダーバー「_」で、第 2 の文字がアルファベットであるいずれかの文字列と一致します。2 つの式は、以下のように OR symbol 「|」で区切られます。

  [0-9]+|[A-Za-z_][A-Za-z_0-9]*は整数あるいはC識別子に一致します。

- 演算子。以下のように様々な特殊記号を使用して、すぐ前の式を変更できます。

  `expr*` `expr` の 0 回の繰り返しに一致する
  `expr+` `expr` の 1 回以上の繰り返しに一致する

`expr?` expr の少なくとも 1 回の発生に一致する
`expr\i` 大文字・小文字の区別に関係なく expr に一致する

- 指定された式。後続の正規表現で使うために、以下のように正規表現を指定できます。

```
$Integer = "[0-9]+"
$CIdntfr = "[A-Za-z_][A-Za-z_0-9]*"
$MyExprn = "$Integer|$CIdntfr"
```

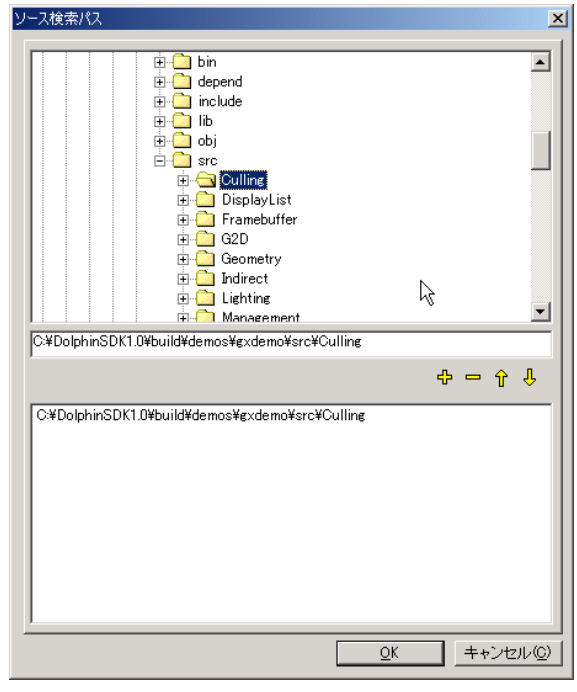
ソースファイルの配置

デフォルトでは、デバッガはアプリケーションのソースファイルを、ELF ファイルがビルドされたときの場所を元に検出します。しかし、ソースファイルが後で移動された場合は、必要なファイルを検出できるよう、新しいパスを指定する必要があります。

ソースファイルを配置するには

1. **デバッグ**メニューの**ソース検索パス**をクリックします。

   下図のような[ソース検索パス]ダイアログが表示されます。



2. 表示されるダイアログのツリー状ビューを使用して、デバッガがアプリケーションのソースファイルを検索するディレクトリの完全パスを選択します。[+] 記号をクリックすると、選択したディレクトリを検索パスに追加でき、[-] 記号をクリックすると、選択したディレクトリを検索パスから削除できます。

3. 上下の矢印をクリックすると、検索パス内でのディレクトリの順序を変更できます。選択したディレクトリの順序を上げるには上方向の矢印を、順序を下げるには下方向の矢印をクリックします。

4. 変更を有効にし、かつソースファイルを検出して開くことができるよう、既存のソースファイル画面を全て終了し、新しい画面を開かなければならない場合あります。

他のソースファイルを閲覧するには

ターゲットで現在実行されているソースコードの行だけでなく、他のどのソースファイルも閲覧できます。

1. **ファイル**メニューの**ソースファイルをロードする**をクリックします。

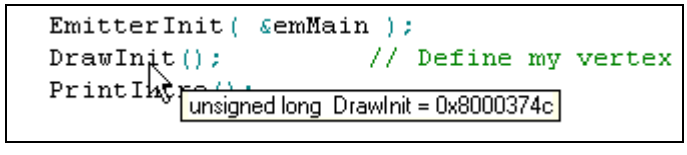
2. 表示されるダイアログで必要なソースファイルを探して選択します。

   新しいソースペインが開き、選択したソースファイルの内容が表示されます。このペインは、プログラムカウンタを含むソースペインと同様に使用でき、このファイルでブレークポイントを設定したり、必要に応じて内容を閲覧できます。

> **注:** ソース検索パスを通じて、そのもともとの場所以外からファイルをロードしている 場合、**ソースファイルをロードする**を使用してファイルをロードした時に、デバッガがオブジェクトコードとの関係を推測できません。この場合**アドレスへ移動する**(ソースペインのショートカットメニュー)を使用して関数名を入力し、ソースコード内の適切な位置へ移動する方が良いでしょう。こうすれば、ソース検索パスとビルド情報から、ファイルを探し当てることができます。

ポップアップヘルプ を使って数式を評価するには

コード行の上にマウスのカーソルを置けば、すぐに数式を 評価できます。カーソルの下にある数式が評価されると、その結果が下図のようなポップアップヘルプに表示されます。



逆アセンブリを閲覧する

逆アセンブリペインが開いている場合、ターゲットが実行中止する度にペインの内容が更新され、アプリケーション逆アセンブリ内のプログラムカウンタの現在位置が表示されます(➪ マークで表示)。

プログラム逆アセンブリを閲覧するには

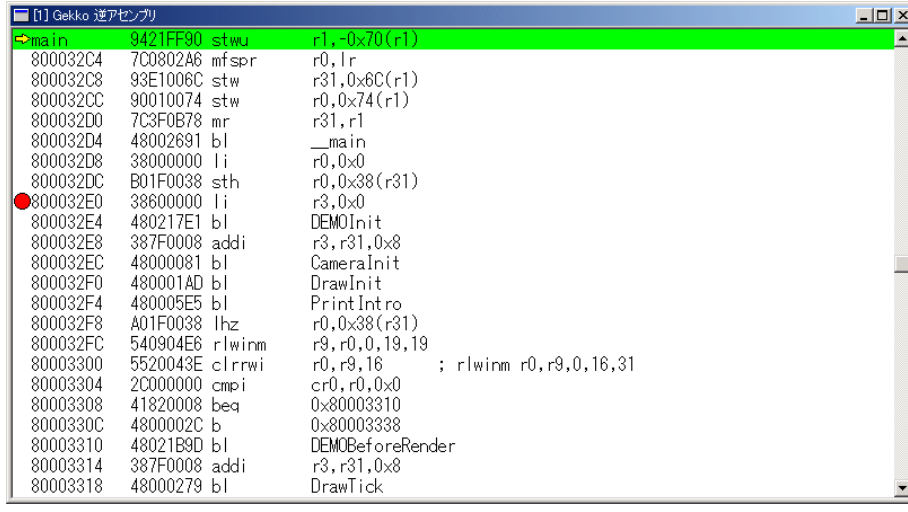
プログラムカウンタの現在位置を含む新しい逆アセンブリペインを開くには、以下の手順に従って下さい。

1. **ウィンドウ**メニューの**新しいウィンドウを開く → 逆アセンブリ**をクリックするか、**逆アセンブリビュー** ツールバーボタンをクリックします。

2. 逆アセンブリペインを含む新しいウィンドウが表示されます。このペインには、現在ゲームキューブ上で実行されているコードの逆アセンブリが表示されます。ショートカットメニューを使用して表示される逆アセンブリの単位を変更したり、**PC へ移動する**コマンドでプログラムカウンタを閲覧できます。

逆アセンブリペイン

逆アセンブリペインでは、ゲームキューブ上のプログラムコードの逆アセンブリを閲覧できます。このペインには、アドレス、オプションコード、逆アセンブリが表示されます。



逆アセンブリペインには、4 つの情報列がありますが、このうち 2 列目は省略可能です。これら 4 つのコラムの表示内容は、以下の通りです。

- 1 列目には、アドレスまたはラベルが表示されます。
- 2 列目には、16 進法のオプションコードが表示されます。
- 3 列目と 4 列目には、逆アセンブルされたプログラム命令が表示されます。

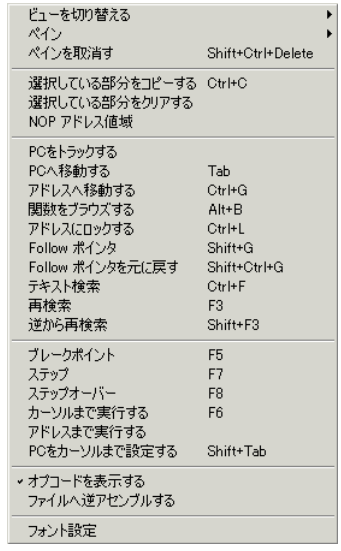
プログラムカウンタの現在位置には、1 列目の前に➪ 記号が表示されます。

ブレークポイントが設定されている行は、違う色で表示されます。

逆アセンブリペインがアクティブである場合、逆アセンブリペインは現在のプログラムカウンタの値を追跡します。

スペースバーを押して、ソースペイン、逆アセンブリペイン、ミックスモードペイン（各行に関連する逆アセンブリ命令に差し込まれるソースコードを表示する）を順次切り替えることができます。順序はソースペイン、逆アセンブリペイン、ミックスモードペイン、ソースペインとなります。

逆アセンブリペインショートカットメニュー



コピー / 選択している部分をコピーする
ある部分が選択されている時は **Copy Selection** と表示され、選択部分がクリップボードにコピーされる。それ以外の場合は **Copy** と表示され、ペイン全体がプレーンテキストでクリップボードにコピーされる。

選択している部分をクリアする
選択をクリアする。

NOP 上書き
現在の選択部にある全てのコードを'nop'命令に置換する。

PC をトラックする
このチェックボックスをオンにすると、逆アセンブリペインのプログラムカウンタの位置が自動的に追跡される。

PC へ移動する
ペインで閲覧する逆アセンブリを変更して、プログラムカウンタが現在設定されている行が表示されるようにする。ターゲット GameCube でアプリケーションが実行中ない時のみ有用。

アドレスへ移動する
閲覧希望の GameCube メモリの特定のアドレス入力を可能にするダイアログを表示する。16 進法のアドレスを入力する場合は、前に 0x をつけるか、デフォルト 16 進法基数チェックボックスをオンにする。必要なアドレスやシンボル、数式を入力して、閲覧したいメモリの部分を指定できる。シンボル名の先頭部分を入力する場合は、自動入力ボタンを使用して、アプリケーション内の表示できるシンボルのリストを表示させることができる。

関数をブラウズする
アプリケーションの関数に応じて逆アセンブリを閲覧できるダイアログを開く。さらに、このダイアログで、アプリケーションの特定の関数にブレークポイントを設定したり、ブレークポイントを解除できる。

アドレスにロックする
逆アセンブリの表示をロックするアドレス数式を指定できる。表示されるダイアログで、必要なアドレスや数式を入力する。表示に設

定されたロックを解除するには、再度[アドレスへロック]ダイアログにアクセスする必要があるが、この時は数式フィールドを空白のままにしておく。

Follow ポインタ 現在の行にあるアドレスに逆アセンブリペインを“ジャンプ”させる。ポインタの数に関係なくフォローできる。

Follow ポインタを元に戻す フォローされたポインタを逆展開し、逆アセンブリウィンドウを前回の場所にジャンプさせる。

テキスト検索 検索のためのテキストストリングを入力できる。ストレートテキストまたは正規表現が可能。

再検索 テキスト検索ストリングの次の一致部を検索する。

逆から検索 テキスト検索ストリングの前の一致部を検索する。

ブレークポイント 逆アセンブリペインの現在選択されている行にブレークポイントを設定できる。もう一度この操作を行なうと、ブレークポイントが解除される。ブレークポイントが設定されている行は、赤色の強調表示で示される。

ブレークポイントプロパティ ブレークポイント プロパティ ダイアログを表示する。54ページの「ブレークポイントプロパティダイアログ」参照。

ステップ アプリケーションを 1 ステップずつ実行できる (呼び出されたサブル ーチンへ移動する)。GameCube ユニットで一回当り一つの命令を実行し、逆アセンブリを適宜ステップするためターゲットを停止してから、このコマンドは使用可。1 ステップ実行する度に、プログラムカウンタが、次に実行される命令に移る。

ステップオーバー アプリケーションを 1 ステップずつ実行できる (呼び出された サブルーチンをとばす)。GameCube ユニットで一回当り一つの 命令を実行し、逆アセンブリを適宜ステップするためターゲットを停止してから、このコマンドは使用可。1 ステップ実行する度に、プログラムカウンタが次に実行される命令へ移動する。

カーソルまで実行する 逆アセンブリペインでカーソルが現在設定されている命令に到達するまでターゲットが実行されるよう、指定できる。

アドレスまで実行する アプリケーションが、GameCube ユニットに指定のアドレスまで到達した行のコードまで実行するよう指定できる。アドレスは、表示されるダイアログで指定する。

PC をカーソルまで設定する プログラムカウンタを、逆アセンブリペインで現在選択されている行に設定する。>記号が、選択されている行まで移動し、プログラムカウンタがリセットされたことを示す。新しいプログラムカウンタの位置までの間にある行は実行されないので注意すること。

オプコードを表示する このチェックボックスをオンにすると、逆アセンブリペインの 2 列目に、16 進法のオプションコードを表示する。

ファイルへ逆アセンブルする GameCube 逆アセンブリの特定部分をテキストファイルとして保存出来ます。**スタートアドレス**、**終了アドレス**、逆アセンブリを保存するファイルの**ファイル名**を入力できるダイアログが表示さ

れる。16 進法のアドレスを入力する場合は、前に 0x を付けること。

ソースと逆アセンブリを同時に閲覧する

ミックスモードペインは、ELF プログラムのソースコードを表示しながら各行に対応する逆アセンブルされた命令を示します。

標準のステップであるステップイン、ステップアウト機能をソースにも逆アセンブルコードにも用いることができます。コンテキストメニューである**ソース　ステッピング**スイッチでふたつのオプション間を切替えることができます。ソースまたは逆アセンブル行の現在の位置には行のとなりに PC のアイコンが表示されます。カーソル位置まで実行という動作も可能です。

ソースコードや逆アセンブル命令にブレークポイントを設定したり、それを解除したりすることができます。ソースステッピングモードがオンになっている場合、ブレークポイントは二段階で高輝度表示されます。一回目は通常のブレークポイントアイコンで、対応するソース行に対して、そして次に、対応する逆アセンブル命令の横にやや薄暗く表示されます。

このモードがオフになっている場合には、ブレークポイントが追加されたところの逆アセンブル命令の横にだけブレークポイントが表示されます。

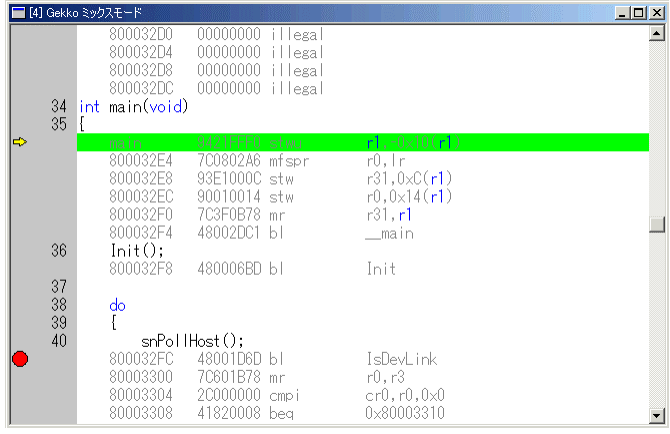
ソースコードの行番号の表示はオン・オフが可能です。このオプションがオンになっている場合、横の境界が広がりソースコード行の脇に行番号が表示されます。この境界部分にマウスを置くと、ツールチップが対応するソースファイルの名前を表示します。

逆アセンブル命令を含む行は空白のままになります。

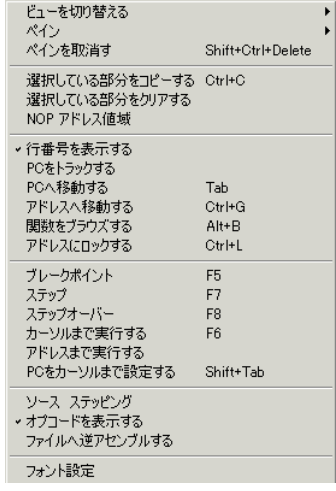
逆アセンブル命令の行にオプコードを表示するかしないかを設定できます。

ソースコードシンタックスの色分けには、アプリケーション設定ダイアログ中で指定された設定が用いられます。アプリケーション設定ダイアログの**フォーマット** タブにある **Mixed** カテゴリーを使うことができます。また、このカテゴリーを使用して、ミックスモードペインの前面、背面、境界の色を設定できます。

ミックスモードペインにアクセスするには、ソースあるいは逆アセンブルペインが選択されている時にスペースバーを押します。そこから順に次のペインに移ることができます。その順は、ソースペイン、逆アセンブルペイン、ミックスモードペイン、またソースペインとなります。



ミックスモードペインショートカットメニュー



コピー / 選択している部分をコピーする	ある部分が選択されている時は**選択している部分をコピーする**と表示され、選択部分がクリップボードにコピーされる。それ以外の場合は**コピー**と表示され、ペイン全体がプレーンテキストでクリップボードにコピーされる。
選択している部分をクリアする	選択をクリアする。
行番号を表示する	これをチェックすると、ソースコードの行番号が表示される。逆アセンブル命令を含む行はブランク。
PC をトラックする	これがチェックされると、逆アセンブル表示中のプログラムカウンタの位置が自動追跡される。
PC へ移動する	プログラムカウンタの現在値がセットされている行をペインに表示されている逆アセンブルに表示する。これは使用中のアプリケーションがターゲットで動作していない場合にのみ有効。

アドレスへ移動する	閲覧希望の GameCube メモリの特定のアドレス入力を可能にするダイアログを表示する。16 進アドレスを入力する場合は `0x` を前につけるか、デフォルト16進法基数チェックボックスをオンにする。表示させたいメモリ部分を指定するアドレス、シンボルあるいは式を入力する。シンボル名の最初を入力し**自動入力**ボタンを押すと、そのアプリケーションで使えるシンボルのリストが表示される。
関数をブラウズする	アプリケーションの関数に対応する逆アセンブルコードを見るためのダイアログを表示する。さらに、アプリケーション中の特定の関数へのブレークポイントのセット、リセットもこのダイアログで可能。
アドレスにロックする	逆アセンブル表示を固定したい部分のアドレス式を指定する。表示されるべきアドレスか式を入力する。この表示を解除したい場合には、式フィールドをブランクにしてアドレスにロックするダイアログを使う。
ブレークポイント	ソースコードまたは逆アセンブル命令にブレークポイントをセットする。これを繰り返すとそのブレークポイントは解除される。**ソーススステッピング**がチェックされている場合、該当する ソース行に赤色のブレークポイントが現れる。**ソーススステッピング**がチェックされていない場合、命令が追加された場所の命令上にアイコンが表示される。
ステップ	アプリケーションからシングルステップの実行を行なう（サブルーチン内部のステップ実行も可能）。GameCube ユニットで一回当り一つの命令を実行し、逆アセンブリを適宜ステップするためターゲットを停止してから、このコマンドは使用可。プログラムカウンタはステップ実行するたびに次の命令に移動する。
ステップオーバー	アプリケーションからシングルステップの実行を行なう（サブルーチン内部では止まらない）。GameCube ユニットで一回当り一つの命令を実行し、逆アセンブリを適宜ステップするためターゲットを停止してから、このコマンドは使用可。プログラムカウンタはステップ実行するたびに次の命令に移動する。
カーソルまで実行する	カーソルがセットされている命令に到達するまでターゲットが実行されるよう指定する。
アドレスまで実行する	指定されたアドレスに命令が到達するまで、GameCube ユニット上でアプリケーションを実行させることができる。アドレスは表示されるダイアログ上で指定する。
PC をカーソルまで設定する	現在選択されている逆アセンブル行にプログラムカウンタをセットする。プログラムカウンタが再設定されたことは、文字＞が選択した行に移動することで確認できる。新しいプログラムカウンタ位置に至るまでの間、プログラム行は実行されないことに注意。
ソーススステッピング	これがチェックされると、ソースレベルでステップ実行が行なわれる。このオプションがチェックされていない場合、ステップ実行は逆アセンブルレベルで行なわれる。

オプコードを表示する	これがチェックされると、逆アセンブル表示の 2 カラム目に 16 進のオプコードが表示される。
ファイルへ逆アセンブルする	GameCube 逆アセンブリの一部をテキストファイルとして保存できる。ダイアログの中で**スタートアドレス**、**終了アドレス**、**ファイル名**を指定する。16 進アドレスを入力する時には、先頭に `0x` をつける。

[このページは意図的に空白にされています]

第 6 章: メモリとレジスタを閲覧・変更する

メモリを閲覧変更する

メモリペインのメモリを閲覧できます。また、全てのメモリアドレスの値を変更できます。

メモリを閲覧・変更するには

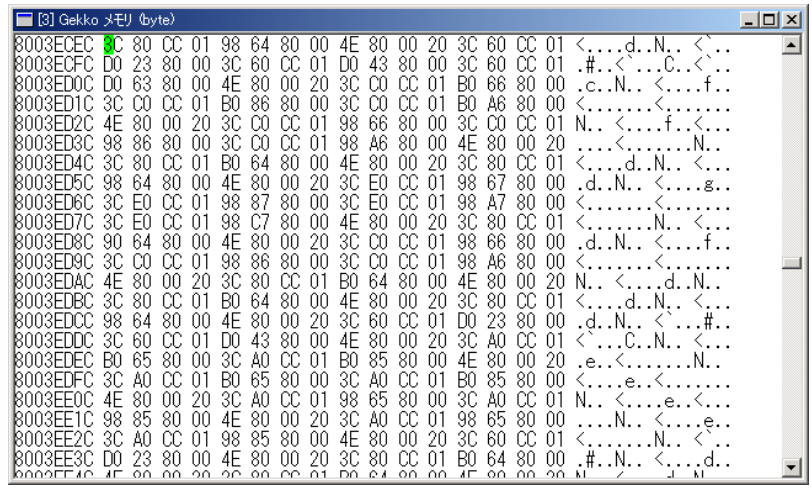
メモリアドレスの値を閲覧するには、以下の手順に従って下さい。

- **ウィンドウ**メニューの**新しいウィンドウを開く →メモリ** をクリックするか、**メモリビュー**ツールバーボタンをクリックします。

  これで、メモリペインを含む新しいウィンドウが開きます。

メモリペイン

メモリペインでは、ゲームキューブメモリの任意の部分の内容を閲覧できます。



メモリ情報は、以下に挙げる 3 つの独立した列に表示されます。

アドレス この最初の列では、残りの行で表現されるターゲット上のメモリ内でのアドレスが表示されます。このアドレスは 16 進法形式で、デフォルトでは 32 ビットです。

データ 2 番目の列では、指定されたメモリアドレスの実際のデータが表示されます。 表示されるデータの量は、ペインの幅によって決まります。データはバイト、ワード、ダブルワード、フロートおよび各種固定形式で表示できます。

ASCII この 3 番目の列では、2 列目で表示されたメモリデータを ASCII 文字で表現します。表示できない文字は、全て “.” と表示されます。

必要に応じ、2 番目の**データ**列で表示されるメモリデータを上書きして、直接ターゲットのメモリを変更できます。また、数字キーの「+」と「-」を使用して、すばやくデータの値を増加・減少できます。

メモリペインでコピー／ペースト機能が使えるようになりました。例えば、メモリアドレス域を選択、クリップボードにコピー、別のペインにドラッグできます。

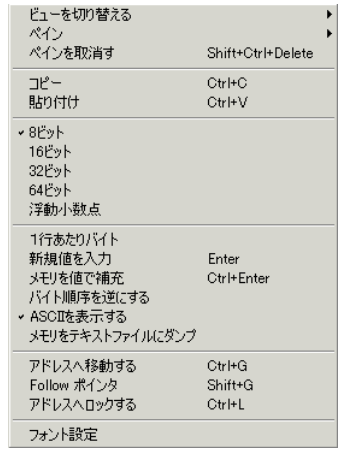
以下の選択方法があります。

- ペインの左側アドレスをダブルクリックして選択します。これを別のメモリペインに**アドレスへ移動する**機能を使用してドラッグするか、ウォッチペインにドラッグして、そのペインに追加できる変数かどうかを調べます。
- アドレス列をクリック、ドラッグしてメモリ行全体を選択します。
- C メモリ値をクリック、ドラッグして個々のメモリ域を選択するため、フリーフォーム選択を実行します。

以上が選択されればクリップボードにテキストとしてコピーできます。

クリップボードにテキストとしてコピーされれば、カーソル位置を替え、ショートカットメニューから**ペースト**を選択して（同じメモリペインか別のメモリペインで）新規メモリ位置にペーストできます。この作業で、既存メモリ位置内容がクリップボードのメモリ位置データによって代替されます。

メモリペインショートカットメニュー



コピー / 選択している部分をコピーする	ある部分が選択されている時は**選択している部分をコピーする**と表示され、選択部分がクリップボードにコピーされる。それ以外の場合は**コピー**と表示され、ペイン全体がプレーンテキストでクリップボードにコピーされる。
選択している部分をクリアする	選択をクリアする。
貼り付け	クリップボードの（テキストベースの）内容を、現在のカーソル位置で始まるメモリにコピーします。
8 ビット	メモリペインを変更して、メモリがバイトで表示されるようにする。
16 ビット	メモリペインを変更して、メモリが半角で表示されるようにする。
32 ビット	メモリペインを変更して、メモリが全角で表示されるようにする。
64 ビット	メモリペインを変更して、メモリが 64 ビットの整数値として表示されるようにする。

<Ctrl+w>ショートカットキーを繰り返し押すと、**8 ビット**、**16 ビット**、**32 ビット**、**64 ビット**、**128 ビット**の表示内容を循環的に表示できます。現在の設定は、ウィンドウのタイトルバーに表示されます。

一行あたりバイト	メモリ内の 1 つの行から次の行までの増加分を指定できる。
新規値を入力	数式を入力して、メモリの 1 領域に新しい値を指定できる。
メモリを値で補充	現在の選択部の全データアイテムをユーザー指定値で満たす。
バイト順序を逆にする	マルチバイトデータアイテムの big-と little-endian 表示をトグルする。
ASCII を表示する	このチェックボックスをオンにすると、2 列目のメモリデータが 3 列目に ASCII 文字で表示される。
メモリをテキストファイルにダンプ	メモリ域をテキストファイルで保存可能に詳しくは77ページの「テキストファイルにメモリを保存するには」を参照。

アドレスへ移動する　メモリペインで閲覧する特定のアドレスやシンボルを指定できる。アドレスは、表示されるダイアログで入力する。16 進法のアドレスを入力する場合は、0x を前につけるか、**デフォルト 16 進法基数**チェックボックスをオンにする。シンボルの先頭部分を入力する場合は、**自動入力**ボタンを使用して、表示することのできるシンボルのリストを表示させることができる。必要なアドレスを入力すると、メモリペインが更新され、ペインの先頭にメモリの指定した部分が表示される。

Follow ポインタ　現在選択されているデータアイテム内に格納されたアドレスにジャンプする。選択されたアイテムが32ビットを下回る場合、ネイバリング（隣り合う）アイテムはアドレスを形成するように連結される。

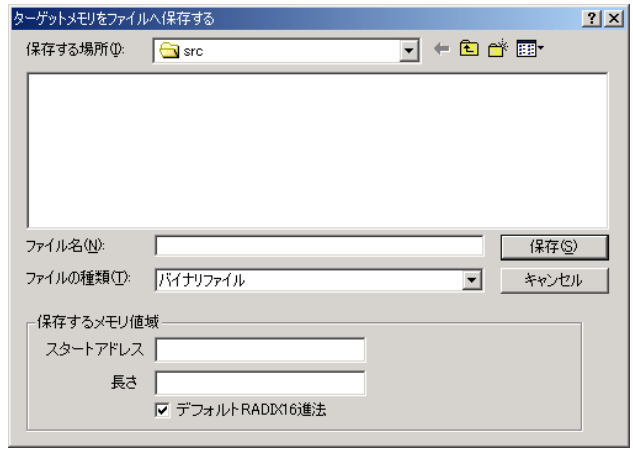
アドレスへロックする　メモリの表示をロックするアドレス数式を指定できる。表示されるダイアログに、必要なアドレス、または数式を入力する。表示に設定されたロックを解除するには、再度 [アドレスへロック]ダイアログにアクセスする必要があるが、この時は数式フィールドを空白のままにしておく。

ターゲットメモリを保存・ダウンロードする

ターゲットメモリの一部をバイナリファイルに保存するか、バイナリファイルの内容をターゲットメモリにダウンロードできます。

ターゲットメモリをバイナリファイルに保存するには

1. **ファイル**メニューの**バイナリを保存する**をクリックすると、以下のダイアログが表示されます。

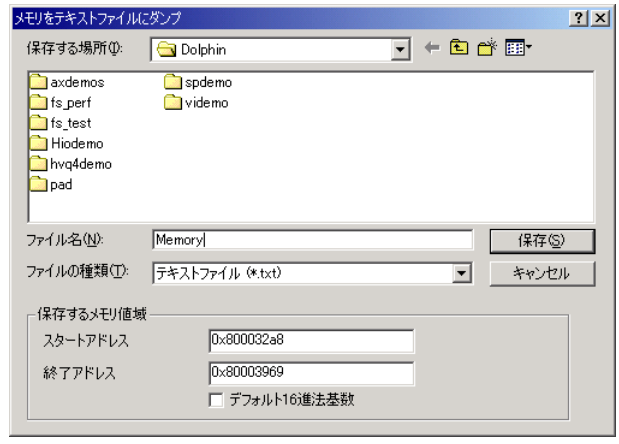


2. メモリ域の**スタートアドレス** (先頭アドレス)と**長さ**を、それぞれの フィール ドに入力します。**デフォルト 16 進法基数** チェックボックス をオンにしている 場合は、**スタートアドレス**と**長さ**フィールドの値は 16 進法と見なされるの で、これらのフィールドに 0x 接頭辞を入力する必要はありません。

3. 既存のファイルを選択するか、メモリの内容を保存したいバイナリファイル名を入力し、**保存**をクリックします。

テキストファイルにメモリを保存するには

1. メモリペインで右クリック、**メモリをテキストファイルにダンプ**を選択して下さい。以下のダイアログが表示されます。



指定のテキストファイル形式で、ディスクにメモリ域を保存します。出力はメモリペイン表示と全く同じになります。

2. 必要なテキストファイルへ移動し、メモリ域先頭アドレス保存のため**スタートアドレス**フィールドで指定します。

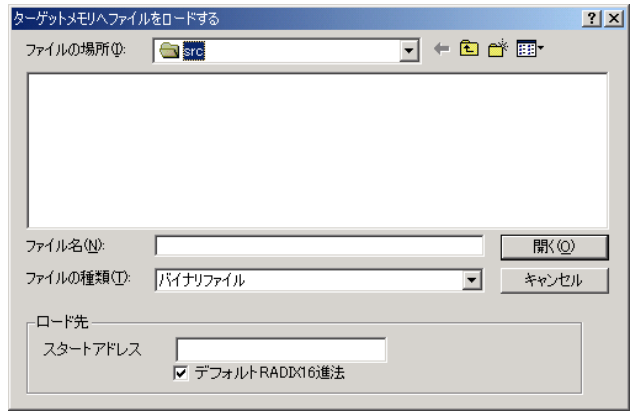
3. メモリ域末尾アドレス保存のため**終了アドレス**フィールドで指定します。**デフォルト 16 進法基数** チェックボックスをオンにしている場合は、**スタート**および **終了アドレス**フィールドの値は 16 進法と見なされるので、これらのフィールドに 0x 接頭辞を入力する必要はありません。

4. **保存**をクリックすると、指定のテキストファイルにメモリ域が保存されます。

> **注:** *メモリ域がこのオプション実行時に既に選択されている場合、スタートおよび最後尾 アドレスもこのセレクションに反映されます。それ以外の場合は現在閲覧中のアドレス域が使用されます。*

バイナリファイルをターゲットメモリにダウンロードするには

1. **ファイル**メニューの**バイナリをロードする**をクリックすると、以下のダイアログが表示されます。



2. 必要なバイナリファイルへ移動し、そのバイナリファイルをダウンロードしたいゲームキューブメモリの先頭アドレスを**スタートアドレス**フィールドで指定します。**デフォルト 16 進法基数** チェックボックスをオンにしている場合は、**スタート アドレス**フィールドの値は 16 進法と見なされるため、このフィールドに 0x 接頭辞を入力する必要はありません。

3. **開く**をクリックすると、指定した先頭アドレスのゲームキューブメモリに、ファイルの内容がダウンロードされます。

> **注:** *ファイル全体がメモリ内に読み込まれます。すなわち、ファイルのうち特定の バイト数を指定してメモリに読み込むことはできません。*

レジスタを閲覧する

レジスタペイン内の全ての GameCube レジスタを閲覧できます。また、全てのレジスタ値を変更できます。

レジスタを閲覧・変更するには

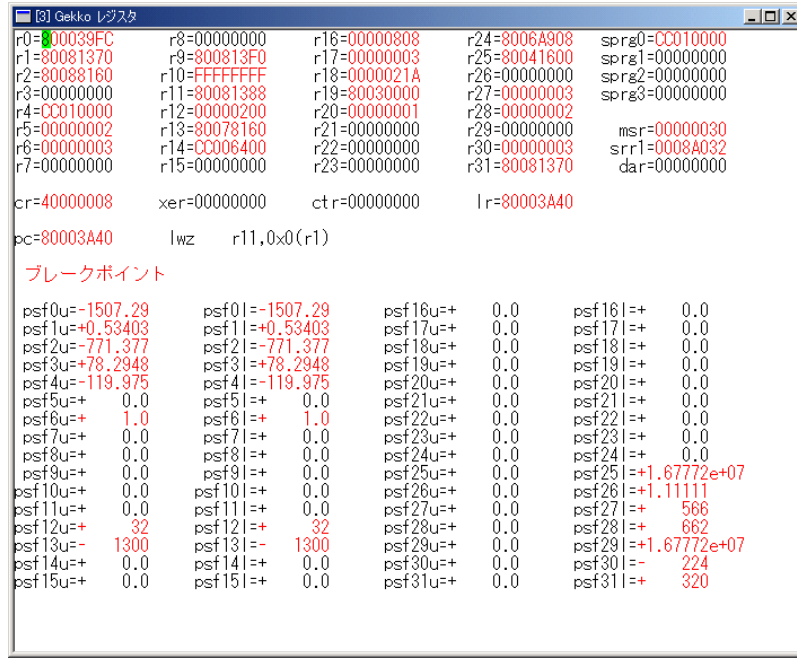
レジスタ値を閲覧するには、以下の手順に従って下さい。

- **ウィンドウ**メニューの**新しいウインドウを開く→レジスタ**をクリックするか、**レジスタ ビュー**ツールバーボタンをクリックします。

  これで、全てのレジスタとその現在の値を示すレジスタペインを含む新しいウィンドウが開きます。

レジスタペイン

レジスタペインは、現在閲覧している GameCube ユニットで情報を入手できる全てのレジスタを表示します。さらに、プログラムカウンタが設定されている逆アセンブリ命令と、ターゲットの現在の状態(赤色で表示)を表示します。



デフォルトレジスタペインビュー

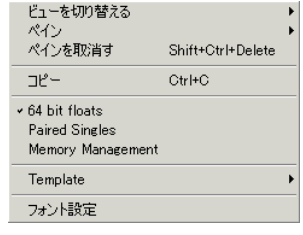
レジスタペインは、ターゲットが停止する度に、自動的に更新されます。レジスタペイン内をクリックすると、修正可能な要素にのみカーソルが表示されます。

レジスタ値を変更するには、レジスタペインの値を直接上書きできます。レジスタ値を変更すると、新たな値がターゲットに送られます。このアクションを取り消す方法はありません。

> **注:** この方法でレジスタ値を変更すると、アプリケーションがターゲットで動かなくなる場合がありますのでご注意下さい。

また、適切なショートカットメニューオプションを使用して 32 ビット、64 ビットまたは 128 ビットレジスタ表示を変更できます。

レジスタペインショートカットメニュー



コピー	ペイン全体をプレーンテキストとしてクリップボードにコピーする。
64bit float	レジスタを 64 ビットフロートとして表示する。
PPC_FP	レジスタを単一ペアとして表示する。

Memory Management	メモリ管理レジスタを表示する。
Template	テンプレート化レジスタビュー（詳しくは80ページの「テンプレート化レジスタビュー」参照）の選択を可能にする。

レジスタ値に対してのみ実行される追加ショートカットメニューがあります。このショートカットメニューへは、実際のレジスタ値を右クリックする方法でのみアクセスできます。ここでは、**10 進法と 16 進法** (メインレジスタ)、または**フロートと 16 進法** (浮動小数点レジスタ) という、各々2 通りの表現方法のいずれかをトグルできます。

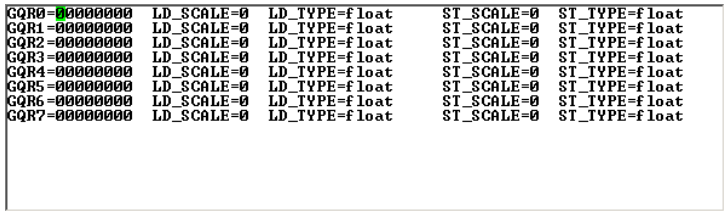
テンプレート化レジスタビュー

レジスタペインはカスタマイズされたハードウェアレジスタの割付を表示するために外部テンプレート(.RGT) ファイルをサポートしました。この機能は組み込まれた **64 ビットフロートと単一ペア**の割付にも及び、カスタマイズできる割付で利用可能などのレジスタでも表示できます。デバッガが起動されるとデバッガが置かれているフォルダ内で.RGT ファイルが検索されます。

これらファイル名（マイナスファイル拡張子）がレジスタペインショートカットメニューのテンプレートサブメニューに追加されます。

代替レイアウトは以下の通りです。

GQR	グラフィック定量化レジスタを表示。



レジスタペイン GQR ビュー

第 7 章: ローカル変数とウォッチ変数を閲覧する

ローカル変数を閲覧する

プログラムカウンタが位置する関数のローカル変数を、ローカル変数ペインで閲覧できます。

ローカル変数を閲覧するには

ローカル変数(現在の関数の変数)の値を閲覧するには、以下の手順に従って下さい。

- **ウィンドウ**メニューの**新しいウィンドウを開く→ローカル**をクリックするか、**ローカル変数ビュー**ボタンをクリックします。

  これで、現在の関数の全変数とその現在の値を示すローカルペインを含む新しいウィンドウが開きます。

ローカルペイン

ローカルペインは GameCube ユニットの現行関数スコープのローカル変数の閲覧に使用されます。ターゲットユニットの関数間で移動するに従い表示は変化し、GameCube が停止次第自動的にアップデートされます。

[4] Gekko ローカル

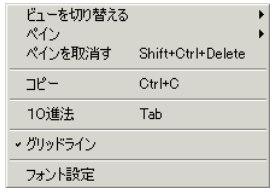
名	形式	値
⊟ v	float [][]	@ 00000008
⊞ [0]	float []	@ 00000008
⊞ [1]	float []	@ 00000018
⊞ [2]	float []	@ 00000028
⊟ pad	struct PADStatus []	@ 00000038
⊞ [0]	struct PADStatus	@ 00000038
⊟ [1]	struct PADStatus	@ 00000044
.button	unsigned short	0x0000
.stickX	char	0x00 ==''
.stickY	char	0x00 ==''
.substickX	char	0x00 ==''
.substickY	char	0x00 ==''
.triggerLeft	unsigned char	0x00 ==''
.triggerRight	unsigned char	0x00 ==''
.analogA	unsigned char	0x00 ==''
.analogB	unsigned char	0x00 ==''
.err	char	0x00 ==''
⊞ [2]	struct PADStatus	@ 00000050
⊞ [3]	struct PADStatus	@ 0000005C

ターゲット上の各変数は、その現在のデータタイプ、および値と共に表示されます。

拡張できる変数の前には、「+」記号が表示されます。これらの変数を拡張するには、その変数をダブルクリックします。現在拡張されている変数は、再度ダブルクリックすると、元に戻ります。配列を拡張すると、全ての配列メンバーが表示されます。

変数の値を変更するには、**値**フィールドの値をダブルクリックし、そこでその値を編集します。

ローカルペインショートカットメニュー



注: [10 進法/16 進法] メニューオプションは、ローカルペイン. にローカル変数が 表示されている時のみ表示されます。

コピー	ペイン全体をプレーンテキストとしてクリップボードにコピーする。
10 進法/16 進法	現在選択しているローカル変数の表示を、10 進法と 16 進法の間で切り換える。
グリッドライン	このチェックボックスをオンにすると、ローカルペインを行と列に分ける表グリッドを表示し、内容を読みやすくする。

変数のモニタリング

さらに、ウォッチペインで特にウォッチしたい変数を設定できます。

ウォッチペインに追加したい変数を正確に選択する際に役立つ C++変数ブラウザがあります。このブラウザでは、全てのアプリケーション変数をクラスごとに分類して閲覧できます。

変数をモニタするには

- **ウィンドウ**メニューの**新しいウィンドウを開く→ ウォッチ**をクリックするか、**ウォッチビュー**ボタンをクリックします。

  空のウォッチペインを含む新しいウィンドウが表示され、このペインに任意のウォッチ変数を追加できます。追加できる変数は、現在の関数の範囲にとどまりません。

ウォッチペイン

ウォッチペインでは、変数や数式をウォッチして、コードを実行するとどのように変化するかを確認できます。ウォッチペインは、始めて開いた時には空白なので、ここに必要なウォッチを追加します。

[5] Gekko監視

名	形式	値
ticks	unsigned int	0x00000000
DrawFsQuad	void ()	@ 00000000
((pad)[1]).substickY	char	0x00 ==’’
pad[0].button	unsigned short	0x0000
PAD_BUTTON_MENU		シンボルが見つかりません
AnimTick(v)		関数コールを実施していません
⊟ v	float [][]	@ 00000008
⊞ [0]	float []	@ 00000008
⊟ [1]	float []	@ 00000018
[0]	float	0
[1]	float	0
[2]	float	0
[3]	float	0
⊞ [2]	float []	@ 00000028

ローカルペインでは、ウォッチ対象として入力した構造や配列は、ダブルクリックすることで、拡張したり、折りたたんだりできます。

ウォッチ変数は、ショートカットメニューの**ウォッチを追加する**をクリックして、表示されるダイアログで必要な変数の名称を入力すれば、追加できます。上記の方法の他に、C++クラスごとに変数を閲覧して、ショートカットメニューの**変数をブラウズする**オプションで、ウォッチしたい変数を正確に指定することもできます。

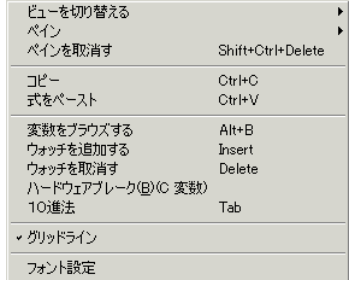
複数のウォッチペインを開いている場合は、各ペインは、そのペインに物理的に追加したウォッチシンボルをのみを表示します。

ウォッチ対象の変数値の表示は、ターゲットが停止する度に自動的に更新されます。

ウォッチペインを閉じると、そのウォッチペインに追加した全てのウォッチ対象変数は、無効となります。しかし、現在の設定を保存し(**ファイル → 設定を保存する**)、その 設定に、ウォッチペインと、その中に追加されたウォッチが含まれている場合は、そのウォッチはデバッグセッションの終了後も保存されます。

変数の値を変更する場合は、**値**フィールドの値をダブルクリックし、そこでその値を編集します。

ウォッチペインショートカットメニュー



注: 上から3番目のメニューオプション**ウォッチを削除する**オプションと **10 進法/ 16 進法**オプションは、現在変数がウォッチされており、それがウォッチペイ ンに表示されている場合のみ、表示されます。

コピー	ペイン全体をプレーンテキストとしてクリップボードにコピーする。
式をペースト	選択において、クリップボードのコンテンツを使用して、新規のウォッチ数式がペインに加えられる。このメニューアイテムは、クリップボードにテキスト情報が含まれる場合のみ表示される。
変数をブラウズする	変数ブラウザを表示する。このブラウザでは、アプリケーション内にある全ての使用する可能性のある変数を、C++クラスごとに閲覧し、追加する必要のある変数を正確に選択できる。
ウォッチを追加する	ウォッチする変数の名称を入力できる。ウォッチする変数の名称を入力するためのダイアログが表示される。
ウォッチを取消す	ウォッチペインから、指定のウォッチ変数を削除する。
ハードウェアブレーク（C変数）	選択アイテム上にハードウェアブレークポイントを追加する。
10 進法/16 進法	現在選択しているウォッチ変数の表示を、10 進法と 16 進法の間で切り換える。
グリッドライン	このチェックボックスをオンにすると、ウォッチペインに列と行に区切られた表グリッドを表示し、内容を読みやすくする。

ウォッチ変数を追加するには

1. ウォッチを追加するには、ショートカットメニューの**ウォッチを追加する**をクリックします。

2. 表示されるダイアログで、ウォッチしたい変数の名称を入力します。ウォッチ変数が新しいウォッチペインに追加され、値が自動的にターゲットから (もし取得できれば) 表示されます。

注: ウォッチウィンドウを閉じると、追加した全てのウォッチが破棄されます。ただし、設定を保存して、ウォッチペインを設定したデバッガを再起動すると、追加したウォッチは保持されます (25ページの「ユーザインタフェースを設定する」参照)。

ウォッチをすばやく追加するには

ソースペインで 1 ステップずつプログラムを実行している時などに、ウォッチを一番上のウォッチペインにすばやく追加するには、以下の手順で [クイックウォッチ] アクセラレータキーを使用します。

1. **設定**メニューオプションから**オプション**を選択して[アプリケーション設定]ダイアログを表示させます。**アクセラレータ**タブが選択されていることを確認して下さい。

2. 右上のリストボックスで **Application** を選択します。

3. **コマンド名**リストを下側にスクロールし、**Quick Watch** のアクセラレータキー設定を表示させて、該当する**キーシーケンス**(あれば) を確認します。

4. **キャンセル**をクリックして[アプリケーション設定]ダイアログを閉じます。

5. ウォッチをすばやく追加するのに必要なキー順序を押します。

ウォッチやローカル変数を拡張、または閉じるには

ローカル変数やウォッチ変数の隣に「+」記号が表示されている場合、その変数は、メンバーや変数の値を表示できるよう拡張できる配列や構造、クラスを示しています。

この配列や構造をすばやく拡張するには、ローカル表示またはウォッチ表示内で、その配列または構造をダブルクリックします。さらに、再度ダブルクリックすると、拡張された変数が直ちに折りたたまれます。

上記の方法以外に、**ウォッチ/ローカルを拡張する**と**ウォッチ/ローカルを閉じる**の各ショートカットメニューオプションを使用して、配列や構造の変数を拡張したり、折りたたむことができます。

変数ブラウザを使用してウォッチを追加する

ウォッチしたい変数の正確な名称がわからない、さらにはアプリケーションの特定の C++ クラスに属している変数を閲覧したい (別のクラスにある変数には、 同じ名称をもつものがある)、という場合があります。この場合、C++変数ブラウザを使用して、アプリケーションの全ての変数を、クラスごとにグループ分けして閲覧できます。

1. ウォッチペインショートカットメニューの**変数をブラウズする**をクリックすると、以下のダイアログが表示されます。

2. アプリケーションが C++でプログラミングされている場合は、必要な変数を含むクラスを**クラス名**フィールドから選択するか、[グローバル]を選択してアプリケーションの全ての変数を閲覧します。アプリケーションが C でプログラミングされている場合、このフィールドにクラスは表示されません。

3. **メンバー名**フィールドのドロップダウンリストで、選択したクラスの変数のリスト、または[グローバル]を選択した場合は、全ての変数のリストを閲覧できます。

   選択したクラスの全ての静的メンバー変数、または[グローバル]が選択されている場合は全てのアプリケーション変数を、ウォッチペインに追加するには、[全て]を選択します。

4. 追加したい変数、または変数群を指定したら、**ウォッチ追加**ボタンをクリックします。ここで表示されるダイアログでは、選択した変数がウォッチペインに追加され

ています。ウォッチペインにそれ以上ウォッチを追加せずにダイアログを終了するには、**閉じる**ボタンをクリックします。

第 8 章: コールスタックとスレッド情報を閲覧する

コールスタックを閲覧する

プログラムスタックの現在の状態を、コールスタックペインで閲覧できます。このペインでは、プログラムカウンタの現在位置に至るまでにアプリケーションが呼び出したプログラム関数の順序が表示されます。

コールスタックを閲覧するには

コールスタックを閲覧するには、以下の手順に従って下さい。

- **ウィンドウ**メニューの**新しいウィンドウを開く → コールスタック** をクリックするか、**コールスタックビュー**ボタンをクリックします。

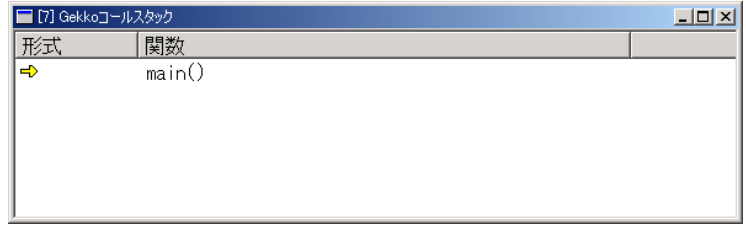
  これで、プログラムスタックの現在の状態を示すコールスタックペインを含む新しいウィンドウが開きます。

コールスタックペイン

コールスタックペインでは、プログラムスタックの内容をいつでも閲覧し、特定の関数コールを選択して、現在のデバッガコンテキストを変更できます。

アプリケーションが実行中止する度に、コールスタックペインは、アプリケーションが現在のプログラムカウンタに到達するまでに呼び出した関数に更新されます。

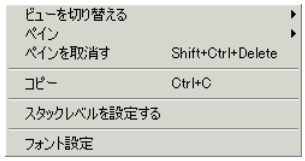
コールスタックペインでリスト表示されている関数のうち、先頭に表示されているものは、最後に呼び出された関数です。



デバッガコンテキストを変更するには、必要な関数をダブルクリックするか、必要な関数を選択して、ショートカットメニューの**スタックレベルを設定する**オプションを使用します。「>」記号は、現在コールスタックで閲覧しているコンテキストを示しています。特定の関数に

新しいコンテキストを設定すると、全てのデバッガペインは更新され、その関数が呼び出された時点のターゲットについての情報を表示します。

コールスタックペインショートカットメニュー



コピー	ペイン全体をプレーンテキストとしてクリップボードにコピーする。
スタックレベルを設定する	デバッガコンテキストを、指定の関数が呼び出された時点のターゲットの状態に変更できる。このコマンドを有効にするには、コールスタックペインで関数コールを選択しておく必要がある。デバッガコンテキストが変更されると、「>」記号がそのコンテキストの前に設定され、現在のスタックコンテキストであることを示す。

スレッド情報を閲覧する

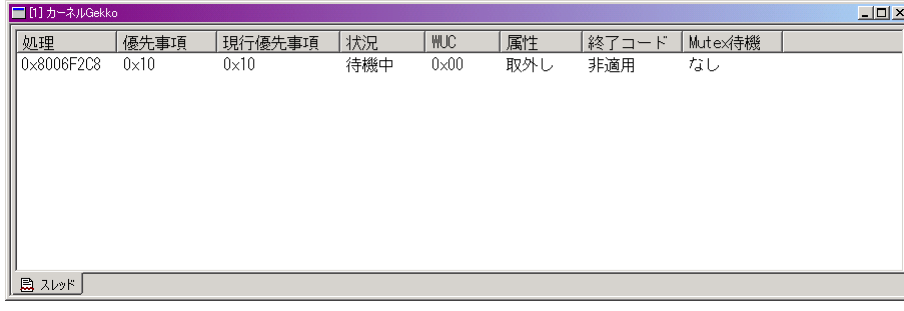
カーネルペインは、ターゲットで実行中のスレッドに関する全ての情報を表示します。

スレッド情報を閲覧するには

スレッド情報を閲覧するには、以下の手順に従って下さい。

- **ウィンドウ**メニューの**新しいウィンドウを開く→ カーネル** をクリックするか、**カーネルビュー**ボタンをクリックします。

  これで、カーネルペインを含む新しいウィンドウが開きます。



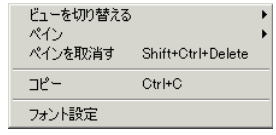
各々のスレッドに従って以下の情報を表示します。

ハンドル	CreateThread()で戻されるスレッド ID。
優先順位	スレッドの元の優先順位
現在の優先順位	スレッドの現在の優先順位
ステータス	スレッドの現在のステータス、準備完了、実行中、待機中、瀕死状

	態、未知など
WUC	スレッドの起動時のカウント
属性	現在のスレッドの属性。分離もしくは「なし」
Exit コード	スレッドの exit コードもしくはスレッドが実行中の場合は「該当なし」
ミューテクス待機	スレッドで待機中のミューテクス ID リスト、もしくは待機中のミューテクスオブジェクトがない場合は「なし」

カーネルペインショートカットメニュー

カーネルペインを右クリックすると、以下のショートカットメニューが表示されます。



コピー	ペイン全体をプレーンテキストとしてクリップボードにコピーする。

[このページは意図的に空白にされています]

第 9 章: TTY 出力を閲覧する

TTY 出力を閲覧する

プログラムからの TTY 出力を、TTY ペインで閲覧できます。このペインでは、デバッグなど printf タイプの出力が表示されます。ショートカットメニューを使用し、TTY ペインは全ての TTY 出力、または PPC および DBG 出力を表示するよう設定可能です。

バッファサイズは各々の TTY バッファ（ALL、PPC および DBG）に独立して設定できます。設定ファイルに新規バッファサイズが保存され、デバッガが実行される度にレストアされます。'全ての TTY'バッファはデフォルトで 64KB、その他の TTY バッファは 32KB に設定されています。

TTY 出力を閲覧するには

TTY 出力を閲覧するには、以下の手順に従って下さい。

- **ウィンドウ**メニューの**新しいウィンドウを開く→TTY** をクリックするか、**TTY コンソールビュー**ボタンをクリックします。

  これで、TTY ペインを含む新しいウィンドウが表示されます。この TTY ペインは、ショートカットメニューオプションを使用して、特定のタイプの TTY ストリームをフィルタリングするよう設定できます。

TTY ペイン

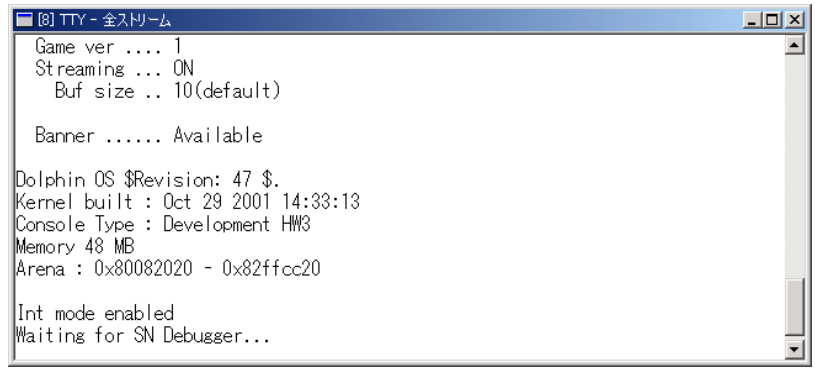
TTY ペインは GameCube で生成された標準出力を表示します。ソースコードで printf コマンドを使用している場合は、このコマンドからの出力を TTY ペインで閲覧可能です。

TTY ペインの出力は、TTY チャネルごとに別の色で表示されます。

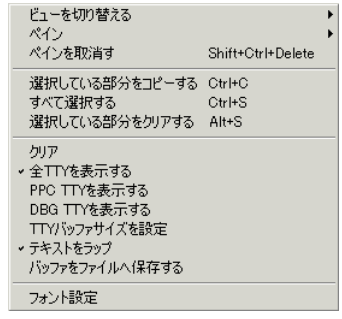
デフォルトでは、TTY ペインにはすべての TTY 出力が表示されます。

ペインを上下にスクロールして、以前の出力を閲覧することもできます。

注: ProDG Target Manager で各々TTY 出力ストリームを閲覧することも可能です。



TTY ペインショートカットメニュー



コピー / 選択している部分をコピーする	ある部分が選択されている時は**選択している部分をコピーする**と表示され、選択部分がクリップボードにコピーされる。それ以外の場合は**コピー**と表示され、ペイン全体がプレーンテキストでクリップボードにコピーされる。
すべて選択	TTY ペインのすべてのテキストを選択する。
選択している部分をクリアする	選択されたテキストをクリアする。
クリア	TTY バッファを完全にクリアする。
全 TTY を表示する	これがチェックされている時、TTY ペインに全ての GameCube 出力ストリームを表示する。
PPC TTY を表示する	このチェックボックスをオンにすると、ゲームキューブの PPC プロセッサからのメッセージ出力を表示する。
DBG TTY を表示する	これがチェックされている時、ProDG デバッガステー タスバーメッセージを表示するようフィルタ。
TTY バッファサイズを設定	これを選ぶと新しいバッファサイズ(KB)を求めるダイアログが表示される。そこで OK をクリックすると、新しいバッファサイズが現在選択されている TTY バッファに適用される。このバッファはサイズが減らされた場合にはクリアされるが、それ以外ではこの値を維持する。
テキストをラップ	これがチェックされると、TTY ペインよりも長い TTY 出力行は

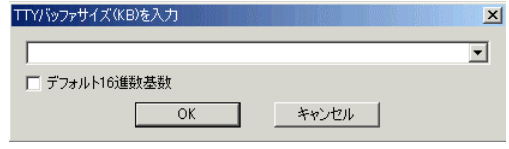
	ペインの幅に合わせて折り返し表示される。
ファイルへバッファを書き込む	TTY ペイン(ヒストリを含む)のコンテンツをファイルへ書き込み可能にする。

TTY バッファのサイズを変更するには

以下の手順に従って、TTY バッファのサイズを変更します。

1. TTY ペインで右クリック、ショートカットメニューにアクセスします。TTY バッファオプションの一つが選択されます(**ALL**、**PPC** または **DBG**)。

2. **TTY バッファサイズを設定**オプションをクリックします。[TTY バッファサイズを入力]ダイアログボックスが表示されます。



3. 編集ボックスに KB で値を入力します。あらゆる値が受け入れられますが、32 以下の値はバッファサイズを 32KB の最小値に設定し、4096 以上の値は 4096KB の最大値に設定します。

4. **OK** がクリックされると、現在選択されている TTY バッファに新規バッファサイズが適用されます。バッファは、サイズを小さくした場合はクリアされますが、それ以外はコンテンツに保持されます。

[このページは意図的に空白にされています]

第 10 章: 基本 Gekko プロファイリング

Gekko プロファイリング概要

NGC 用 ProDG デバッガに新しいデバッガウィンドウタイプである、プロファイル ペインを追 加しました。このペインで、お使いの NGC アプリケーションで使用の Gekko CPU コア時間の消費状況など、すばやいリアルタイムでのビューが可能となります。NGC はお使いのアプリケーション自体のプロファイリングのみの、シンプルなライブラリを提供するだけですが、このペインはプロファイルデータコレクションの制御、また蓄積された結果のコントロールなども提供します。

このプロファイラは開発者からの要望に応えて追加され、よくある「なぜ動作が遅くなったのか？」という疑問の解消を意図しています。ゲーム全体の実行状態の、詳細なシステム分析を目的とするものではありません。かかる詳細なプロファイリングは、このようなシンプルなデバッガアドオンのカバーする範疇外です。

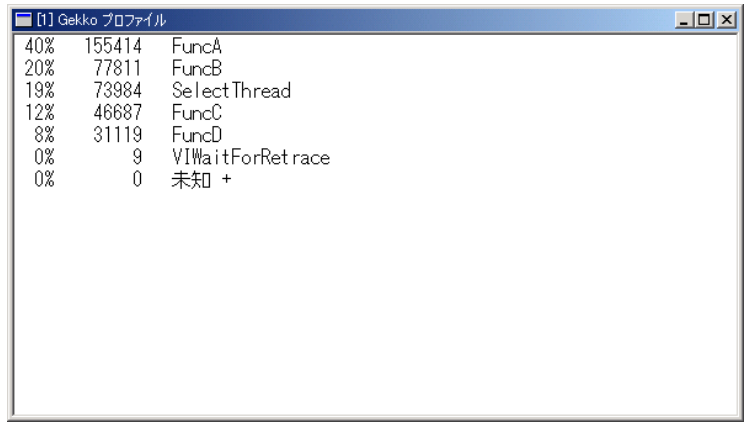
プロファイルペインは、Gekko コアで実行中の小型で効果的なインタラプト処理機能により作成されたデータを蒐集、分析します。漸減例外を使用し、アプリケーションからは完全に不可視です。漸減例外の使用は（OSAlarm が使用します）、NGC ライブラリの関数 PPCMtdec()をデクリメントレジスタに設定してさえあれば可能です。

注: レジスタに直接アクセスすると、途中結果を作成します。

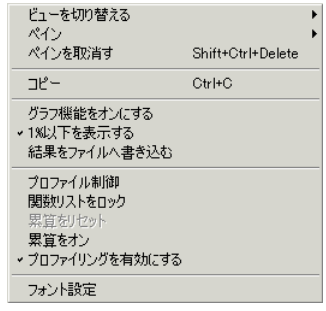
プロファイラは、コードがどこでもっとも頻繁に使われているかをランダムにサンプルする通常インタラプトを実行します。このデータは Gekko に蒐集され、実行中のアプリケーションのシンボルファイルデータに備えて分析できるよう、時折デバッガに送り返されます。

プロファイルペイン

プロファイルペインは CPU がプロファイルインターバルで使用した時間を表示します。デフォルトで表示はテキスト形式、降順で表示されます（消費量の大きい関数が一番上）。標準の表記ではビットに対して動的なため、デバッガは表示の簡略化、アップデートの制御など、いくつかのオプションを提供します。



プロファイルペインショートカットメニュー



注: 累算をリセットオプションはプロファイリングの最中のみ有効です	

コピー	ペイン全体をプレーンテキストとしてクリップボードにコピーする。
グラフ機能をオンにする	このサブメニューを使用すると、ウィンドウの2列目にグラフィカルにサンプル全体での CPU 使用状況 vs を 10 進法で表す。グラフィカルな表示はプロファイルデータが動的なときに特に有効な指針となる。
1%以下を表示する	CPU タイムの1%以上を消費している項目だけを表示する。統計上無視されてしかるべき低いパーセンテージをセットする小さいサンプルなどに特に有効。
結果をファイルへ書き込む	現在のプロファイルデータをコンマで区切られたテキストとしてディスクに保存できる。ファイルへのエントリには、CPU 使用時間、コール回数、関数名が含まれる。
プロファイルコントロール	サンプルレート、プロファイルアドレス域、および蒐集された、また却下されたプロファイルサンプルを決定するマスクまで、GameCube ターゲットサイドプロファイルオプションを設定するダイアログを呼び出す。このフィールドを空にしておくと、デフォルトとして認識する(PC 域 0 から 0xFFFFFFFF まで、マスク= 0xFFFFFFFF を承認)。

注: ランタイム時に指定可能な、プログラムの特定部分からデータを蒐集できる マスク値の使用法については Gekko profiler API（*Power User Guide ProDG for Nintendo GameCube2 Build Tools* 参照）

サンプルレートはお使いのアプリケーションのプロファイリング、データ蒐集比率、またその後のプロファイル表示のアップデートに直接作用します。サンプルレートを 4KHz から、例えば 20KHz に上げたとすると、表示は5倍アップデートすることになりますが、動作への影響はそれよりも若干大きいものとなります。より高いレートでリアルタイム反応に影響することなく、大きなプロファイルバッファの使用が可能となります。

関数リストをロック	デフォルトは確認されるごとに新しい関数がリストに付加されるなど、完全に動的なリスト。ただしリストが動的になりすぎ、読みづらいことがある。現行関数をリスト上ロックする（すなわち追加関数は「未知」としてファイルされる）、または累算をオンすることにより解決 。
累算をリセット	データの蓄積が進み、リアルタイムでなくなってきたと感じたときに累算をリセット。
累算をオン	デバッガはデフォルトで、プロファイルインターバルをその都度破棄し、最初からやり直す。異なる関数を使用して異なるゲームのセクション間を GameCube アプリケーションが往来している場合に有用。プロファイリング中、関数が絶え間なく設定されている場合、 累算をオン設定することにより、作業をスムーズ化可能。累算モードではデバッガプロファイルサンプルは累積、前回分のインターバルデータに追加される。結果は増加し続け、サンプルセットの変更にも時間がかかる。
プロファイリングを有効にする	細かい処理のため、プロファイル表示画面を凍結する。

留意すべき点

もしプロファイル表示が相当の時間をかけて SelectThread()のような待機状態をブロックしているようであれば、その作業が有効でなかったとみなすか、CPU 許容量を増やすなどしてみて下さい。これは CPU が外部イベントのために待機状態であったことえお示し、また、この時間を他に有効に使える、ということでもあります。任天堂サンプルプログラムのほとんどがプロファイル時、デモ版の CPU 消費量はかなり少なく、そのほとんどが SelectThread()に費やされていることになります。

特定の状況下でのターゲットプロファイリング

プロファイルが有効なコードの場所を選択し、定義できます。snProfSetRange()関数に引き渡されたアドレス域の場所を指定することもできますが、より有効な方法として、プログラム実行中、フラグ値を特定のインターバルや時間でプロファイルするよう限定するという方法もあります。

例えば、ここにシンプルな仮定としてフラグ値を設定し、ゲームのキャラクターごとに CPU のコード使用状況を閲覧するとします。以下のコードシークェンスを Gekko CPU のゲームメインループ内に発生させます。

```
snProfSetRange( -1, 0, -1);
// set profiler to accept all flag values
snProfSetFlagValue(0x01);
 // do AI calculations for this one
ProcessCharacterAI( Player1Object);
snProfSetFlagValue(0x02);
 // and process the other character
ProcessCharacterAI( Player2Object);
 // set flags to "none of the above"
snProfSetRange( 4, 0, -1);
```

ゲーム実行中、プログラムを実行を妨げることなく、Player1 または Player2 もしくはその両方、あるいはどちらでもないものの処理のために、プロファイルデータを表示させるよう、デバッガからマスク値を設定することもできます。そのためには、マスク値を各々4つの状況に応じ、順次 1、2、3、~3 (すなわち 0xFFFFFFFC)と設定して下さい。

上記を、論理的に拡張すると、フラグ値は 3 ではなく、4 となります。3 に設定すると、プロファイラがそれらサンプルをビット 0 もしくはビット 1 と区別できなくなります。例えば、ランタイムでデバッガから、32 の異なるオブジェクトにサンプルデータを任意に設定できます。

```
// Process all 32 non-player bots
for(bot=0; bot<32; bot++)
{
snProfSetFlagValue( 1<<bot );
ProcessBotAI( BotData[bot] );
}
```

デバッガのプロファイル制御ダイアログから、最初のプレイヤーをプロファイルするにはマスク値を 1 に設定、二人めはマスク値 2 に、3 人目は 4 に設定します。

注: フラグの設定はその場所以降蒐集されたプロファイルデータにのみ適用されるので、ターゲットがブレークポイントを中断しても、それ以上プロファイルデータが蒐集されるとはないので、フラグ値の変換がこれまでに蒐集されたサンプルデータに影響することはありません。プロファイラライブラリとデバッガサポートの今後のリリースで、ターゲットが現在プロファイリングしていなくてもこれまでに蒐集されたサンプルセットのフィルタを適用するかもしれません。

第 11 章: GX ディスプレイリストを閲覧する

ディスプレイリストを閲覧する

ディスプレイリストは、DMA により GX に伝えられる、Gekko メインメモリ中の一連の GX 命令です。

ディスプレイリストを閲覧するには

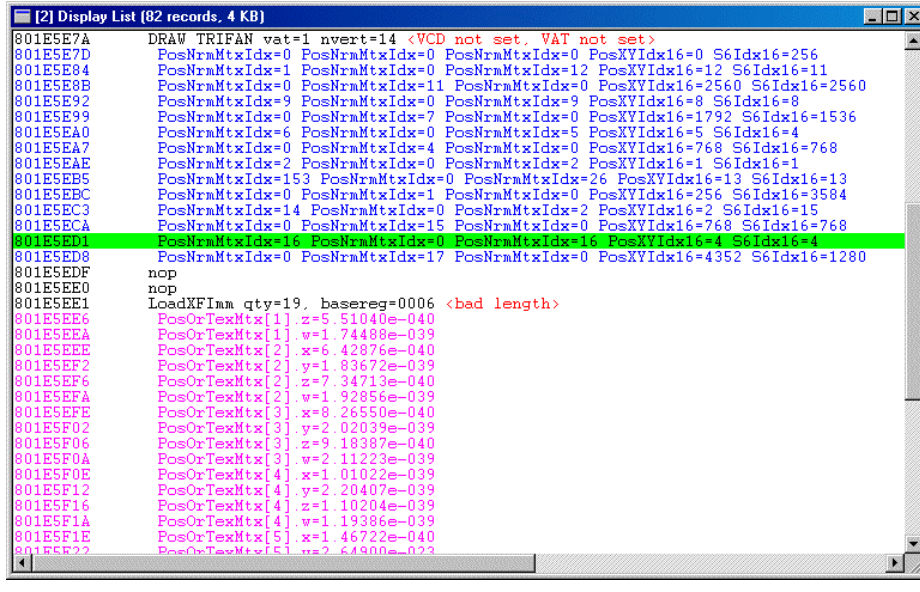
新規ディスプレイリストペインを開くには、以下の手順に従って下さい。

- **ウインドウ**メニューの**新しいウインドウを開く** → **リストを表示**をクリックするか、ツールバーの**ディスプレイリストの閲覧**ボタンをクリックします。

  これで、ディスプレイリストペインのある新しいウインドウが開きます。

ディスプレイリストペイン

ディスプレイリストペインを使うことにより、GX ディスプレイリストの逆アセンブリを閲覧することができます。表示を始めるには、ペインのショートカットメニューから **Set list start address** オプションを使い、メモリ中のリストの開始を定義します。ディスプレイリストはカーソルやキーボードの[Page move] ショートカットを使って閲覧することができます。



キーボードショートカット

下記にあるキーボードのショートカットを使い、ディスプレイリストペインを操作します。

キーボードショートカット	説明
カーソルキー	左、右、上、下へ移動
Home ホーム	ディスプレイリストの初めへ行く
End 終わり	ディスプレイリストの終わりへ行く
Page Down ページダウン	ディスプレイリストの次のページを表示する
Page Up ページアップ	ディスプレイリストの前のページを表示する

ディスプレイリストエラーを検出する

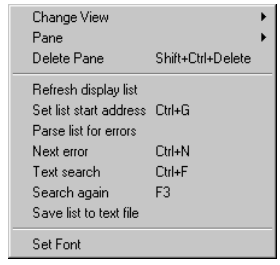
デバッガーは次のエラーの種類を検出します。

1. 不正
2. 現在選択されたバーテックス記述子（VCD）が設定されていません。
3. バーテックス属性表（VAT）が設定されていません。
4. 長さが許容レンジを超えています。

逆アセンブリは GX レジスタの内容次第なため、以前の GX コンテキスト（例えば、プログラムコードによって GX に直接コピーされた設定など）の内容によっては、ディスプレイリストを正確に逆アセンブルできないこともありますので注意してください。GX レジスタは読み取り専用なため、デバッガは以前の指令により同じディスプレイリスト内に設定された GX レジスタの内容しか識別できません。しかし、ジオメトリー指令を設定する前にディスプレイリストが独自の VCD および VAT コンテンツを確実に設定すれば、これは問題となりません。

- デバッガのディスプレイリスト逆アセンブラがディスプレイリスト中にエラーを見つけた場合、該当する行がはっきりと目立つよう、赤いテキストで注釈します。次のエラーを見るには、ディスプレイを進めるショートカットメニューもあります。
- エラーがあるかどうか、長いディスプレイリストを解析したい場合、リスト全体を 1 回で解析するようデバッガに指示できます（<Page Down>キー操作の繰り返しを省けます）。これを行うには、開始アドレスおよびディスプレイリストの長さを指定する必要があります。指定後、デバッガは迅速に逆アセンブリリスト全体のリストを生成し、エラーがあれば最初に検出したエラーへ自動的に進みます。

ディスプレイリストペインのショートカットメニュー



ディスプレイリストをリフレッシュ	現在のバッファ付き逆アセンブリのリストを破棄して、同じアドレスレンジから再びデータを取り出し、リストを再生成します。
リスト開始アドレスを設定	逆アセンブリを始める新たな位置を入力します。現在のディスプレイリストは破棄されます。
エラー検出のためリストを解析	ディスプレイリストのアドレスレンジを指示してから、ディスプレイリストを解析してエラーを検出します。エラーがあった場合、このペインの表示は最初に検出されたエラーへ自動的に進みます。
次のエラー	次のエラーへ進みます。
テキスト検索	現在表示されているリストで指定されたストリングを検索します。検索は、現在カーソルがある位置から始まり、以前生成されたディスプレイリストの範囲まで行えるため、現在選択された範囲のメモリを含むディスプレイリストの終わりを拡張しません。メモリ内の範囲を検索したい場合は、先ず**エラー検出のためリストを解析**を使い、その範囲全体が入るよう、逆アセンブリリストを強制できます。
再検索する	検索を続けます。
リストをテキストファイルに保存する	現在選択されているリストを指定したテキストファイルに保存します。**検索**オプションのように、保存されたリストの範囲は既に生成されたものに制限され、リストは拡張されません。メモリ内の逆アセンブリの範囲を保存したい場合は、先ず**エラー検出のためリストを解析**を使い、その範囲全体が入るように逆アセンブリリストを強制できます。

トラブルシューティング

ニンテンドーゲームキューブ GX のハードウエアの性質により、GX コンテキストは書き込み専用となっています（読み込みはできません）。また、GX ディスプレイリスト逆アセンブリは GX コンテキストに依存します。これは、デバッガがディスプレイリストを正しく逆アセンブルするには、その後に続くリスト中の GX オペコードのコンテキスト情報を設定するすべての情報および指示が、リストに含まれる必要があることを意味します。

これはバーテックスデータにとって特に重要です。単一バーテックス用のデータセットの長さは GX 設定（バーテックス記述子）に左右されるため、バーテックスが処理される段階で先に独自のバーテックス記述子を設定しない任意のディスプレイリストを逆アセンブルすることはできません。ディスプレイリストにバーテックスデータが含まれていても、同時にバーテックスデータの前にバーテックス記述子（VCD および VAT）を初期化する GX オペコードのエントリが含まれない場合、バーテックスデータブロックの長さについてデバッガには見当

がつかず、これより先のオペコードを正確に逆アセンブルできません。これは、ディスプレイリスト外で GX 指令を備えた VAT および VCD エントリを初期化し、それらバーテックス記述子の値を仮定する GX リストを逆アセンブルした場合、ディスプレイリストビューが機能しないことを意味します。ですから、先ず必要な VCD および VAT エントリを初期化しない、バーテックスデータ（ジオメトリデータ）を含んでいるディスプレイリストは、正しく逆アセンブルできません。ジオメトリを処理するリストを逆アセンブルしたい場合は、ディスプレイリストに必ず VCD および VAT の初期化を含んでください。

GX オペコードは任天堂により正式にドキュメント化されていません。したがって、デバッガが検知する GX オペコードのリストは不完全な可能性があります。ディスプレイリストに不明な GX オペコードが含まれていた場合、デバッガはその地点より先を逆アセンブルできません。不明な GX オペコードは、不明か不正なオペコードとして表示される可能性が高く、デバッガは GX オペコードデータの長さが分らないため、それより先にあるリストを正確に逆アセンブルできません。ディスプレイリストでそのような不明の GX オペコードを見かけた場合は、詳しい内容を SN System へお知らせください（2 ページの「テクニカルサポートとアップグレード」を参照）。新たに見つかったオペコードを認識できるよう、当社はデバッガをアップデートします。

第 12 章: デバッガー スクリプト

デバッガー スクリプトとは

注:デバッガー スクリプトは、ProDG Plus for Nintendo GameCube のみで使用 可能です。

C スクリプト インタプリタは、ProDG Debugger の実行時に関数を追加/更新可能な 1 つの C プログラムです。新しい「*スクリプト ペイン*」タイプを使用することにより、 異なる 処理を実行するプログラムを簡単かつインタラクティブに記述することができます。スクリプトは、一度だけ使用する「処理完了まで実行」タイプのスクリプトを記述することができ、ターゲットやデバッガー イベント (例外、ウィンドウ更新、キー入力など) にバインドすることもで きます。ターゲットに対しては、これらのスクリプトを使用してアクセスしてメモリの内容を取得することができ、コンソールからの出力をデバッガーのペインに表示することもできます。

Debugger に搭載された C インタプリタを構成する本製品は、多くの ANSI C に対応し、搭載された多くの関数にアクセスできるため、スクリプトによる Debugger やターゲット コンソールとのインタラクションが可能になります。スクリプトが読み込まれると (または イミーディエイト モードのスクリプト関数が実行された場合)、 バイトコード形式にコンパ イルされた後、変換されます。

実行速度も非常に高速で、通常は、1 秒間に約 100 万の C ステートメントを実 行することができます。ProDG Plus のスクリプト インタプリタには、一般的な Windows スクリプト機能やコード コンパイル方式と比較して多くの 利点があります。この機能は、 COM インターフェイスを使用してアプリケーションを制御する方法より非常に高速なため、 多くの処理が実行可能となり、また使用も簡単です。スクリプト インターフェイスは C で記述されているため、C プログラマーにとっては非常に使用しやすくなっています。

スクリプト インタプリタのコアの部分は EiC となっています。EiC に関する詳細は、http://www.kd-dev.com/~eic/ を参照してください。SN System は、標準技術ライセンスのもと、EiC パッケージを使用して製品を開発しています。ソース コードを含むオリジナルの標準 EiC パッケージは、上記の URL から入手できます。

インストールの注意

注: スクリプティングをご使用される場合、ProDG Plus for Nintendo GameCube ライセンスを取得してください。

スクリプティング機能は、\TestScripts サブディレクトリ内にインストールされています。 スクリプティングについてのインストール及び設定方法の詳細は、このサブディレクトリに ある \autoexec.eic ファイルのインストラクションに従ってください。

トラブルシューティング

問題が発生した場合は、コマンドラインスクリプトによって読み込み時エラー と実行時エラーが出力される DBG TTY ストリームを確認してください。

スクリプトの使用方法

スクリプト ファイルは、Debugger に読み込んで実行できる C のソース コードが記 述されたテキスト ファイルです。このテキスト ファイルは、任意のエディタを使用 して作成/編集できます。

注: 現在のバージョンでは、Debugger 内でスクリプトファイルを編集できません。

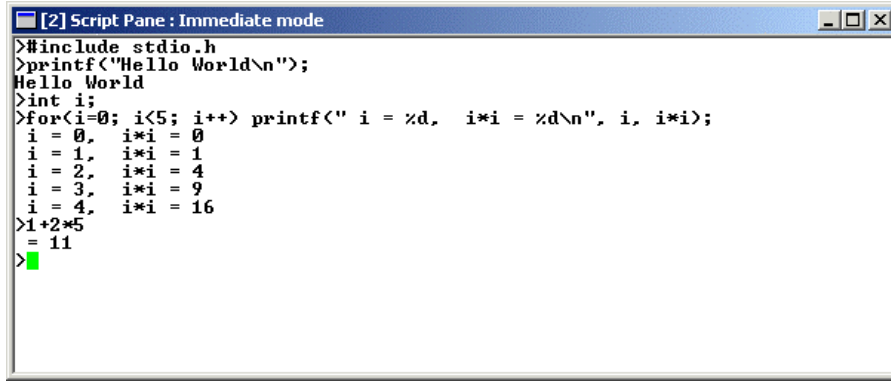
スクリプト ファイルの拡張子は特に決められていませんが、基本的に ANSI C なため「.c」を使用することにより、フォーマット方法や構文の色設定をエディタに認識させることができます。また、コンソール プロジェクト内のソース コードに使用しているほかの C プロジェクト ファイルと区別する必要がある場合は、別の拡張子を使用してください。

スクリプトは、以下の用途に使用できます。

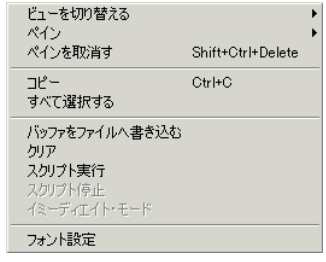
- 標準の C ライブラリ サポート (ファイル アクセスを含む) を含む ANSI C の変換
- Debugger ペインや Debugger TTY バッファへの stdout の出力
- C プリプロセッサ命令への対応
- 「ターゲット例外」などの Debugger イベントのフック
- Debugger キー入力の処理
- ターゲットのリセットと ELF ファイルの読み込み/実行
- ターゲット コンソールとのメモリの送受信
- シンボル テーブルへのアクセスによる、プロジェクトのシンボルを含 む数式の評価
- ウォッチ/ローカル ペインでのタイプのカスタム表示形式の設定
- カスタム デバッガー ペインの表示とデバッガーのコンテキスト メニューへのフック
- デバッガーの起動に合わせたオートランによる、カスタム関数ペインと新しいデバッガー ペインの表示
- 外部エディタとデバッガーのソース ペインとのインターフェイスの制御

スクリプト ペイン

スクリプト ペインでは、Nintendo GameCube 上でスクリプトを実行する ことが できます。このペインは TTY ペインと似ていますが、基本的に編集機能が用意され ているため、スクリプトの記述と実行がその場で可能になります。



スクリプト ペインのショートカット メニュー



コピー	ペイン全体をプレーンテキストとしてクリップボードにコピーする。
すべて選択する	スクリプト ペイン内のすべてのテキストを選択します。
バッファをファイルへ書き込む	スクリプト ペインの内容を、ファイルに書き込みます。
クリア	スクリプト ペインの内容を削除します。
スクリプト実行	スクリプト ファイルを選択して実行します (107ページの「ショートカットメニューからスクリプトダイアログを実行する」を参照)。
スクリプト停止	スクリプトの実行を停止します。
イミーディエイト・モード	このオプションが選択されている場合は、スクリプト ペインに入力されたテキストを即座に実行できます (106 ページの「イミーディエイトモードのステートメント」を参照)。

イミーディエイトモードのステートメント

スクリプト エンジンが動作中でない場合は、スクリプト ペインを使用してその 場で実行するステートメント（一般的な C の構文）を入力することができます。この機能は通常、読み込まれている（キー入力やコールバックにフックされていない）関数の呼び出しに使用します。

この機能は、すべての一般的な C ステートメントやプリプロセッサ命令 に対応します が、ステートメント（複数可）は 1 行に限られます。ここでは、2 行以上使用することが できないため、長いステートメントは、1 行に短縮する必要があります。

通常、以下のように 2 行にわたるステートメントは有効ですが、

```
#include stdio.h   // note that quotes are not required
printf("Hello World\n");
```

イミーディエイト モードでは、以下の 2 行にわたるステートメントは違反になります。

```
for(i=0; i<10; i++)
    printf("i=%d\n", i);
```

このため、以下のように 1 行に短縮する必要があります。

```
for(i=0; i<10; i++) printf("i=%d\n", i);
```

イミーディエイト モードのステートメントでは、最後のセミコロン（″;″）は必要ありません。

- 最後のセミコロンを指定しない場合、セミコロンが必要な場合やステートメントが完了している場合に自動的にセミコロンが追加され、戻り値が出力されます。
- 最後のセミコロンを入力しない場合、戻り値は表示されません。

イミーディエイト モードのスクリプト ペインでは、以下の限定的な編集機能を使用できます。

- <left-arrow> と <right arrow>：現在の行に文字を挿入または文字を 削除します。
- <home> と <end>：現在の行の先頭や最後に移動します。
- <esc>：現在の行を削除し、GraphMode をデフォルトの表示モードに強 制的に設定します。
- <up-arrow>と<down-arrow>：コマンド履歴をもとに、前の行を復元し ます。

スクリプトの読み込みと実行

スクリプトの読み込みと実行には、以下の 3 つの方法があります。

autoexec.eic を使用する

Debugger の起動時には、Debugger の実行ファイルと同じディレクトリ (ngcdbg.exe と同じディレクトリなど) にある autoexec.eic が検索されます。この ファイルが見つかる

と自動的に実行されますが、この時点で、EiC インタプリタの設定、およびあらかじめインクルードしておくファイルのインクルードを行うことができます。このモードで実行されるスクリプトの stdout では、DBG TTY ストリームが指定されます。

> **注**:手動インストールを行う場合は、ヘッダ ファイルをインクルードする前に、スクリプト ファイル autoexec.eic でデフォルトのインクルード ディレクトリを指定 する必要があり ます。

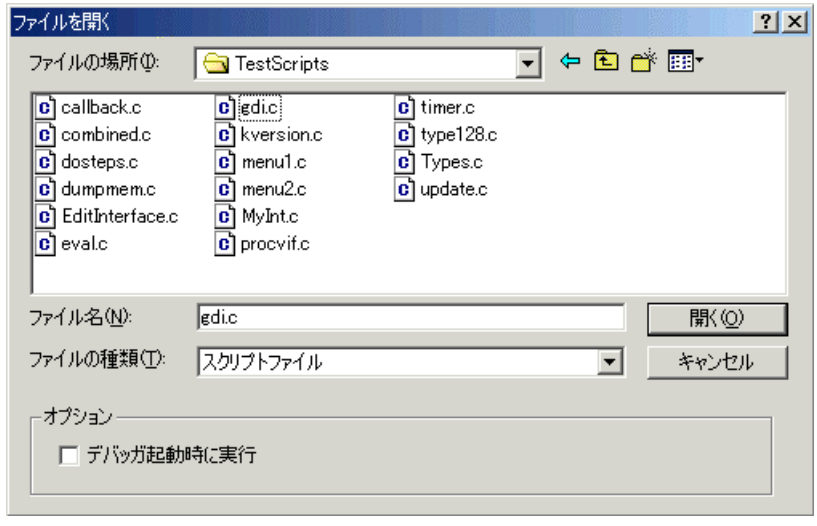
ngcdbg コマンドラインから

ngcdbg –Sscriptname コマンドライン スイッチを使用した場合は、Debugger の起動とターゲット コンソールへの接続の直後に実行するスクリプトを指定するこ とができます。autoexec.eic の場合と同様に、このモードで実行されるスクリプトの stdout では、DBG TTY ストリームが指定されます。スクリプトは、コマンドライン で指定されている順番、左から右の順番で実行されます。

ショートカットメニューからスクリプトダイアログを実行する

スクリプト ペインが開いている場合は、ペインのショートカット メニューから [**スクリプト実行**] を選択することによって、そのコンテキストのスクリプトを実行することができます (105 ページの「スクリプト ペインのショートカット メニュー」を参照)。

表示される [スクリプト実行] ダイアログ ボックスでは、実行するスクリプト ファイルの名前を指定します (以下参照)。



このダイアログ ボックスに用意されている [**デバッガ起動時に実行**] オプションを使用 すると、起動時にスクリプト ペインを開いて特定スクリプトを実行させることができます。コマンドラインのスクリプトとは異なり、これらのスクリプトの出力には、デフォルトでスクリプト ペインが使用されるため、コールバックのフックと GDI 出力の実行が可能 になると いう利点があります。

デバッガーの起動と同時にスクリプトを読み込む

デバッガーの起動時、以下の順番でスクリプトが実行されます。

1. デバッガーの実行ファイルと同じディレクトリの autoexec.eic
2. -Sscriptname スイッチを使用した ngcdbg コマンドラインで指定したスクリプト
3. [スクリプト実行] ダイアログ ボックスの [**デバッガ起動時に実行**] オプションで 指定したスクリプト

使用可能なライブラリ関数

標準 C ライブラリ関数

インタプリタには、標準の C ライブラリが効果的に組み込まれていますが、ヘッダ ファイルは #include でタイプを正しく宣言してから使用する必要があります。ヘッダ ファイルが既にインクルードされている場合は、再度インクルードする必要はありません。

使用可能な標準ライブラリ関数は、ヘッダ ファイルを参考にして確認してください。

SN Systems 関数

ProDG Debugger には、スクリプトに便利な命令を実行させるため、Debugger の機能を活用する関数ライブラリが収録されています。これらの関数は、ヘッダ ファイル snscript.h で宣言します。

```
// For hooking scripts into the debugger
int   SNSetCallback(char* pFuncName);
int   SNHookMessage(int message);
int   SNUnhookMessage(int message);
int   SNBindKey(int keycode, int keydata, char* pstatement);
int   SNSetstdout(int ttychannel);

// For text output
int   SNTxtSetPos(int xpos, int ypos);
int   SNTxtHome(void);
int   SNTxtClrEol(void);
int   SNTxtClrEop(void);
int   SNTxtSetConsole(int width, int height);
int   SNTxtSetTTY(int buffersize);

// For using debugger functions
char* SNEvaluate(snval_t* result, int unitnum, char* pexpstr);
int   SNGetMemory(void* buffaddr, int unitnum, unsigned int addr,
unsigned int count);
int   SNSetMemory(void* buffaddr, int unitnum, unsigned int addr,
unsigned int count);
int   SNLoadElf(int unit, char* pfilename, char* pcmdline, int flags);
int   SNLoadBin(int unit, char* pfname, unsigned int address);
int   SNSaveBin(int unit, char* pfname, unsigned int address, unsigned
int count);
void  SNUpdateWindows(int mask, int unitnum);
void  SNRefreshAll(void);
int   SNSetTypeDisplay(char* ptypestring, char* ptemplate);
int   SNGetTypeDisplay(char* pdest, char* ptype, int index);
int   SNDelTypeHandlers(char* ptype);

// execution control
int   SNSetBP(int unit, unsigned int addr);
```

```
int   SNClrBP(int unit, unsigned int addr);
int   SNIsRunning(int unit);
int   SNStart(int unit);
int   SNStop(int unit);
int   SNStep(int unit, int flags);
int   SNStepOver(int unit, int flags);
int   SNAddr2Line(int unit, unsigned int addr, char** pnamebuff,
unsigned int* pline);
int   SNLine2Addr(int unit, unsigned int* paddr, char* pname, unsigned
int line);
int   SNAddr2Label(int unit, unsigned int addr, char* pout, int*
plen);
int   SNRun2Addr(int unit, unsigned int addr);

// For extending debugger context menus
int   SNAddMenu(int panetype, char* menutext, char* submenutext, char*
script);
int   SNGetMenuInfo(int panetype, int command, char* menutext, char*
submenutext);
int   SNGetPaneInfo(unsigned int hwnd, struct paneinfo * pout);
int   SNGetPaneTitle(unsigned int hwnd, char* pout);

//Other Things that are useful
char* getcwd( char* buffer, int maxlen);
int   chdir( char* dirname);
int   ShellExecute(char* pverb, char* pfname, char* pparms, int
showcmd);
int   SNExit(int retval);  // abort the debugger (parameter currently
ignored)

// GDI functions for graphical output to a script pane:-
int     SNGetWindowSize(int* width, int* height);
int SNCreateBmp(int width, int height);
int SNClearBmp(void);
int SNSetPixel(int x, int y);
int SNSetLine(int x0, int y0, int x1, int y1);
int SNSetColor(int red, int green, int blue);
int     SNRectangle(int x0, int y0, int x1, int y1);
int     SNEllipse(int x0, int y0, int x1, int y1);
int     SNPolygon(int count, int* points);
int     SNTextOut(int x, int y, char* pstr);
int SNSetFont(int size, int flags, char* pname);
int SNEnumFonts(void);
int SNSetGraphMode(int flags, int mode);
```

上記関数のプロトタイプと戻り値に関する詳細は、「*Appendix: Debugger Scripting API*」を参照してください。

スクリプト サンプル

スクリプト サンプルは、ディレクトリ ¥TestScripts に収録されています。以下は、現在使用可能なスクリプトの一覧です。

autoexec.eic	Autoexec 起動スクリプト。スクリプトに対応させるには、このファ イルを変更。
callback.c	SNSetCallback を使用する (111 ページの「タイマーコールバックの設定」を参照)。

dosteps.c	SNStep/SNStepOver を使用する。
editinterface.c	外部エディタとのインターフェイス。
eval.c	デバッガーのシンボル テーブルを使用した数式の評価 (111 ページの「数式の評価」を参照)。
gdi.c	シンプルなグラフィック出力 (110 ページの「GDI 関数」を参照)。
menu1.c	コンテキスト メニューに関数を追加するスクリプト (114 ページの 「ショートカット メニュー項目の追加」を参照)。
menu2.c	コンテキスト メニューに関数を追加するスクリプト。GDI 出力。
myint.c	ユーザー独自のタイプ用のカスタム表示形式 (113 ページの 「各種タイプ用のカスタム表示テンプレートの設定」を参照)。
type128.c	ユーザー独自のタイプ用のカスタム表示形式 (113 ページの 「各種タイプ用のカスタム表示テンプレートの設定」を参照)。
types.c	ユーザー独自のタイプ用のカスタム表示形式 (113 ページの 「各種タイプ用のカスタム表示テンプレートの設定」を参照)。
timer.c	タイマー イベントを使用 (111 ページの 「タイマーコールバックの設定」を参照)。
update.c	デバッガー ウィンドウを強制更新。

GDI 関数

Debugger のスクリプト ペインはデフォルトで、通常の TTY ペインとほぼ同様に 機能しますが、グラフィック出力を含むビットマップ レイヤーを、テキスト ベースの キャラクタ マップ表示に重ねることができます。このモードのスクリプト ペインは、テキスト レイヤーとグラフィック レイヤーの 2 つのレイヤーに効果的に分かれており、 重なって表示されますが、これらのレイヤーは個別に処理されます。またこのレイヤーには、2 つの組み合わせを設定するための各種オプションが用意されています。

以下は、スクリプト ペインをグラフィック出力モードに設定するための一般的なシー ケンスです。

```
//
// Example graphics output using debugger scripting
//
void MyGraphics()
{
    int x,y;
    if(SNGetWindowSize(&x,&y))   // Get size of this script pane
    {
        SNCreateBmp(x,y);        // create a bitmap of the same size
         // Note, default display mode will be TEXT+GRAPHICS, SRCAND
         // - see SNSetGraphMode() for more info
        SNSetColor(255,255,0);   // Set drawing color (Yellow)
        SNEllipse(0,0,x,y);      // Draw an Ellipse to fill the
                                 // whole pane
        SNSetColor(0,0,255);     // (Blue)
```

```
        SNRectangle(x/3,y/3,x/3*2,y/3*2);
                                // Smaller Rectangle
        SNSetColor(255,0,0);    // (Red)
        SNSetFont(24,0,"Arial"); // select font for graphical text
        SNTextOut(250,50,"This is a TEST STRING!");
                                // write some text to the bitmap
        SNUpdateWindows(int mask, int unitnum);
                                // force a repaint of the pane
    }
}
```

グラフィック出力は、通常のキャラクタ マップ TTY 出力に対して異なる「レイヤー」となるため、通常のテキストは重なり合うグラフィックとは別にスクロールします。スクロールバックバッファ機能やテキスト選択機能など、TTY のすべての標準機能は通常に動作し ます。

タイマーコールバックの設定

タイマーをセットすることにより、SM_TIMER メッセージを含むペインのコールバック 関数を、特定の間隔でコールバックすることができます。この機能は、スクリプト ペイン を定期的に更新して、ターゲット コンソールから取得した最新情報を表示する 場合に便利で す。

以下のシンプルな例は、タイマー コールバックの一般的なシーケンスです。

```
#include <stdio.h>
#include "SNscript.h"

int MyCallback(SNPARAM message, SNPARAM param0, SNPARAM param1,
SNPARAM param2)
{
    static int count = 0;

    switch(message)
    {
    case SM_TIMER:
        count++;
        printf("Timer callback %08X\n", count);
        break;
    }
    return 0;
}

int main(int argc, char** argv)
{
    SNSetCallback("MyCallback");  // set our callback handler
    SNHookMessage(SM_TIMER);      // hook the "timer" message
    SNSetTimer(100);                 // 1 per second

    return 0;
}
```

数式の評価

SNEvaluate 関数は、デバッガーの数式評価機能を使用して PC 上の現在のコンテキ ストのある数式を評価します。

例:

```
snval_t     result;
SNEvaluate(&result, 0, "a");
```

上記の例では、ターゲットとなる数式 "a" が評価され、スクリプト変数 "result" に結果 が出力されます。

以下の例では、名前付きレジスタの値の評価が可能になります。

```
void dumpreg(char* regname)
{
    snval_t result;
    unsigned int* addr;
    char* pexperr = NULL;
    unsigned int * pquad;
    char    namebuff[16] = {'$', 0};

    strcpy(namebuff+1, regname);
    pexperr = SNEvaluate(&result, 0, namebuff);
    if(pexperr) // if error
        printf("%s: %s\n", regname, pexperr);
    else
    {
        pquad = result.val.u128.word;
        printf("%s = %08X_%08X_%08X_%08X\n",
            regname, pquad[3], pquad[2], pquad[1], pquad[0]);
    }
}
```

これに続き、イミディエート モードでレジスタの名前を dumpreg() に対する引数として 入力(以下参照) します。

dumpreg("s0");

ターゲット変数に新しい値を割り当てる

スクリプトでは、指定したターゲット側の変数に値を割り当てることもできます。この場合は、単に代入式を評価しますが、その際、現在のターゲット コンソールのコンテキス トにある数式の左側と右側が評価されます。

GameCube 用アプリケーションの実行中は、ローカル変数のコンテキスト範囲がないため、数式がグローバル変数の場合は非常に便利な機能となります。代入式の戻り値が、割り当てられる値となります。

スクリプト エンジン自体はビットフィールド (ビットフィールド構造体は、コンパイラの機能 とターゲット コンソールに依存) に対応していないため、デバッガーの 数式評価機能 はターゲットのビットフィールド構造体を理解することができません。このため、ビットフィールド式を割り当てに使用すると違反となります。

```
snval_t     result;
SNEvaluate(&result, 0, "gMyCheat.CurrentLevel=1");
```

上記の例では、整数値 1 がグローバル変数 gMyCheat.CurrentLevel に割り当てられ、スクリプト変数 "result" には、戻り値 1 が返されます。

各種タイプ用のカスタム表示テンプレートの設定

SNSetTypeDisplay 関数は、ウォッチ ペインやローカル変数ペインでの特定タ イプに使用するカスタム表示テンプレートを設定する場合に使用します。ptypestring 文字列としてパスされる情報は、ウォッチ/ローカル変数ペインの [形式] フィールドで 確認できます。

ウォッチ/ローカルが表示対象として評価される際、結果のタイプ文字列が登録したタイプと比較 (文字列中の空白は無視) されます。タイプが一致した場合は、該当するテンプレート文字列が結果の表示に使用されます。

タイプには、複数のテンプレートを設定することもできます。1 つのタイプに対して複数の テンプレートが設定されている場合は、そのエントリのコンテキスト メニュー (右クリック) にすべてのテンプレートが表示されます。

注:テンプレート文字列には、以下のように特殊なケースもあります。

[テンプレートの最初の文字が '[' の場合は、次の ']' までの文字列が無視され、コンテキスト メニューの表示タイトルまたは表示名となります。タイトルが指定され ていない場合は、テンプレートの非 ws セクションの最初の部分が名前となります。

: 最初の文字 (任意名前フィールド以降) がコロンの場合は、それ以外のテンプレート文字列はスクリプト関数の名前となります。デバッガーはそのスクリプト関数をコールし、関数からはポインタが返され、ウォッチ/ローカル変数ペインの結果文字列に表示されます。このスクリプト関数は、 2 つのパラメータをパスします。

int unit　　ターゲット CPU を識別するインデックス (メイン CPU は 0)

char* pexpr　　NUL でターミネートされる数式文字列に対するポインタ

この ':' のケースでは、多少の記述が必要になりますが、 特定タイプの表示形式を柔軟に設定できます。コールされるスクリプト関数は、シンプルな形式テンプレートでは不可能な非常に複雑な計算を実行することができます。

テンプレート文字列の仕様

この文字列は、通常の printf 形式と同様の仕様となっていますが、パラメー タ値が、個別のパラメータではなく、文字列に組み込まれます。

%d、%f、%x、%g、%s などの通常の % スタイルは、精度指定子やフィー ルド幅指定子を含め、printf とまったく同様に使用しますが、プリントする値は、 形式指定子に続けて {} を使用し、文字列自体に組み込む必要があります。

{} で囲んだ文字列はデバッガーの数式評価機能にパスされ、有効になっているター ゲットのシンボル テーブルを使用して、その CPU の現在の PC ロケーションの コンテキストで評価されます。文字列中のキャラクタ '@' は、ウォッチ数式 (表示対象と評価された値) に置き換えられます。

例:

```
SNSetTypeDisplay("int", "[ABC]a = %d{a}, b = %d{b}, c= %d{c}");
```

上記のテンプレートは、実用性を無視した例です。このテンプレートは、タイプ 'int' のすべての表示に使用されます。また、評価されたウォッチ数式は無視され、3 つの 変数の 値が常に表示されます。ここでは、任意の名前 'ABC' が指定されており、この タイプに複 数のテンプレートが設定されている場合は、この名前がコンテキスト メニュー に表示されます。

```
SNSetTypeDisplay("int", "[NIBREV]:MyIntHandler");
```

これは、スクリプト関数インターフェイスの例です。このテンプレートは、テキスト テンプレートには複雑過ぎる情報を処理する場合に使用します。上記の場合、int タイプのハ ンドラがスクリプト関数 char* MyIntHandler(int unit, char* pexpr); をコールし、元の整数値のニブル反転に基づいた文字列が返されます。このテンプレートの名前は、"NIBREV" となります。

```
SNSetTypeDisplay("class test",
"[.members]a = %d{@.m_a}, b = %d{@.m_b}, c= %d{@.m_c}");
```

これは、クラス "test" の 3 つのメンバーを 1 行に表示するシンプルな例です。 クラス は、ウォッチ ペインに開いてメンバーの 値を表示することができますが、この テンプレート が使用されると、すべてまたは一部の値がデフォルトで表示されるようになるため、クラスを開く必要がなくなります。このテンプレートの名前は、".members" となります。

```
SNSetTypeDisplay("class test *",
"[->members]a = %d{@->m_a}, b = %d{@->m_b}, c= %d{@->m_c}");
```

このテンプレートも同様ですが、同じクラスのポインタに対して機能します。名前は、"->members" です。

```
SNSetTypeDisplay("class test", "[string]pstr = %s{@.m_pstr}");
```

最後の例は、クラス "test" の表示形式を変更するためのものです。このクラスには 2 つのテンプレートが使用できるため、コンテキスト メニュー (右クリック) からの選択が可能になります。名前は "string" です。ここでは、ターゲットのメモリを ASCII 文字列として表示するための形式指定子 %s が使用されています。

ショートカット メニュー項目の追加

SNAddMenu 関数を使用することにより、指定したタイプのデバッガー ペインでのショ ートカット メニューにカスタム メニュー項目を追加することができます。

戻り値は、追加したメニュー項目に割り当てられた個別のコマンド ID となります。メニュー項目が呼び出されると、この値がコールバック ハンドラにパスされて識 別が行われます。この ID はそのペインでのみ個別となるため、ほかのペインのメニュー項目と ID が重複することもあります。このためハンドラでは、メニュー項目を正しく識別するためにペインの種類も確認されます。

この関数は、以下の 2 つの方法で使用します。

1. スクリプト パラメータが NULL 以外の場合は、実行するスクリプト関数の名前となっている文字列に対するポインタとして認識されます。この方法では、指定した関数が直接コールされます。またコンテキストの専用スクリプト ペインもあり ません。このスクリプトは、デバッガーの起動スクリプトと同様にグローバル スクリ プトコンテキストで実行されるため、スクリプトペインへのテキスト出力やグラフ ィック出力は不可能となっています。DBG TTY チャンネルに対する stdout の実 行や、通常のファイル入出力などは可能です。

2. スクリプト パラメータが NULL の場合は、メニュー項目が選択された際に SM_MENU コールバックが行われます。 このコールバックのフックに 関しては、117 ページの「付表：デバッガスクリプティング API

3. 」を参照してください。この種類のコールバックメッセージのパラメータは、以下のようになっています。

   msg = SM_MENU
   param0 = コンテキスト メニューが属するペインの処理
   param1 = メニュー項目のコマンド ID
   param2 = WT_ ペイン タイプ（SNScript.h で指定）

   直接的な関数マッピングではなく、コールバックを使用した場合は、固定されたコンテキストをコールバックによって維持できるという利点があります。たとえば、特定のスクリプト ペインに属すコールバックを使用して、 ペインの状態を変更することによってグラフィック出力や異なる種類のテキスト出力の使用が可能になります。また、メニューから専用のスクリプト ペインに出力（テキスト/グラフィック）を行う場合も、この方法を使用します。

スクリプティングの制限

ProDG デバッガのスクリプティングサポートは、まだ開発初期の段階にあるため、その仕様は変更の対象となっています。SN Systems は、できる限りデバッガスクリプティング API の統制に継続して努めていきますが、ユーザーは今後のデバッガーアップデートを適用するために、デバッガスクリプトに関するマイナーな変更を求められる場合があることをご了承ください。

以下は、現行バージョンに適用され、現在知られている制限事項です。

- スクリプトはポインタ-セーフですが、デバッガー内部とのインタフェー スに関する他の ビルトイン関数が現時点で安全ではない場合があります。つまり、これらの異常な値により、デバッガがクラッシュしたり終了する可能性があります。

- ビットフィールドはサポートされていません。したがって、ビットフィールドを有する構造は使用できません。Cにおけるビットフィールドのインプリメンテーションは、コンパイラに依存しており、スクリプト構造ビットフィールドとターゲットシステムのビットフィールドとの間の正確な整合性を期待するには無理があります。ただし、デバッガ式評価機能はビットフィールドをサポートしているので、整数にビットフィールドを用いるターゲット式を評価したい場合、式評価機能にビットフィールドを含む式を提示するたけで実行できます。

- スクリプト解析機能は、デバッガ総体と同様のスレッドコンテキストで実行されるので、ターゲット通信の同期化に伴う問題はありません。ただし、スクリプトが CPU 時間を多く費やせば、デバッガを応答不能にするおそれがあるため、（それが気になる場合は）長時間ブロックするスクリプトを書き込まない方が無難です。同じ理由から、scanf()や getchar()などの関数をいつまでもブロックすることに関しては現時点ではサポートされていません。終了しないスクリプトペインに基づくスクリプト関数を間違って実行した場合、ウィンドウがキーボードフォーカスを有している時に<Ctrl+C>をタイプ入力することにより、割り込むことができます。

- 原則的に、デバッガスクリプトは1つしかありません。他のファイルから、さらに多くのスクリプト関数をいつでもロードしたり捕捉したりできますが、それらの全ての関数は1つのスクリプトプログラムの一部となります。既存の関数と同名の関数を含むファイルをロードすれば、既存関数は同名の新規関数に置換され、それ以降、その既存関数の以前の状態を呼び出そうとする関数は、新規の関数を呼び出すことになります。したがって、異なるスクリプトファイルの中の全関数名（およびグローバル変数）は一意であることに留意する必要があります。

- 即時モードで以下の 3 行を実行することは、スクリプトファイルをロードすることと同じです。

```
int main(int argc, char** argv){return 0;}
#include "scriptname.c"
main(argc argv);
```

main()と呼ばれるそれまでの既存関数は削除されます。scriptname.c ファイル内のコードは、その後ロードされコンパイルされて、既にロードされているスクリプト関数に加えられます。新規ファイル内に main()という関数がある場合は、その関数が呼び出されます。main()という関数がない場合は、スクリプトがロードされコンパイルされますが、何も実行されません。

付表：デバッガスクリプティング API

SNSetCallback

プロトタイプ

```
int   SNSetCallback(
 char*                    pFuncName);
```

内容

各スクリプトウィンドウは、特定のデバッガイベントに対応して呼び出せる callback 関数を有することができます。この関数に渡される関数名は、ペインが捕捉する全てのメッセージに対して Callback 件数として設定されます。この関数に NULL を渡すと、このスクリプト／ペインへの callback 設定を消去するため、ペインはそれ以降の callback を受理しません。

実行スクリプトがスクリプトペインで実行しており、callback が捕捉状態でターミネートする場合、そのペインは、即時モードではなく、callback 待機状態になります。その後、スクリプトがこの関数に NULL を渡すことにより callback をキャンセルした場合、そのスクリプトの終了時にペインは即時モードに戻ります。また、コンテキストメニューの**即時モード**コマンドを使用することにより、いつでもそのペインを手動で、即時モードに戻すことができます。

Callback 関数インプリメンテーションの詳細：

```
int MyCallback(SNPARAM message, SNPARAM param0, SNPARAM
param1, SNPARAM param2 )
```

パラメータの詳細はメッセージに基づきます。戻り値は現時点では無視されています。これ以降の対応性のため、ノンゼロに戻したほうが無難です。

パラメータ

この関数のパラメータは以下の通りです。

pFuncName　*入力:*callback 関数の名称にポインタ。Callback 関数は、コールタイムでコンパイルさていれるので、関数ポインタではなく、NULL ターミネートストリング（char*として）です。

戻り値

実行された場合: 0 以外の値。　*実行されなかった場合:* 0。 この関数は、コンテキストの関連デバッガペインを有さないスクリプトから呼び出された場合、実行されません（例：デバッガコマンドラインから開始されたスクリプト）。

参照事項

SNHookMessage, SNUnhookMessage, SNSetTimer, SNAddMenu

SNHookMessage

プロトタイプ

```
int   SNHookMessage(
 int                      message);
```

内容　この関数は、スクリプト callback 関数に渡されるメッセージを指定します（それまでの設定は SNSetCallback()）。メッセージが“捕捉”されなかった場合、メッセージは callback 関数に渡されません。

シングル callback 関数は、いくつかのタイプの callback メッセージを受理できます。単に、捕捉したい各メッセージタイプに対して SNHookMessage()コールを発令します。その後、異なるタイプのメッセージに応えるため、callback 関数（通常、*switch(msg)*ステートメント）に条件付きコードを加える必要があります。

パラメータ　この関数のパラメータは以下の通りです。

message　*入力:*インターセプトするメッセージ数

メッセージ数は SNSCRIPT.H ヘッダファイルで定義されます。

SM_TARGETATN ターゲットコンソールが状態変更情報を送信
SM_CHAR　スクリプトペインで文字キーが押下
SM_KEYDOWN　スクリプトペインでキー(システムキーを含む)が押下
SM_TIMER　プログラム可能タイマーインターバル
SM_MENU　デバッガペインのコンテキストメニューからのメッセージ

以下のメッセージがリザーブされます。

SM_MOUSECLICK　スクリプトペインでクリックされたマウス
SM_DBLCLICK　スクリプトペインでダブルクリックされたマウス
SM_MOUSEACTIVATE　アクティブになったスクリプトペイン

戻り値　*実行された場合*: 0 以外の値。　*実行されなかった場合:* 0

参照事項　SNUnhookMessage

SNUnhookMessage

プロトタイプ

```
int   SNUnhookMessage(
 int                    message);
```

内容　この関数は、現在のスクリプト callback 関数により要求された通知リストから、指定されたメッセージを削除します。つまり、SNHookMessage()関数の結果を元に戻します。なお、捕捉された全てのメッセージを削除した場合でも callback は削除されません。この場合、全てのメッセージを捕捉解除した後、同じ callback 関数にさらにメッセージを捕捉することができます。Callback を削除したい場合は、SNSetCallback(NULL)を使用します。

パラメータ　この関数のパラメータは以下の通りです。

message　*入力:*これ以降インターセプトしないメッセージ数

メッセージ数は SNSCRIPT.H ヘッダファイルで定義されます。

SM_TARGETATN ターゲットコンソールが状態変更情報を送信
SM_CHAR　スクリプトペインでキャラクターキーが押下
SM_KEYDOWN　スクリプトペインでキー(システムキーを含む)が押下

SM_TIMER　　プログラム可能タイマーインターバル
SM_MENU　　デバッガペインのコンテキストメニューからのメッセージ

以下のメッセージがリザーブされます:

SM_MOUSECLICK　　スクリプトペインでクリックされたマウス
SM_DBLCLICK　　スクリプトペインでダブルクリックされたマウス
SM_MOUSEACTIVATE　アクティブになったスクリプトペイン

戻り値　*実行された場合:* 0 以外の値。　*実行されなかった場合:* 0

参照事項　SNHookMessage

SNSetTimer

プロトタイプ

```
int   SNSetTimer(
 int                    interval);
```

内容

この関数は、指定されたインターバルでペインの callback 関数に、SM_TIMER メッセージを callback するタイマーをセットアップします。これは、ターゲットコンソールから回収した情報でスクリプトペインを定期的に更新したい場合に効果的です。

注:インターバルを 0 に指定した場合、タイマーは削除されます。ただし、callback は削除されないので、これ以降の SNSetTimer は再開されます。

パラメータ　この関数のパラメータは以下の通りです。

interval　*入力:* タイムインターバルはミリ秒単位

戻り値　*実行された場合:* 0 以外の値。　*実行されなかった場合:* 0

参照事項　SNSetCallback, SNHookMessage, SNUnhookMessage

SNBindKey

プロトタイプ

```
int   SNBindKey(
 int                    keycode,
 int                    keydata,
 char*                  pstatement);
```

内容

指定されたキーが押されれば、提供されたステートメントが実行されます。Keybindings は、スクリプトペインのコンテキストを要求しません。つまり、Keybindings は、デバッガの startup またはコマンドラインのスクリプトでセットアップができ、デバッガが実行中のスクリプトペインでもセットアップできます。

'myfunc()'というコールに<Shift+Ctrl+F2> を連動させる例:

```
#include <snscript.h>
#include <vk.h>
SNBindKey(VK_F2, KS_SHIFT|KS_CONTROL, "myfunc();");
```

注:“Esc”、“<”(コンマ)、“>”(ドット)の 3 つのキーは、リザーブされています。これらのキーはカスタム関数に連動できません。

パラメータ この関数のパラメータは以下の通りです。

keycode *入力:*インターセプトのキーコード。WinUSer.h.内の Microsoft Windows 定義を参照。効果的に作業を行うため、これらの Scancode 値の有用なサブセットは、ProDG デバッガを伴うヘッダファイル vk.h 内で定義されています。詳細は、vk.h の内容を参照。

keydata *入力:*SHIFT、CTRL または ALT キーが押されたかどうかを示すビットフラグ。これらのフラグは、シフトキーの組み合わせを示すため、共に OR することができます。snscript.h 内の定義を参照。

pstatement *入力:*キーが押された時に実行されるステートメント。提供されたステートメントは、コールタイムでコンパイルされるので、関数ポインタではなく、簡単な NULL ターミネート文字ストリングです。必要に応じて、Cステートメントは、末尾の ‘;’ 文字を含ませる必要があります NULL に設定された場合、そのキーに対する現在の keybinding を消去する必要があることを示します。

戻り値 *実行された場合:* 0 以外の値。 *実行されなかった場合:* 0

SNSetstdout

プロトタイプ

```
int SNSetstdout(
 int                    ttychannel);
```

内容 デフォルトでは、スクリプトペインで開始するスクリプトは stdout 表示としてそのペインを使用し、デバッガコマンドラインで開始したスクリプトは stdout 出力として DBG TTY を使用します。スクリプトは、出力として特定のデバッガ TTY ストリームを使用するように指定することで、このデフォルトを変更できます。

パラメータ この関数のパラメータは以下の通りです。

ttychannel *入力:*出力に使用する TTY ストリーム

TTY チャネル索引例:

ALLTTY	0 // バッファ 0 は全コンソール出力を取得
DBGTTY	1 // バッファ 1 は DBG 内部診断テキストを取得
TMTTY	2 // バッファ 2 はターゲットマネージャー出力を取得
MAINTTY	3 // バッファ 3 はメイン CPU を取得

戻り値 それまでの TTY チャネル

参照事項 SNTxtSetConsole, SNTxtSetTTY

SNTxtSetPos

プロトタイプ

```
int   SNTxtSetPos(
 int                    xpos,
 int                    ypos);
```

内容 これ以降のテキスト出力のためのカーソル位置を設定します。この関数は、2D textmap（コンソール）モードでのみ効果的です。Stdout が TTY バッファへ指令された場合、またはスクリプトペインが TTY バッファモードである場合、効果は全くありません。

パラメータ この関数のパラメータは以下の通りです。

xpos *入力:*カーソル水平位置 （0－ウィンドウ幅）

ypos *入力:*カーソル垂直位置 （0－ウィンドウ幅）

戻り値 *実行された場合:* 0 以外の値。 *実行されなかった場合:* 0。 スクリプトがデバッガペインと関連がない場合、この関数は実行されません。例：デバッガコマンドラインからスクリプトが実行された場合。

参照事項 SNTxtSetConsole, SNTxtSetTTY, SNTxtHome, SNTxtClrEol, SNTxtClrEop

---

SNTxtSetConsole

プロトタイプ

```
int   SNTxtSetConsole(
 int                    width,
 int                    height);
```

内容 スクリプトペインを 2D コンソール出力モードに移行させます。適正なサイズの textmap は、これ以降のウィンドウへの全てのテキスト出力に割り当てられ使用されます。現在のスクリプトのウィンドウサイズが新規 textmap より小さい場合、ユーザーが textmap 上をスクロールできるよう、デバッガウィンドウがスクロールバーを表示します。このモードはフォーマットされたテキスト表示のためにより効果的なこともありますが、scrollback バッファをサポートしていません。ペインが TTY モードからコンソールモードにスイッチされた場合、TTY バッファは削除 されません。したがって、これ以降の SNTxtSetTTY(0) は、それまでの TTY ストリームに基づく表示を回復できます。

パラメータ この関数のパラメータは以下の通りです。

width *入力:*文字単位のウィンドウバッファの幅

height *入力:*文字単位のウィンドウバッファの高さ

戻り値 *実行された場合:* 0 以外の値。 *実行されなかった場合:* 0。 スクリプトがデバッガペインと関連がない場合、この関数は実行されません。例：デバッガコマンドラインからスクリプトが実行された場合。

参照事項 SNTxtSetTTY

SNTxtSetTTY

プロトタイプ

```
int   SNTxtSetTTY(
 int                    buffersize);
```

内容 現在のスクリプトペインを TTY 出力モードにすることを設定します。現在、ペインに付属している全ての 2D textmap は破棄され、このパネルの stdout は代わりにプライベート TTY ストリームに送信されます。このモードは、大きな scrollback バッファを有することができますが、フォーマットされたテキストを表示している場合、2D テキスト出力には効果的でない場合があります。

現在、スクリプトが標準的デバッガ TTY ストリームへ出力するように設定されている場合、このコマンドは、このペイン専用のプライベート TTY チャネルに再指令を出します。

パラメータ この関数のパラメータは以下の通りです。

buffersize *入力:*TTY バッファ。最小値は 1024 (1KB)、最大値は 1048575 (1MB)。また、バッファサイズを 0 として指定でき、この場合、バッファサイズは未変更のまま維持されますが、スクリプトペイン出力は 2D コンソールモードから TTY バッファモードに切り替わります。

戻り値 *実行された場合:* 0 以外の値。 *実行されなかった場合:* 0。スクリプトがデバッガペインと関連がない場合、この関数は実行されません。例:デバッガコマンドラインからスクリプトが実行された場合。

参照事項 SNTxtSetConsole

SNTxtHome

プロトタイプ int **SNTxtHome**(void);

内容 カーソルをホーム位置にします。2D コンソール表示モードでは、この関数はカーソルをテキストマップの左上の位置に設定します。TTY ストリームモードでは、この関数は TTY バッファをクリアするので、それまでの保存されていたテキスト履歴は全て破棄されます。これ以降のテキスト出力は、バッファの起動から再開されます。バッファのサイズは変更されません。

パラメータ なし

戻り値 *実行された場合:* 0 以外の値。 *実行されなかった場合:* 0。スクリプトがデバッガペインと関連がない場合、この関数は実行されません。例:デバッガコマンドラインからスクリプトが実行された場合。

SNTxtClrEol

プロトタイプ int **SNTxtClrEol**(void);

内容 テキストを行の末尾までクリアします(EOL)。2D コンソール表示モードでは、この関数は現在のカーソル位置から、その行の末尾(textmap の右端)までの textmap をクリアします。TTY ストリーム出力モードでは、この関数は何の影響も及ぼさず、0 に戻ります。この関数は、それまでのテキスト表示を上書きする場合に特に効果的です。

パラメータ なし

戻り値 *実行された場合:* 0 以外の値。 *実行されなかった場合:* 0

SNTxtClrEop

プロトタイプ int **SNTxtClrEop**(void);

内容 テキストをページの末尾までクリアします(EOP)。2D コンソール表示モードでは、この関数は現在のカーソル位置から、そのページの末尾(textmap の末尾)までの textmap をクリアします。TTY ストリーム出力モードでは、この関数は何の影響も及ぼさず、0 に戻ります。この関数は、それまでのテキスト表示を上書きする場合に特に効果的です。

パラメータ なし

戻り値 *実行された場合:* 0 以外の値。 *実行されなかった場合:* 0

SNEvaluate

プロトタイプ

```
char* SNEvaluate(
 snval_t*        result,
 int             unitnum,
 char*           pexpstr);
```

内容 この関数は、デバッガの式評価機能を使用して、指定された CPU 上にある現在の PC のコンテキスト内の式を評価します。**注**:あるターゲットは、唯一の CPU コンテキストを有し、それを 0 として指定する必要があります。

パラメータ この関数のパラメータは以下の通りです。

result *入力:*結果を保持するためバッファにポインタ。 *出力:*結果

unitnum *入力:*ターゲット CPU を識別する索引(0 はメイン CPU)

pexpstr *入力:*評価するため NULL ターミネートされたテキスト式にポインタ

戻り値 *実行された場合:*NULL ポインタ。 *実行されなかった場合:*プレインテキストエラーメッセージにポインタ

SNGetMemory

プロトタイプ

```
int SNGetMemory(
 void*           buffaddr,
 int             unitnum,
 UINT            addr,
 UINT            count);
```

内容 ターゲットからホスト PC 上のバッファへのメモリを取得します。

パラメータ この関数のパラメータは以下の通りです。

buffaddr *入力:*ホスト上の行き先バッファにポインタ

unitnum *入力:*ターゲット CPU を識別する索引(0 はメイン CPU)

addr *入力:*ターゲット CPU 上のメモリアドレス

count *入力:*取得するバイト数

戻り値 *実行された場合:* 0 以外の値。 *実行されなかった場合:* 0

参照事項 SNSetMemory

SNSetMemory

プロトタイプ

```
int   SNSetMemory(
 void*                buffaddr,
 int                  unitnum,
 UINT                 addr,
 UINT                 count);
```

内容 メモリをホスト PC 上のバッファからターゲット CPU 上のメモリへ送信します。

パラメータ この関数のパラメータは以下の通りです。

buffaddr *入力:*ホスト上のソースバッファにポインタ

unitnum *入力:*ターゲット CPU を識別する索引（0 はメイン CPU）

addr *入力:*ターゲット CPU 上のメモリアドレス

count *入力:*送信するバイト数

戻り値 *実行された場合:* 0 以外の値。 *実行されなかった場合:* 0

参照事項 SNGetMemory

SNLoadElf

プロトタイプ

```
int   SNLoadElf(
 int                  unit,
 char*                pfilename,
 char*                pcmdline,
 int                  flags);
```

内容 この関数は、ELF 実行可能ファイルをデバッガおよび／またはターゲット CPU にロードします。

パラメータ この関数のパラメータは以下の通りです。

unit *入力:*ターゲット CPU を識別する索引 (0 はメイン CPU、)

pfilename *入力:*ロードするファイルの名称

pcmdline *入力:*その実行可能ファイルに渡すコマンドライン

flags *入力:*ロード方法を設定するフラグ。フラグ値は、デバッガにシンボルをロードするか、ターゲットコンソールにコードをロードするか、最初にターゲット CPU をリセットするかを示すことができます。下記参照。

```
LFLOAD        // コードをロード
LFSYM         // シンボルをロード
LFEXEC        // コードがロードされた後に実行
LFSTARTUP     // ロード＆実行を行うが、メイン／開始で bp
```

```
LFRESET        // コードをロードする前にリセット
LFRESETFS      // FileServer ルートディレクトリをリセット
LFIMPBPS       // MS Visual Studio から bps をインポート
LFSLOWSYM      //完全に (ショートカットではなく) シンボルを ロード
LFCLRBPS       // ブレークポイントをクリア
LFRESETHOME // fileserver ホームディレクトリをリセット
```

戻り値 *実行された場合:* 0 以外の値。 *実行されなかった場合:* 0

SNLoadBin

プロトタイプ

```
int   SNLoadBin(
 int                    unit,
 char*                  pfname,
 UINT                   address);
```

内容 ホスト PC 上のファイルからターゲットメモリにバイナリイメージをロードします。

パラメータ この関数のパラメータは以下の通りです。

unit *入力:*ターゲット CPU を識別する索引 (0 はメイン CPU)

pfname *入力:*ロードするファイルのパス名

address *入力:*ターゲット CPU 上のメモリアドレス

戻り値 *実行された場合:* 0 以外の値。 *実行されなかった場合:* 0

参照事項 SNSaveBin

SNSaveBin

プロトタイプ

```
int   SNSaveBin(
 int                    unit,
 char*                  pfname,
 UINT                   address,
 UINT                   count);
```

内容 ターゲットメモリからホスト PC 上のファイルにバイナリイメージを保存します。

パラメータ この関数のパラメータは以下の通りです。

unit *入力:*ターゲット CPU を識別する索引 (0 はメイン CPU)

pfname *入力:*ロードするファイルのパス名

address *入力:*ターゲット CPU 上のメモリアドレス

count *入力:*保存するバイト数

戻り値 *実行された場合:* 0 以外の値。 *実行されなかった場合:* 0

参照事項 SNLoadBin

SNStart

プロトタイプ

```
int    SNStart(
 int                    unit);
```

内容 指定されたターゲット CPU を開始します（デバッガの RUN を押すことと同じ）。

パラメータ この関数のパラメータは以下の通りです。

unit *入力:*ターゲット CPU を識別する索引 (0 はメイン CPU)

戻り値 *実行された場合:* 0 以外の値。 *実行されなかった場合:* 0

参照事項 SNStop

SNStop

プロトタイプ

```
int    SNStop(
 int                    unit);
```

内容 指定されたターゲット CPU を停止します（デバッガの STOP を押すことと同じ）。

パラメータ この関数のパラメータは以下の通りです。

unit *入力:*ターゲット CPU を識別する索引 (0 はメイン CPU)

戻り値 *実行された場合:* 0 以外の値。 *実行されなかった場合:* 0

参照事項 SNStart

SNSetBP

プロトタイプ

```
int    SNSetBP(
 int                    unit,
 UINT                   addr);
```

内容 ターゲット CPU 上のブレークポイントを設定します。

パラメータ この関数のパラメータは以下の通りです。

unit *入力:*ターゲット CPU を識別する索引 (0 はメイン CPU)

addr *入力:*ターゲット CPU 上のメモリアドレス

戻り値 *実行された場合:* 0 以外の値。 *実行されなかった場合:* 0

参照事項 SNClrBP

SNClrBP

プロトタイプ

```
int    SNClrBP(
 int                    unit,
 UINT                   addr);
```

内容 ターゲット CPU 上のブレークポイントをクリアします。

パラメータ この関数のパラメータは以下の通りです。

unit *入力:*ターゲット CPU を識別する索引 (0 はメイン CPU)

addr *入力:*ターゲット CPU 上のメモリアドレス

戻り値 *実行された場合:* 0 以外の値。 *実行されなかった場合:* 0

参照事項 SNSetBP

SNIsRunning

プロトタイプ

```
int   SNIsRunning(
 int                    unit);
```

内容 この関数を使用して、特定の CPU 上のコードが実行中か停止状態かを把握します（例：ブレークポイントであるか例外であるか）。**注:** この関数は、ターゲットコンソールに状態確認要求コマンドを送信します。ターゲットコンソールがこれに応答しない場合、このコマンドは実行できないことがあり-1 に戻ります。

注:あるターゲットは、SNStart()コマンドが与えられた直後に“running（実行中）”とうい状態を戻さない場合があります。これは、ターゲット OS 内の競合状況をによるもので、残念ながら当社では対応できません。したがって、SNStart()を発令した直後に SNIsRunning()をコールすることは推奨できません。善処策は、SNSetCallback()を使用して、SM_TARGETATN のイベントを捕捉し"target started" および "target stopped"のイベントに応えることです。

パラメータ この関数のパラメータは以下の通りです。

unit *入力:*ターゲット CPU を識別する索引 (0 はメイン CPU)

戻り値 *実行された場合:*ターゲット実行中は 1、停止状態では 0。 *実行されなかった場合:* -1 の値

SNUpdateWindows

プロトタイプ

```
Void  SNUpdateWindows(
 int                    mask,
 int                    unit);
```

内容 デバッガウィンドウを更新します。この関数は、スクリプトが何らかのオペレーションが行なわれたため、ターゲットコンソール上で何かが変更され、新しい状態を正確に反映するためにデバッガ表示を更新する必要がある場合に効果的です。

パラメータ この関数のパラメータは以下の通りです。

mask *入力:*更新されるべきウィンドウタイプを指定します。下記のフラグは、論理的 OR を組み合わせて更新させるウィンドウタイプの組み合わせを選択できます。-1 のマスク値は、常に全てのウィンドウという意味です（M_ALL に変換）。0 のマスク値は、現在のスクリプトペインのみを更新させます（ペイン関連のグラフィックス／ビットマップデータを更新させる場合に効果的です）。

M_ALL 全ペインタイプ

M_NONE	なし
M_REGISTERS	レジスタペイン
M_MEMORY	メモリペイン
M_DISASM	逆アセンブリペイン
M_SOURCE	ソース／テキストファイルペイン
M_WATCH	ウォッチペイン
M_LOCALS	ローカル変数ペイン
M_BREAK	ブレークポイントペイン
M_STACK	コールスタックペイン
M_NGCTTY	NGC TTY ペイン
M_WORKSPACE	ワークスペースペイン
M_SCRIPT	スクリプトペイン
M_NGCPROFILE	NGC プロファイラペイン
M_KERNELPANE	カーネルおよびスレッド作成情報ペイン

unit　*入力:*ターゲット CPU を識別する索引 (0 はメイン CPU)

戻り値　なし

参照事項　SNRefreshAll

SNRefreshAll

プロトタイプ　void **SNRefreshAll**(void);

内容　この関数は、デバッガがターゲットコンソールに関して知りうる全てを破棄するようにした後、全てを再取得、再描写します。これは、SNUpdateWindows の代替として効果的でもあり、もう1つの利点として、デバッガがコンソールの全ての状態を再取得できるようにします。

全てのターゲットユニットに関するデバッガメモリキャッシュをクリア

開かれたソースペインに関するソースファイルを再ロード

全てのデバッガウィンドウを更新

パラメータ　なし

戻り値　なし

参照事項　SNUpdateWindows

SNSetTypeDisplay

プロトタイプ

```
int   SNSetTypeDisplay(
 char*                ptypestring,
 char*                ptemplate);
```

内容　この関数は、ウォッチまたはローカル変数ペイン内の指定されたタイプに使用されるカスタム表示テンプレートの設定に適用できます。この関数を、ウォッチまたはローカル変数ペイン内での特定タイプの結果の表示方法を上書きしたい場合に適用します。
Ptypestring ストリングとして何を渡すかを把握するには、タイプフィールド内のウォッチまたはローカルペインに何が表示されるかを確認してください。

表示に関してウォッチまたはローカルが評価されれば、結果のタイプのストリングが、登録されたタイプリストと比較されます（いずれのストリングでも、白いスペースは無視されます）。一致した場合、結果を表示するために、対応したテンプレートストリングが使用されます。

どのタイプに対しても複数のテンプレートが設定でき、全てのテンプレートがそのエントリに関するコンテキストメニューに現れます（マウスを右クリック）。

詳細および事例については、113ページの「各種タイプ用のカスタム表示テンプレートの設定」参照。

パラメータ この関数のパラメータは以下の通りです。

ptypestring *入力:*ストリングタイプ名にポインタ

ptemplate *入力:*フォーマット仕様または関数名へのポインタ

戻り値 このタイプに割り当てられたテンプレートの更新数

参照事項 SNGetTypeDisplay, SNDelTypeHandlers

SNGetTypeDisplay

プロトタイプ

```
int   SNGetTypeDisplay(
 char*                pdest,
 char*                ptype,
 int                  index);
```

内容 Ptype でタイプに関するN番目のテンプレートが見つかれば、行き先バッファへそれがコピーされます。

パラメータ この関数のパラメータは以下の通りです。

pdest *入力:*テンプレートを受理する行き先バッファへのポインタ

ptype *入力:*SNSetTypeDisplay と同様に、タイプストリングにポインタ

index *入力:*戻すテンプレートの索引数

戻り値 *実行された場合:* 0 以外の値。 *実行されなかった場合:* 0

参照事項 SNSetTypeDisplay, SNDelTypeHandlers

SNDelTypeHandlers

プロトタイプ

```
int   SNDelTypeHandlers(
 char*                ptype);
```

内容 この関数は、ptype で指定されたタイプに関する*全てのテンプレート*を削除します。

パラメータ この関数のパラメータは以下の通りです。

ptype *入力:*SNSetTypeDisplay に関しては、タイプストリングにポインタ

戻り値 *実行された場合:* 0 以外の値。 *実行されなかった場合:* 0

参照事項 SNSetTypeDisplay, SNGetTypeDisplay

---

getcwd

プロトタイプ

```
char* getcwd(
 char*                    buffer,
 int                      maxlen);
```

内容 この関数は、現在作業中のディレクトリに戻ります。変更がなかった場合、これはデバッガ処理に割り当てられる現在のディレクトリとなります。

パラメータ この関数のパラメータは以下の通りです。

buffer *入力:* 行き先 char バッファ

maxlen *入力:* そのバッファへコピーされる最大長

戻り値 渡すイベントがあった行き先バッファにポインタ

参照事項 Chdir

---

chdir

プロトタイプ

```
int   chdir(
 char*                    dirname);
```

内容 この関数は、現在作業中のディレクトリを設定します。これは、デバッガ処理のために、現在作業中のディレクトリの変更を行います。したがって、ディレクトリを変更した後、元に戻さなかった場合、デバッガ終了時に保存される構成ファイルの場所に影響を及ぼす場合があります。

パラメータ この関数のパラメータは以下の通りです。

dirname *入力:* 現在作業中のディレクトリとして設定されるディレクトリ名にポインタ

戻り値 *実行された場合:* 0。 *実行されなかった場合:* 戻り値−1 は指定されたパスが見つからなかったことを示します。

参照事項 Getcwd

ShellExecute

プロトタイプ

```
int   ShellExecute(
 char*                    pverb,
 char*                    pfname,
 char*                    pparms,
 int                      showcmd)
```

内容 この関数は、MS Windows ShellExecute() API 関数のインタフェースを提供します。これによりスクリプトは、他のアプリケーションを起動し、sendmail:、open:などのシェルアクションを実行するができます。これに関する詳細は、各 API 文書を参照してください

（また、SNScript.h の値をチェックしてください）。

パラメータ この関数のパラメータは以下の通りです。

pverb Microsoft 文書を参照。

pfname Microsoft 文書を参照。

pparms Microsoft 文書を参照。

showcmd Microsoft 文書を参照。

戻り値 Microsoft 文書を参照。

例 即時モードのスクリプトペイン内に以下をタイプ入力できます。

```
// SN ビルトインを表すヘッダファイルを使用
// 関数
#include SNscript.h

// デフォルト関連のテキストファイルを開く
// アプリケーション
ShellExecute("open","c:\\readme.txt",NULL,SW_SHOW)

// Windows 実行可能プログラムファイルを起動
ShellExecute("open","c:\\myprogram.exe",NULL,SW_SHOW)

// デフォルトエディタのテキストファイルを編集（通常、
// ノートパッド）
ShellExecute("edit","c:\\autoexec.bat",NULL,SW_SHOW)
```

SNAddMenu

プロトタイプ

```
int     SNAddMenu(
 int                    panetype,
 char*                  menutext,
 char*                  submenutext,
 char*                  script);
```

内容 この関数は、デバッガペインの指定されたタイプのために、コンテキストメニューへカスタムメニューエントリを加えます。組み合わされた Menutext と submenutext が既存のアイテムを識別する場合、そのアイテムは置換されます。

戻り値は、そのメニューエントリに割り当てられた固有のコマンド ID です。メニューアイテムが呼び出されれば、この値は原因を識別するために callback ハンドラに渡されます。この ID は、そのペイン固有のものです。したがって、他のタイプペイン上のメニューアイテムが同じ ID を共有している可能性があり、ハンドラは、固有の原因を識別するためにペインタイプも確認する必要があります。

これに関する詳細および事例は、114ページの「ショートカット メニュー項目の追加」参照。

パラメータ この関数のパラメータは以下の通りです。

panetype	*入力:*WT_ ペインのタイプ。詳細は SNScript.H を参照。
menutext	*入力:*コンテキストメニューに表示されるテキスト。メニューの各エントリを個別に識別するために Menutext が使用されます。したがって、同じテキストで複数のメニューエントリを加えることはできません。
submenutext	*入力:*サブメニューに表示されるテキスト。単一のサブメニューに複数のエントリを加えるには、単に、各エントリに対して同じ menutext で異なる submenutext を指定します。サブメニューがなく NULL の場合、コンテキストメニューの最上層レベルに新規エントリを加えます。
script	*入力:*実行される関数の名称。このコールはコールタイムでコンパイルされているので、関数の名称である char*であり、関数のポインタではありません。NULL の場合、SM_MENU callback が発令されます。
戻り値	*実行された場合:*このメニューアイテムに割り当てられたコマンドコード。 *実行されなかった場合:* -1。 最大数のメニューアイテム(現行最大数:16メインメニューX16サブメニュー)を超過した場合、この関数は実行されません。
参照事項	SNGetMenuInfo, SNGetPaneInfo, SNGetPaneTitle

SNGetMenuInfo

プロトタイプ

```
int     SNGetMenuInfo(
 int                    panetype,
 int                    command,
 char*                  menutext,
 char*                  submenutext);
```

内容

この関数は、ペインに関する固有コマンド ID を、それを生起させたメニューエントリに変換します。これは、callback ハンドラが、callback の原因が特定のメニューアイテムであることを確認する1つの方法として提供されています。

パラメータ

この関数のパラメータは以下の通りです。

panetype	*入力:*WT_ ペインのタイプ。詳細は SNScript.H を参照。
command	*入力:*そのメニューアイテムのコマンド ID
menutext	*入力:*メニューテキストを受理するバッファにポインタ。 *出力:*バッファ内のメニューテキスト
submenutext	*入力:*サブメニューテキストを受理するバッファにポインタ。 *出力:*バッファ内のサブメニューテキスト

戻り値

実行された場合: 0 以外の値。 *実行されなかった場合:* 0

参照事項

SNAddMenu

SNGetPaneInfo

プロトタイプ

```
int     SNGetPaneInfo(
 UINT                   hwnd,
 struct paneinfo*       pout);
```

内容 この関数は、callback ハンドラが、SM_MENU callback を生起したペインについてより多くの情報を取得できるよう提供されます。

パラメータ この関数のパラメータは以下の通りです。

hwnd *入力:*コーリングペインを個別に識別するハンドル

pout *入力:*情報受理の構造にポインタ。 *出力:*ペイン情報。Paneinfo 構造の内容は、変更される場合があります。これについては、SNScript.H 内の定義を参照。

戻り値 *実行された場合:* 0 以外の値。 *実行されなかった場合:* 0

参照事項 SNGetPaneTitle

SNGetPaneTitle

プロトタイプ

```
int     SNGetPaneTitle(
 UINT                hwnd,
 char*               pout);
```

内容 この関数は、callback ハンドラが、SM_MENU callback を生起したペインの titlebar テキストを取得できるよう提供されます。あるペインでは、これは、そのペインに関するより多くの情報(例:ソースペインのソースファイルと行、メモリメインのバイト/ワードモードなど)を取得する方法として効果的です。

パラメータ この関数のパラメータは以下の通りです。

hwnd *入力:*コーリングペインを個別に識別するハンドル

pout *入力:*タイトルストリング受理のバッファにポインタ。 *出力:* ペインタイトルバー

戻り値 *実行された場合:* 0 以外の値。 *実行されなかった場合:* 0

参照事項 SNGetPaneInfo

SNStep

プロトタイプ

```
int     SNStep(
 int                 unit,
 int                 flags);
```

内容 この関数は、指定された CPU が、1つの命令または1つのソース行をシングルステップで実行できるようにします。

パラメータ この関数のパラメータは以下の通りです。

unit *入力:*ターゲット CPU を識別する索引(0 はメイン CPU)

flags *入力:*0=asm レベル、1=ソースレベル

戻り値 *実行された場合:* 0 以外の値。 *実行されなかった場合:* 0

参照事項 SNStepOver

SNStepOver

プロトタイプ

```
int     SNStepOver(
 int                    unit,
 int                    flags);
```

内容

この関数は、指定された CPU が、1つの命令または1つのソース行をステップオーバーできるようにします。

パラメータ

この関数のパラメータは以下の通りです。

unit　*入力:*ターゲット CPU を識別する索引（0 はメイン CPU）

flags　*入力:*0＝asm レベル、1＝ソースレベル

戻り値

実行された場合: 0 以外の値。 *実行されなかった場合:* 0

参照事項

SNStep

SNAddr2Line

プロトタイプ

```
int     SNAddr2Line(
 int                    unit,
 UINT                   addr,
 char**                         pnamebuff,
 UINT*                  pline);
```

内容

この関数は、アドレスをファイル名／行番号にコンバートします。

パラメータ

この関数のパラメータは以下の通りです。

unit　*入力:*コンテキストに関する CPU のユニット数（0 はメイン CPU）

addr　*入力:*探知するアドレス

pnamebuff　*入力:*ファイル名ストリングにポイントするように設定される char*にポインタ。 *出力:* ファイル名

pline　*入力:*行番号に設定される未署名の int 変数にポインタ。 *出力:*行番号

戻り値

実行された場合: 0 以外の値。 *実行されなかった場合:* 0

SNLine2Addr

プロトタイプ

```
int     SNLine2Addr(
 int                    unit,
 UINT*                  paddr,
 char*                  pname,
 UINT                   line);
```

内容

この関数は、ファイル名／行番号をアドレスにコンバートします。

パラメータ

この関数のパラメータは以下の通りです。

unit　*入力:*コンテキストに関する CPU のユニット数（0 はメイン CPU）

paddr	*入力:*アドレスに設定される未署名の int にポインタ。 *出力:*アドレス
pname	*入力:*NULL ターミネートされたファイル名ストリングにポインタ
line	*入力:*行番号
戻り値	*実行された場合:* 0 以外の値。 *実行されなかった場合:* 0

SNAddr2Label

プロトタイプ

```
int     SNAddr2Label(
 int                    unit,
 UINT                   addr,
 char*                  pout,
 UINT*                  plen);
```

内容

この関数は、指定されたアドレスに応答するラベルを戻します。ラベルは、通常、アセンブリ言語ラベルまたは C/C++関数名です。リターンが C++関数名の場合、括弧とパラメータタイプを含む demangled 形式です。

注:ポインタパラメータのいずれかまたは両方は、オプションであり、必要でない場合は NULL として渡すことができます。

パラメータ

この関数のパラメータは以下の通りです。

unit	*入力:*コンテキストに関する CPU のユニット数(0 はメイン CPU)
addr	*入力:*探知するアドレス
pout	*入力:*ASCIIZ ストリングとしてラベルを受理するバッファにポインタ、または、必要でない場合は NULL。 *出力:*ラベル
plen	*入力:*変数長を受理する未署名の int 変数にポインタ、または必要でない場合は NULL。 *出力:*ラベルの長さ
戻り値	*実行された場合:* 0 以外の値。 *実行されなかった場合:* 0

SNRun2Addr

プロトタイプ

```
int     SNRun2Addr(
 int                    unit,
 UINT                   addr);
```

内容

この関数は、指定されたアドレスで一時的なブレークポイントを設定し、ターゲット CPU を起動します。逆アセンブリとソースペインの"Run to Cursor(カーソルまで実行)"機能に類似した動作をします。ブレークポイントは一時的であるため、ターゲット停止時に自動的に削除されます。

注:この関数は、即時戻されます。スクリプトにオペレーション完了を認識させたい場合は、SNSetCallback()関数を使用して、SM_TARGETATN メッセージを捕捉できます。

パラメータ

この関数のパラメータは以下の通りです。

unit	*入力:*コンテキストに関する CPU のユニット数(0 はメイン CPU)
addr	*入力:*可及部のアドレス
戻り値	*実行された場合:* 0 以外の値。 *実行されなかった場合:* 0

SNExit

プロトタイプ

```
int      SNExit(
 int                    retval);
```

内容

この関数は、デバッガを無条件で終了します。これは、主にデバッガをスタンドアローンのスクリプトエンジンとして使えるようにします(例:デバッガのコマンドライン上で起動スクリプトが指定できます)。デバッガは、ターゲットコンソールにコネクトし、スクリプトを実行した後、終了します。

パラメータ

この関数のパラメータは以下の通りです。

retval	*入力:*アプリケーションの戻り値(現在無視されている)

戻り値

なし

SNGetWindowSize

プロトタイプ

```
int      SNGetWindowSize(
 int*                   width,
 int*                   height);
```

内容

この関数は、スクリプトペインの現在のサイズをピクセル単位で取得します。この情報は、グラフィック出力のために作成するビットマップレイヤのサイズを設定するのに効果的です。

パラメータ

この関数のパラメータは以下の通りです。

width	*入力:*スクリプトペインの幅を受理する変数にポインタ
height	*入力:*スクリプトペインの高さを受理する変数にポインタ

戻り値

実行された場合: 0 以外の値。 *実行されなかった場合:* 0。 この関数は、スクリプトコンテキストが関連ペインを有さない場合、実行されません(例:autoexec スクリプトで実行された場合、または、デバッガのコマンドラインから指定されたスクリプトを経由して実行された場合)。

参照事項

SNCreateBmp

SNCreateBmp

プロトタイプ

```
int SNCreateBmp(
 int                    width,
 int                    height);
```

内容

この関数は、ビットマップを実行しているスクリプトペインに関連するビットマップを作成します。ビットマップは指定のサイズとなり、デフォルト表示モードは

TEXTON+GRAPHON と SRCAND になります。つまり、テキストとグラフィックスの両方のレイヤが有効化され、組み合わせモードは、比較的妥当な方法で両方のレイヤを表示するように設定されます。ビットマップは、デフォルトの背景カラーになるようにクリアされます。

ペインが既にビットマップを有している場合、それまでのビットマップは破棄され、指定されたサイズの新規ビットマップに置換されます。この関数は、width=0 と height=0 のパラメータを受理した場合、テキストペインに関連するビットマップを削除します。つまり、そのペインに関連するグラフィカルレイヤがなくなり、通常のテキストのみのスクリプトペインになります。

パラメータ この関数のパラメータは以下の通りです。

width *入力:* 作成するビットマップの幅

height *入力:* 作成するビットマップの高さ

戻り値 *実行された場合:* 0 以外の値。 *実行されなかった場合:* 0。 この関数は、スクリプトコンテキストが関連ペインを有さない場合、実行されません（例：autoexec スクリプトで実行された場合、または、デバッガのコマンドラインから指定されたスクリプトを経由して実行された場合）。

参照事項 SNGetWindowSize, SNSetGraphMode

SNSetGraphMode

プロトタイプ

```
int SNSetGraphMode(
 int                    flags,
 int                    mode);
```

内容 この関数は、スクリプトペインに関連するグラフィックレイヤとテキスト（文字マップ）のための表示モードを設定します。フラグ値は、テキストのみ、グラフィックスのみ、またはそれらの両方を表示するようにペインを設定でき、また、テキスト文字マップと同サイズまで伸長するかどうかが設定できます。モード値は、テキストとグラフィックスの両方が表示のために有効化された場合に、それら2つのレイヤを組み合わせる方法を指定します。

この関数は、実行中のスクリプトが現在アクティブなビットマップを伴う関連スクリプトペインを有する場合のみ問題なく実行されます。テキストのみの表示を選択することで、グラフィックス表示を無効にしますが、ビットマップを削除せず、それ以降の描写も防止しません。つまり、ビットマップは実質的にそのまま残されていて、その後、SNSetGraphMode()を呼び出すことにより、表示を有効にすることができます。

SRCAND は、2つのレイヤの標準的な組み合わせになるので、最も妥当な組み合わせモードです。SNCreateBitmap()コールの後のデフォルト graphmode は、以下を実行した場合と同じです。

```
SNSetGraphMode(TEXTON+GRAPHON, SRCAND);
```

パラメータ この関数のパラメータは以下の通りです。

flags *入力:* テキストおよび／またはグラフィックスを指定します。値（SNScript.h で定義さ

れる)は、以下の通りです。オプションは、論理的 OR と組み合わせることができます。

TEXTON　　テキスト(文字マップ)レイヤの表示を有効化
GRAPHON　　ビットマップ(グラフィックス)レイヤの表示を有効化
STETCHON　　ビットマップをスクリプトペイン内で最大限に伸長

mode　　*入力:*テキスト+グラフィックスの組み合わせモードを指定。スクリプトペイン内のグラフ+テキストの組み合わせモードは以下の通りです(dest=テキストペイン、ソース=ビットマップ)。

SRCCOPY dest = source
SRCPAINT dest = source OR dest
SRCAND dest = source AND dest
SRCINVERT dest = source XOR dest
SRCERASE dest = source AND (NOT dest)
NOTSRCCOPY dest = (NOT source)
NOTSRCERASE dest = (NOT src) AND (NOT dest)
MERGEPAINT dest = (NOT source) OR dest
DSTINVERT dest = (NOT dest)
BLACKNESS dest = BLACK
WHITENESS dest = WHITE

戻り値　　*実行された場合:* 0 以外の値。 *実行されなかった場合:* 0

参照事項　　SNGetWindowSize, SNCreateBmp

SNClearBmp

プロトタイプ　　int **SNClearBmp**(void);

内容　　この関数は、スクリプトペインのビットマップレイヤを消去します。つまり、関連スクリプトペインのビットマップをペインの背景カラーになるようにクリアします。

パラメータ　　なし

戻り値　　*実行された場合:* 0 以外の値。 *実行されなかった場合:* 0。 この関数は、スクリプトコンテキストが関連ペインを有さない場合、実行されません(例:autoexec スクリプトで実行された場合、または、デバッガのコマンドラインから指定されたスクリプトを経由して実行された場合)。また、関連スクリプトペインがビットマップレイヤを有さない場合も実行されません。

SNSetColor

プロトタイプ　　int **SNSetColor**(
int　　red,
int　　green,
int　　blue);

内容　　この関数は、現在の描写カラーを設定します。このカラーは、それ以降の全ての描写コマンドに使用されます。

パラメータ この関数のパラメータは以下の通りです。

red *入力:*赤の値（0－255）

green *入力:*緑の値（0－255）

blue *入力:*青の値（0－255）

戻り値 *実行された場合:* 0 以外の値。 *実行されなかった場合:* 0。 これは、実行中のスクリプトが、現在のアクティブなビットマップを伴う関連スクリプトペインを有する場合のみ実行されます。

SNSetPixel

プロトタイプ

```
int SNSetPixel(
 int        x,
 int        y);
```

内容 この関数は、単一ピクセルを現在の描写カラーに設定します。他の描写コマンドと同様に、出力がペインのビットマップサイズにクリップされます。現在のビットマップサイズを超過するパラメータはエラーとして取り扱われずに、単に不可視の出力となります。

パラメータ この関数のパラメータは以下の通りです。

x *入力:* x 座標

y *入力:* y 座標

戻り値 *実行された場合:* 0 以外の値。 *実行されなかった場合:* 0

参照事項 SNSetColor, SNSetLine

SNSetLine

プロトタイプ

```
int SNSetLine(
 int        x0,
 int        y0,
 int        x1,
 int        y1);
```

内容 この関数は、現在の描写カラーで直線を描写します。他の描写コマンドと同様に、出力がペインのビットマップにクリップされます。終点は、ペインのビットマップ領域を越えることも可能ですが、指定された線の可視部のみがウィンドウに描写されます。負の x 座標および y 座標の値が適用できます。

パラメータ この関数のパラメータは以下の通りです。

x0 *入力:*最初の終点の x 座標

y0 *入力:*最初の終点の y 座標

x1 *入力:*2 番目の終点の x 座標

y1 *入力:*2 番目の終点の y 座標

戻り値 *実行された場合:* 0 以外の値。 *実行されなかった場合:* 0

SNRectangle

プロトタイプ

```
int     SNRectangle(
 int                    x0,
 int                    y0,
 int                    x1,
 int                    y1);
```

内容 この関数は、現在の描写カラーで塗りつぶされた矩形を描写します。他の描写コマンドと同様に、出力がペインのビットマップにクリップされます。座標は、ペインのビットマップ領域を越えることも可能ですが、指定された矩形の可視部のみがウィンドウに描写されます。負の x 座標および y 座標の値が適用できます。

パラメータ この関数のパラメータは以下の通りです。

x0 *入力:*左上端部の x 座標

y0 *入力:*左上端部の y 座標

x1 *入力:*右下端部の x 座標

y1 *入力:*右下端部の y 座標

戻り値 *実行された場合:* 0 以外の値。 *実行されなかった場合:* 0

SNEllipse

プロトタイプ

```
int     SNEllipse(
 int                    x0,
 int                    y0,
 int                    x1,
 int                    y1);
```

内容 この関数は、現在の描写カラーで塗りつぶされた楕円を描写します。パラメータで記述された矩形に収まる最大限の楕円を描写します。他の描写コマンドと同様に、出力がペインのビットマップにクリップされます。座標は、ペインのビットマップ領域を越えることも可能ですが、指定された楕円の可視部のみがウィンドウに描写されます。負の x 座標および y 座標の値が適用できます。

パラメータ この関数のパラメータは以下の通りです。

x0 *入力:*左上端部の x 座標

y0 *入力:*左上端部の y 座標

x1 *入力:*右下端部の x 座標

y1 *入力:*右下端部の y 座標

戻り値 *実行された場合:* 0 以外の値。 *実行されなかった場合:* 0

SNPolygon

プロトタイプ

```
int     SNPolygon(
 int                    count,
 int*                   points);
```

内容

この関数は、塗りつぶされた多角形を描写します。2番目のパラメータは、整数値の配列にポイントされる必要があります。各ポイントの2つの値は、それぞれの x と y の座標の組を表しています。下記の例を参照。他の描写コマンドと同様に、出力がペインのビットマップにクリップされます。座標は、ペインのビットマップ領域を越えることも可能ですが、指定された楕円の可視部のみがウィンドウに描写されます。負の x 座標および y 座標の値が適用できます。

パラメータ

この関数のパラメータは以下の通りです。

count　*入力:*多角形のポイント数

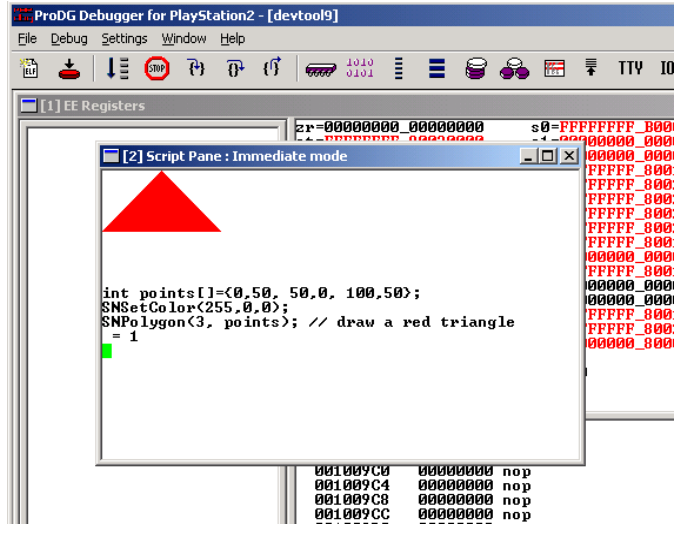
points　*入力:*ポイントの配列にポインタ

戻り値

実行された場合: 0 以外の値。 *実行されなかった場合:* 0

Example

This immediate mode example draws a simple polygon:



SNTextOut

プロトタイプ

```
int     SNTextOut(
 int                    x,
 int                    y,
 char*                  pstr);
```

内容

この関数は、指定されたビットマップ座標にテキストを描写します。このテキストは、ペインの文字マップの一部ではなく、ビットマップレイヤに描画的にレンダリングされます。他の描写コマンドと同様に、出力がペインのビットマップにクリップされます。座標は、ペインのビットマップ領域を越えることも可能ですが、指定された楕円の可視部の

みがウィンドウに描写されます。負の x 座標および y 座標の値が適用できます。

パラメータ この関数のパラメータは以下の通りです。

X *入力:*左上端部の x 座標

Y *入力:*左上端部の y 座標

pstr *入力:*ACSIIZ テキストストリングにポインタ

戻り値 *実行された場合:* 0 以外の値。 *実行されなかった場合:* 0

参照事項 SNSetFont, SNEnumFonts

SNSetFont

プロトタイプ

```
int SNSetFont(
 int          size,
 int          flags,
 char*        pname);
```

内容 この関数は、これ以降の SNTextOut()コマンドに使用されるフォントを設定します。全てのフォントがあらゆるフォントサイズを備えてはいないため、試行錯誤が必要な場合があります。また、ホストシシステムにより書体名が変わる可能性もあり、使用可能なフォントの書体名についての知識が必要です。使用可能な書体名の認識に役立つよう、SNEnumFonts 関数を提供しています。

パラメータ この関数のパラメータは以下の通りです。

size *入力:*ピクセル単位の文字の垂直サイズ

flags *入力:*リザーブ（0 であること）

pname *入力:*フォント名を指定する ACSIIZ テキストストリングにポインタ

戻り値 *実行された場合:* 0 以外の値。 *実行されなかった場合:* 0

参照事項 SNTextOut, SNEnumFonts

SNEnumFonts

プロトタイプ int **SNEnumFonts**(void);

内容 この関数は、使用可能な全ての書体名を列挙し、stdout にそれらの名称を表示します（これは、通常、スクリプトペイン TTY 出力となります）。この関数は、SNSetFont で使用する適切なフォントを選択するために提供されており、診断／テストの用途に限られます。

戻り値 *実行された場合:* 0 以外の値。 *実行されなかった場合:* 0

インデックス

autoexec.eic を使用する 106
C++関数名を再構築するには 20
chdir 130
C 変数ハードウェアブレークポイントの設定 50
ELF ファイルのロード、実行 38
GDI 関数 110
Gekko プロファイリング
  概要 95
getcwd 130
ngcdbg コマンドラインから 107
ngcdbg のコマンドラインの具体例 8
ngcdbg のコマンドライン書式 6
Nintendo GameCube 上でアプリケーションを実行するには 40
ShellExecute 130
SN Systems 関数 108
SNAddMenu 131
SNAddr2Label 135
SNAddr2Line 134
SNBindKey 119
SNClearBmp 138
SNClrBP 126
SNCreateBmp 136
SNDelTypeHandlers 129
SNEllipse 140
SNEnumFonts 142
SNEvaluate 123
SNExit 136
SNGetMemory 123
SNGetMenuInfo 132
SNGetPaneInfo 132
SNGetPaneTitle 133
SNGetTypeDisplay 129
SNGetWindowSize 136
SNHookMessage 117
SNIsRunning 127
SNLine2Addr 134
SNLoadBin 125
SNLoadElf 124
SNPolygon 141
SNRectangle 140
SNRefreshAll 128
SNRun2Addr 135
SNSaveBin 125
SNSetBP 126
SNSetCallback 117
SNSetColor 138
SNSetFont 142
SNSetGraphMode 137
SNSetLine 139
SNSetMemory 124
SNSetPixel 139
SNSetstdout 120
SNSetTypeDisplay 128
SNStart 126
SNStep 133
SNStepOver 134
SNStop 126
SNTextOut 141
SNTxtClrEol 122
SNTxtClrEop 123
SNTxtHome 122
SNTxtSetConsole 121
SNTxtSetPos 120
SNTxtSetTTY 121
SNUnHookMessage 118, 119
SNUpdateWindows 127
target manager
  アップデート 3
  キーボードショートカット 30
TTY バッファのサイズを変更するには 93
TTY ペイン 91
TTY ペインショートカットメニュー 92
TTY 出力を閲覧する 91
TTY 出力を閲覧するには 91
Web を経由した ProDG のアップデート 2
アクセラレータキー 29

アクセラレータキーを解除するには 30
アクセラレータキーを設定するには 29
アクセラレータキー設定を保存／ロードする 30
アセンブリレベルでのハードウェアブレークポイントの設定 51
イミーディエイトモードのステートメント 106
インストールの注意 103
ウィンドウとペイン 10
ウィンドウ間を移動するには 12
ウォッチペイン 82
ウォッチペインショートカットメニュー 84
ウォッチやローカル変数を拡張、または閉じるには 85
ウォッチをすばやく追加するには 84
ウォッチ変数を追加するには 84
カーネルペインショートカットメニュー 89
キーボードショートカット 30
キーボードでテキストを選択するには（ソースペインのみ） 18
キーボードで選択する（ソースペインのみ） 18
ゲームを実行 39
コールスタックペイン 87
コールスタックペインショートカットメニュー 88
コールスタックを閲覧する 87
コンパイル時のブレークポイントの設定 47
ショートカット メニュー項目の追加 114
ショートカットメニューからスクリプトダイアログを実行する 107
スクリプティング
  制限 115
スクリプティング API
  SNBindKey 119
  SNHookMessage 117
  SNSetCallback 117
  SNSetstdout 120
  SNTxtSetConsole 121
  SNTxtSetPos 120
  SNTxtSetTTY 121
  SNUnHookMessage 118, 119
スクリプティング API
  chdir 130
  getcwd 130
  ShellExecute 130
  SNAddmenu 131
  SNAddr2Label 135
  SNAddr2Line 134
  SNClearBmp 138
  SNClrBP 126
  SNCreateBmp 136
  SNDelTypeHandlers 129
  SNEllipse 140
  SNEnumFonts 142
  SNEvaluate 123
  SNExit 136
  SNGetMemory 123
  SNGetMenuInfo 132
  SNGetPaneInfo 132
  SNGetPaneTitle 133
  SNGetTypeDisplay 129
  SNGetWIndowSize 136
  SNIsRunning 127
  SNLine2Addr 134
  SNLoadBin 125
  SNLoadElf 124
  SNPolygon 141
  SNRectangle 140
  SNRefreshAll 128
  SNRun2Addr 135
  SNSaveBin 125
  SNSetBP 126
  SNSetColor 138
  SNSetFont 142
  SNSetGraphMode 137
  SNSetLine 139
  SNSetMemory 124
  SNSetPixel 139
  SNSetTypeDisplay 128
  SNStart 126
  SNStep 133
  SNStepOver 134
  SNStop 126
  SNTextOut 141
  SNTxtClrEol 122
  SNTxtClrEop 123
  SNTxtHome 122
  SNUpdateWindows 127
スクリプト サンプル 109

スクリプト ペイン 105
スクリプト ペインのショートカット メニュー 105
スクリプトの使用方法 104
スクリプトの読み込みと実行 106
スレッド情報を閲覧する 88
スレッド情報を閲覧するには 88
ソースコードの編集 61
ソースと逆アセンブリを同時に閲覧する 68
ソースファイルの配置 63
ソースファイルを配置するには 63
ソースファイル内のテキストを検索するには 60
ソースペイン 55
ソースペイン、逆アセンブリペイン、ミックスモードペインを行き来する 13
ソースペインショートカットメニュー 57
ソースペインでのホバー評価 20
ソースペインの書式の色を設定するには 27
ソースを閲覧する 55
ソース内の検索 59
その他 18
ターゲットとプロセッサステータス 17
ターゲットに接続する 36
ターゲットへの接続 36
ターゲットメモリを保存・ダウンロードする 76
ターゲットを再起動するには 39
ターゲット変数に新しい値を割り当てる 112
タイプキャストとタイプデフ 22
タイマーコールバックの設定 111
テキストファイルにメモリを保存するには 77
テキスト選択およびクリップボード 17
テクニカル サポートとアップグレード 2
デバッガー スクリプトとは 103
デバッガーの起動と同時にスクリプトを読み込む 107
デバッガのペイン更新 16
デバッガの概要 5
デバッガをアップデート 3
デバッガを起動する 5
デバッガ設定を保存/リロードするには 28
デバッグメニュー 40
デバッグ用ビルド 35
テンプレート化レジスタビュー 80
テンプレート文字列の仕様 113
ドラッグ＆ドロップとソースペイン 24
トラブルシューティング 104
バイナリファイルをターゲットメモリにダウンロードするには 77
ブックマークを使用してプログラムソース内を移動するには 60
ブレークポイントの使用 47
ブレークポイントの設定閲覧 48
ブレークポイントプロパティダイアログ 54
ブレークポイントペイン 20, 52
プログラムを 1 ステップずつ実行する 48
プログラム逆アセンブリを閲覧するには 64
プロジェクト設定を保存する 27
プロファイルペイン 95
プロファイルペインショートカットメニュー 96
ペインタイプを変更するには 14
ペインのショートカットメニュー 13
ペインの色とフォント 26
ペインを横方向または縦方向に分割するには 14
ペイン修飾キー 25
ペイン分割バーを移動するには 16
ペイン別ビューを作成する 14
ペイン間でデータを移動する 23
ペイン間でフォーカスを移動するには 15
ポップアップヘルプ を使って数式を評価するには 64
マウスでテキストを選択するには 18
マウスで選択する 17
ミックスモードペインショートカットメニュー 69

メニューオプション 8
メモリアクセス、またはハードウェアブレークポイント 49
メモリペイン 73
メモリペインショートカットメニュー 75
メモリを閲覧変更する 73
ユーザインタフェースを設定する 25
ラベル 22
レジスタペイン 78
レジスタペインショートカットメニュー 79
レジスタを閲覧する 78
レジスタ名 22
ローカルペイン 81
ローカルペインショートカットメニュー 82
ローカル変数を閲覧する 81
ログラムのソースを閲覧するには 55
ワークスペースペインショートカットメニュー 42
ワークスペースペインを使用する 40
他のソースファイルを閲覧するには 64
使用の際の注意事項 18
全てのペインを手動で更新するには 16
全てのペインを自動で更新するには 16
制限 115
各種タイプ用のカスタム表示テンプレートの設定 113
名称の一致作業の優先順位 23
名称の再構築 20
名称の補完 21
変数のモニタリング 82
変数ブラウザを使用してウォッチを追加する 85
数式 19
数式アドレスを入力する 20
数式の評価 111
数式をビルドする 21
新しいデバッガウィンドウを作成する 11
既存のペインを削除するには 15
条件付きブレークポイント 51
条件付きブレークポイントを設定する 52
概要 1, 5
標準 C ライブラリ関数 108
正規表現 62
特定の状況下でのターゲットプロファイリング 97
現在の関数からステップアウトするには 49
留意すべき点 97
複数ユーザ 37
逆アセンブリペイン 65, 67
逆アセンブリペインショートカットメニュー 66
逆アセンブリを閲覧する 64
関数 22
関数ブラウザを使用する 19
ースまたは逆アセンブリにブレークポイントを設定するには 48